BEZPIECZEŃSTWO SYSTEMÓW TELEINFORMATYCZNYCH

Dr Hab. Inż. Mieczysław Jessa, prof. PP

Politechnika Poznańska,
Wydział Elektroniki i Telekomunikacji,
ul. Polanka 3 (pok. 125), 61-131 Poznań, Poland
tel.: +48 61 665 38 54, fax: +48 61 665 38 30
e-mail: mieczyslaw.jessa@put.poznan.pl

Zakres merytoryczny przedmiotu:

Część I:

Zagrożenia dla danych przesyłanych i przetwarzanych w systemie teleinformatycznym

Część II:

Administracyjne metody ochrony danych

Część III:

Kryptograficzne metody ochrony danych

Część III
Kryptograficzne metody ochrony danych

Zakres merytoryczny przedmiotu:

1. System kryptograficzny
   1.1. Rola kryptografii w bezpieczeństwie informacji
   1.2. Terminologia
   1.3. System kryptograficzny symetryczny (z kluczem tajnym)
   1.4. System kryptograficzny asymetryczny (z kluczem publicznym)
   1.5. System kryptograficzny mieszany (hybrydowy)
   1.6. Klasyczne techniki szyfrowania

2. Bezpieczeństwo systemu kryptograficznego
   2.1. Postulat Kerckhoffsa
   2.2. Tajność doskonała
   2.3. Bezpieczeństwo obliczeniowe
   2.4. Bezpieczeństwo „udowadnialne”
   2.5. Standardowe metody łamania szyfrów

Zakres merytoryczny przedmiotu:

3. Szyfry blokowe
   3.1. Konstrukcja szyfru blokowego
   3.2. Tryby pracy (użycia) szyfru blokowego
      3.2.1. Tryb elektronicznej książki kodowej (ECB)
      3.2.2. Tryb wiązania bloków szyfrogramu (CBC)
      3.2.3. Tryb sprzężenia zwrotnego szyfrogramu (CFB)
      3.2.4. Tryb sprzężenia zwrotnego wyjścia (OFB)
      3.2.5. Tryb licznikowy (CTR)
   3.3. Przykłady szyfrów blokowych

4. Szyfry strumieniowe
   4.1. Konstrukcja szyfru strumieniowego
   4.2. One-time-pad
   4.3. Generatory liczb losowych
   4.4. Generatory bezpiecznych liczb pseudolosowych
   4.5. Przykłady szyfrów strumieniowych

Zakres merytoryczny przedmiotu:

5. Szyfry z kluczem publicznym
   5.1. Schemat wymiany kluczy Diffiego-Hellmana
   5.2. Szyfr RSA
   5.3. Szyfr Rabina
   5.4. Szyfr ElGamala
   5.5. Wady i zalety szyfrowania z kluczem tajnym i publicznym

6. Kryptograficzna funkcja skrótu
   6.1. Funkcja skrótu
   6.2. Ataki przeciwko funkcji skrótu
   6.3. Przykłady funkcji skrótu
   6.4. Kody uwierzytelniające wiadomość (MAC)

Zakres merytoryczny przedmiotu:

7. Przykłady użycia metod kryptograficznych w praktyce
    - 7.1. Uwierzytelnianie
    - 7.2. System GSM
    - 7.3. System UMTS
    - 7.4. Protokoły SSL i TLS
    - 7.5. Protokół SSH

Literatura

Podstawowa:

1. A. J. Menezes, P. C. van Oorschot, S. A. Vanstone „Kryptografia stosowana”, WNT, Warszawa 2005.
2. B. Schneier „Kryptografia dla praktyków”, WNT, Warszawa, 2002.
3. J. A. Buchmann „Wprowadzenie do kryptografii”, PWN, 2006.
4. W. Stallings „Kryptografia i bezpieczeństwo sieci komputerowych”, Wyd. V, Helion 2012.

Uzupełniająca:

1. M. Karbowski, Podstawy kryptografii, Helion, 2014.
2. M. Kutyłowski, W-B. Strothmann „Kryptografia, teoria i praktyka zabezpieczania systemów komputerowych”, Read Me, Warszawa, 1999.
3. N. Ferguson, B. Schneier „Kryptografia w praktyce”, Helion, 2004
4. R. J. Sutton, „Bezpieczeństwo telekomunikacji”, WKŁ, Warszawa, 2004.
5. R. Oppliger „Contemporary Cryptography”, Artech House, 2011.
6. Ch. Paar, J. Pelzl „Understanding Cryptography”, Springer, 2010.
7. J. Katz, Y. Lindell „Introduction to Modern Cryptography, CRC, 2015.
8. J. Hoffstein, J. Pipher, J. H. Silverman „An Introduction to Mathematical Cryptography, Springer, 2008.

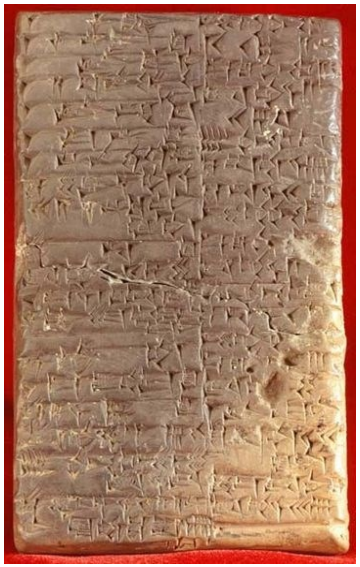
1. System kryptograficzny

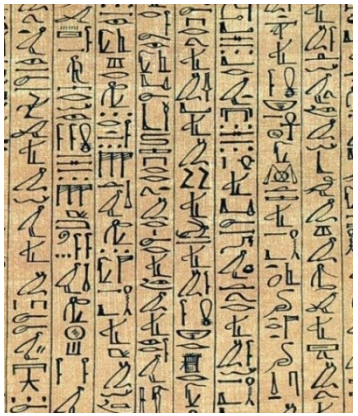
1.1. Rola kryptografii w bezpieczeństwie informacji
1.2. Terminologia
1.3. System kryptograficzny symetryczny (z kluczem tajnym)
1.4. System kryptograficzny asymetryczny (z kluczem publicznym)
1.5. System kryptograficzny mieszany (hybrydowy)
1.6. Klasyczne techniki szyfrowania

Rola kryptografii w bezpieczeństwie informacji

Użycie mechanizmów kryptograficznych dla realizacji bezpiecznego obiegu informacji w instytucji/organizacji jest warunkiem **koniecznym ale nie wystarczającym.** Doświadczenie wskazuje, że najsłabszym ogniwem jest **człowiek**, a najbardziej skutecznym atakiem na system kryptograficzny jest **atak socjotechniczny**. Decydującą rolę w zapewnieniu bezpieczeństwa odgrywają **procedury** wsparte mechanizmami kryptograficznymi, a nie sama kryptografia. Między innymi dlatego obecnie na szeroka skalę wprowadza się do praktyki zapisy norm **PN-ISO/IEC 270xy** oraz **zapisy** rozporządzenia Parlamentu Europejskiego i Rady UE 2016/679 z dnia 27 kwietnia 2016 roku w sprawie ochrony osób fizycznych *w związku z przetwarzaniem danych osobowych i w sprawie swobodnego przepływu takich danych.* Rozporządzenie weszło w życie 17 maja 2016 roku i obligowało wszystkie kraje UE do jego wdrożenia z dniem **25 maja 2018 roku**. Jest to tzw. **GDPR** (*ang*. General Data Protection Regulation) znane w Polsce jako **RODO** (Rozporządzenie o Ochronie Danych Osobowych). GDPR wiąże wszystkich, którzy przetwarzają dane osobowe w związku z prowadzoną **działalnością gospodarczą**.

Rola kryptografii w bezpieczeństwie informacji





- Pismo klinowe i tabliczki gliniane (Mezopotamia 3500 p.n.e)
- Hieroglify (Egipt 4500 p.n.e.)
- Skytale – starożytna Grecja
- Tora (Księga Jeremiasza) i szyfr Atbash (500-600 p.n.e)
- „Szyfry" biblijne (a raczej symbole np. 666)
- „Kamasutra" a kryptografia (między I i VI wiekiem) – posługiwanie się szyfrem w rozmowie jedna z 64 nauk pomocniczych
- Enigma (lata 30-te/40-te)
- DES – pierwszy standard (1975/77)
- Szyfry asymetryczne (Diffie&Hellman; 1976)







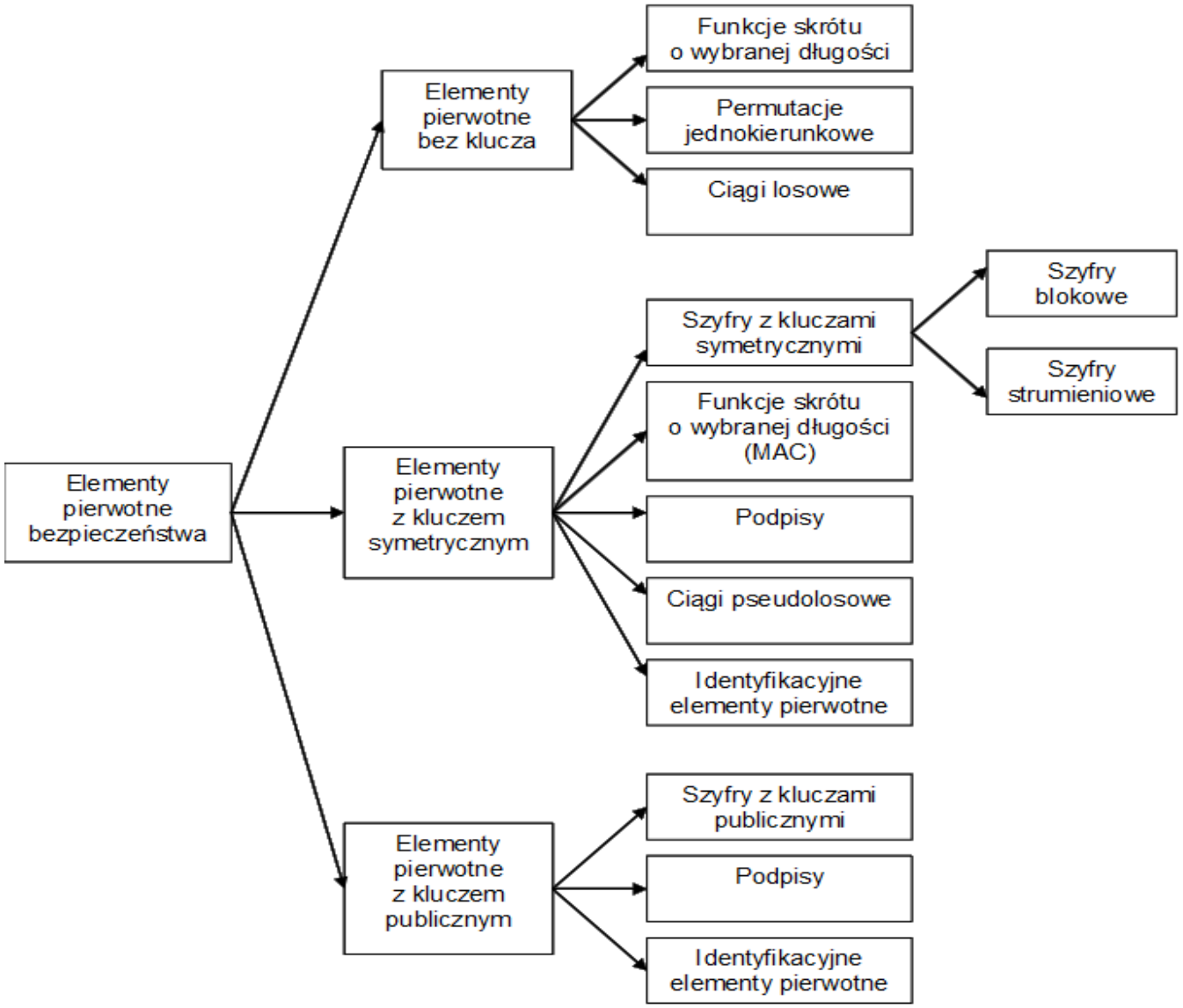
Rola kryptografii w bezpieczeństwie informacji

Z historycznego puntu widzenia kryptografia to nauka o metodach zabezpieczania informacji przed nieuprawnionym dostępem i modyfikacją. Słownik języka polskiego PWN definiuje kryptografię jako „Sztukę przekształcania tekstu pisanego, zrozumiałego dla wszystkich, w tekst zaszyfrowany zrozumiały tylko dla wtajemniczonych znających dany szyfr."

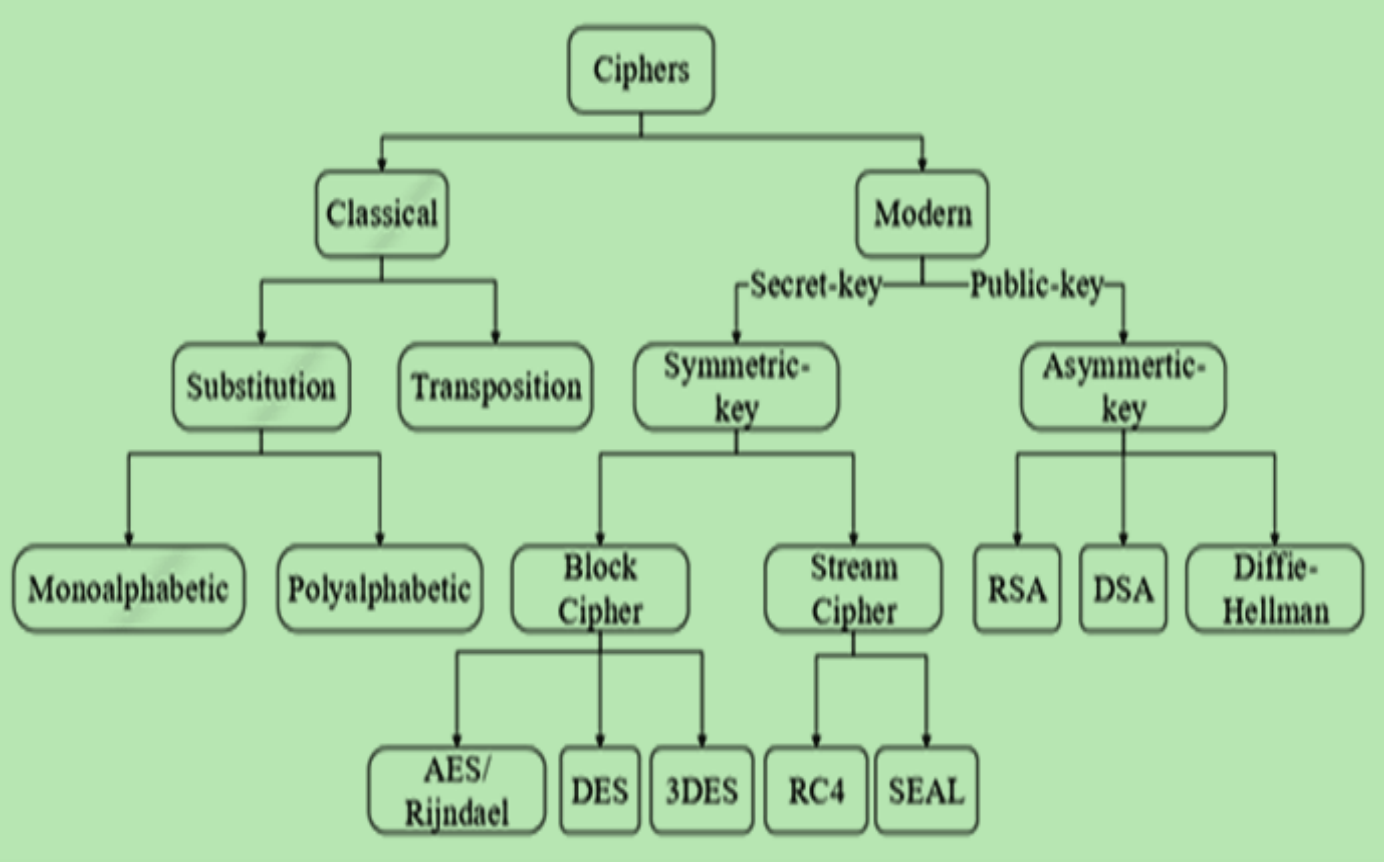
Obecnie kryptografia jest postrzegana znacznie szerzej, **jako nauka o metodach zapewnienia bezpieczeństwa informacji w różnych sytuacjach i z punktu widzenia różnych celów.** Podstawowe cele stosowania kryptografii to:

Rola kryptografii w bezpieczeństwie informacji

Poufność	ochrona przed ujawnieniem informacji osobom innym niż uprawnione
Integralność	zapewnienie, że informacja nie uległa nieuprawnionym modyfikacjom lub wymazaniu
Uwierzytelnianie podmiotu/źródła	potwierdzanie tożsamości podmiotu lub źródła informacji (potwierdzenie, że dany podmiot/źródło jest tym za który się podaje)
Niezaprzeczalność	ochrona przed wyparciem się zobowiązań lub działań, które miały miejsce
Autoryzacja	nadanie podmiotowi formalnego uprawnienia przez podmiot nadrzędny do wykonania czynności, np. dostęp do usługi
Kontrola dostępu	zapewnienie dostępu do zasobów jedynie uprawnionym podmiotom (autoryzowanym)
Podpis cyfrowy	środki służące związaniu informacji z podmiotem
Certyfikacja	formalna aprobata dla informacji wyrażona przez zaufany podmiot
Świadectwo	zweryfikowanie faktu wytworzenia lub istnienia informacji dokonane przez podmiot inny niż twórca
Pokwitowanie	potwierdzenie otrzymania wiadomości
Zatwierdzenie	potwierdzenie dostarczenia usług
Własność	środki zapewniające przekazanie podmiotowi prawa do użycia lub przekazania zasobu innym podmiotom, np. prawa autorskiego
Anonimowość	ukrycie tożsamości podmiotu uczestniczącego w danym procesie
Unieważnienie	wycofanie certyfikacji lub autoryzacji



Podstawowe narzędzia wykorzystywane dla realizacji celów stawianych przed kryptografią - tzw. elementy pierwotne kryptografii.

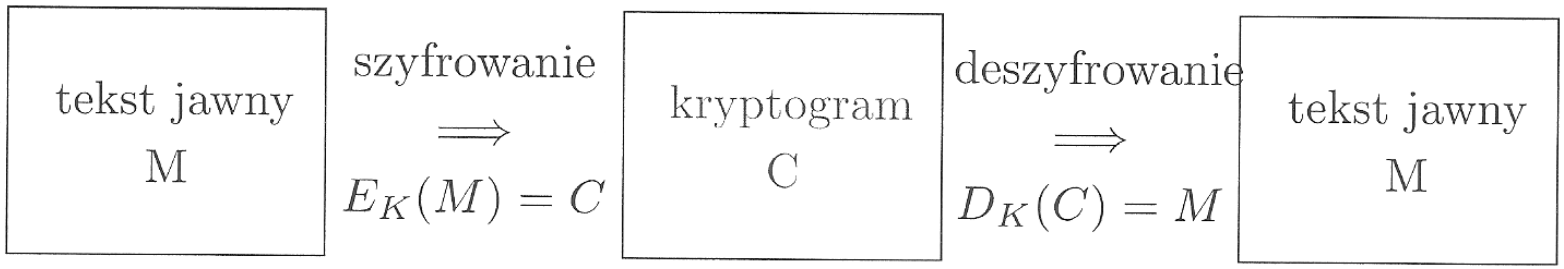


Klasyfikacja szyfrów z przykładami szyfrów używanych obecnie

Terminologia

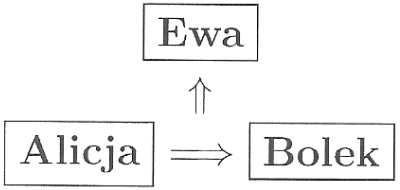
- Krypto grafos - grec. ukryte pismo
- Wiadomość pochodzącą od nadawcy nazywamy tekstem jawnym (*plaintext*) i oznaczamy literą *M*.
- Operację ukrywania, maskowania wiadomości nazywamy szyfrowaniem (*encryption*) i oznaczamy przez *E*.
- Zaszyfrowana wiadomość *C* nazywa się szyfrogramem (*ciphertext*), kryptogramem (*j. polski*) lub szyfrem (*j. polski*) (szyfr wg. Słownik J. pol. PWN to „Rodzaj kodu, zapisu tekstu za pomocą systemu umownych znaków w celu zatajenia treści tekstu przed osobami niepowołanymi").
- Odtworzenie treści wiadomości, czyli szyfrogramu nazywane jest deszyfrowaniem (*decryption*) i oznaczane przez D.
- W procesie szyfrowania tekst wiadomości łączony jest z sekretną informacją nazywaną kluczem. Osoba deszyfrująca wiadomość musi wykazać się znajomością klucza (key) *K*, aby uzyskać dostęp do tekstu jawnego.
- Przepis na przekształcenie tekstu jawnego w szyfrogram nazywamy algorytmem szyfrującym. Z kolei przepis na proces odwrotny nazywamy algorytmem deszyfrującym.

Terminologia



W literaturze przyjęto, że

- Nadawca informacji to Alicja (*Alice*)
- Odbiorca informacji to Bolek (*Bob*)
- Osoba podsłuchująca to Ewa (Eve)



Terminologia

- Proces zabezpieczania wiadomości nazywamy kryptografią (*cryptography*), a zajmują się nią **kryptografowie** (*cryptographers*).
- Kryptoanalitycy (*cryptoanalysts*), to osoby zajmujące się łamaniem, odtajnieniem szyfrów czyli **kryptoanalizą** (*cryptoanalysis*).
- Dziedzina matematyki obejmująca kryptografię i kryptoanalizę to kryptologia (*cryptology*), a zajmujący się nią ludzie to **kryptolodzy** (*cryptologists*).

W ogólności każdy system szyfrujący powinien realizować tzw. funkcję jednokierunkową z zapadką (ang. *one-way trapdoor function*). Polega to na tym, że obliczenie szyfrogramu na podstawie wiadomości jest łatwe, a obliczenie wiadomości na podstawie szyfrogramu jest niemożliwe chyba, że dysponujemy dodatkowym elementem – zapadką. Tym elementem jest tajny klucz znany nadawcy i odbiorcy.

Terminologia

Funkcja jednokierunkowa:

- f: $X \rightarrow Y$
  - Dla każdego $x \in X$ – wartość f(x) wyznacza się w czasie wielomianowym
  - Dla każdego $y \in Y$ – wartość $f^{-1}(y)$ wyznacza się w czasie wykładniczym, nawet jeżeli funkcja f jest znana
- Nie udowodniono, że istnieje chociaż jedna taka funkcja; nie udowodniono także, że taka funkcje nie istnieje,
- Za jednokierunkowe uważa się:
  - Mnożenie dużych liczb pierwszych (funkcja odwrotna to rozkład liczby na czynniki pierwsze),
  - $y = x^2 \bmod n$ (dla liczb 100 cyfrowych) (pierwiastkowanie modulo)
  - $y = a^x \bmod n$ (dla dużych liczb) (logarytmowanie dyskretne, od 1976)
  - Logarytmy dyskretne na krzywych eliptycznych (od 1985)
  - Rozkład na sumy częściowe $y = \sum_i x_i$

Terminologia

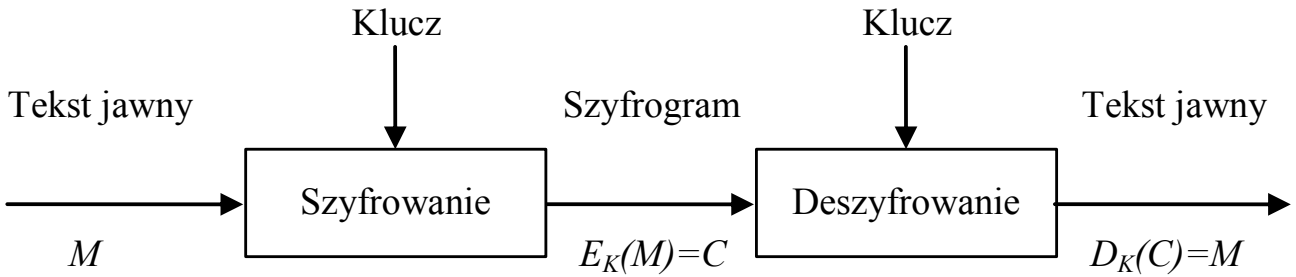
Uwaga:

- Bezpośrednie użycie funkcji jednokierunkowej f(x) może nie zapewnić w bliskiej przyszłości bezpieczeństwa systemom asymetrycznym (komputery kwantowe).
- Publiczne ostrzeżenie NSA (*ang.* National Security Agency, USA) z 25.10.2015 roku przed wdrażaniem systemów kryptograficznych opartych na krzywych eliptycznych (przestarzałe w dobie komputerów kwantowych).
- NSA deklaruje pomoc firmom (USA) w przejściu na techniki kryptograficzne odporne na ataki za pomocą komputerów kwantowych.
- Systemy symetryczne i wielorundowe z niezależnymi kluczami rund (złożenie wielu funkcji jednokierunkowych, nie zawsze możliwych do zapisania za pomocą jednego wzoru, z niezależnymi kluczami rund) możemy uważać za bezpieczne.

System kryptograficzny symetryczny (z kluczem tajnym)

- W zależności od sposobu użycia klucza do szyfrowania i deszyfrowania algorytmy dzielimy na **symetryczne** i **asymetryczne**.
- W algorytmach symetrycznych (*symmetric algorithms*) klucz szyfrujący jest wyznaczany z klucza deszyfrującego i odwrotnie. Bardzo często zdarza się, że klucze szyfrujące i deszyfrujące są takie same.
- Algorytmy symetryczne są często nazywane algorytmami z kluczem tajnym (*secret-key algorithms*), algorytmami z pojedynczym kluczem (*single-key algorithms*) lub z jednym kluczem (*one-key algorithms*).
- Wymagają one wcześniejszego uzgodnienia klucza między nadawcą, a odbiorcą w celu prawidłowego odczytania zakodowanej informacji (wiadomości). Uzgodnienie takie powinno nastąpić przed rozpoczęciem przesyłania wiadomości. Klucz tajny, jak sama nazwa wskazuje, musi być utrzymany w tajemnicy przed osobami „trzecimi".

System kryptograficzny symetryczny (z kluczem tajnym)



$$E_K(M) = C$$
$$D_K(C) = M$$

Działanie algorytmu symetrycznego:

- Nadawca i odbiorca uzgadniają algorytm i klucz, jakich będą używać.
- Nadawca szyfruje wiadomość używając uzgodnionego algorytmu i klucza uzyskując wiadomość zaszyfrowaną (kryptogram).
- Nadawca przesyła zaszyfrowaną wiadomość do odbiorcy.
- Odbiorca deszyfruje otrzymaną wiadomość za pomocą tego samego algorytmu i klucza uzyskując tekst jawny.

System kryptograficzny symetryczny (z kluczem tajnym)

- Algorytmy **symetryczne** można podzielić w zależności od sposobu przetwarzania informacji na:
  - strumieniowe (potokowe) (*stream algorithms*), często nazywane **szyframi strumieniowymi**
  - blokowe (*block algorithms*), nazywane **szyframi blokowymi**.
- W szyfrach strumieniowych przetwarzaną jednostką informacji jest 1 bit lub bajt; szyfrowaniu podlega każdy znak tekstu jawnego osobno, na przykład z użyciem funkcji XOR.
- W szyfrach blokowych szyfrowane są niewielkie bloki danych (znacznie mniejsze od typowej wiadomości), współcześnie jest to najczęściej 128 albo 256 bitów, choć do niedawna przeważały 64-bitowe bloki.
- Dzielenie wiadomości na bloki pozwala na zastosowanie znacznie mniejszych kluczy, zwykle od 128 do 256 bitów. Typowy szyfr blokowy składa się z kilkunastu dość prostych tzw. **rund** przekształcających blok.

System kryptograficzny symetryczny (z kluczem tajnym)

- Operacje używane w szyfrach symetrycznych są zwykle proste, ale pochodzą z „różnych światów", np. używa się dodawania, XOR, przesunięć cyklicznych, mnożenia modulo liczb pierwszych itd.
- Już kilka rund takich operacji zupełnie zaburza jakikolwiek porządek i jest bardzo trudne do analizowania.
- Ponieważ szyfr blokowy szyfruje jedynie niewielką ilość informacji, używane są różne tryby szyfrowania, które umożliwiają szyfrowanie większych wiadomości.

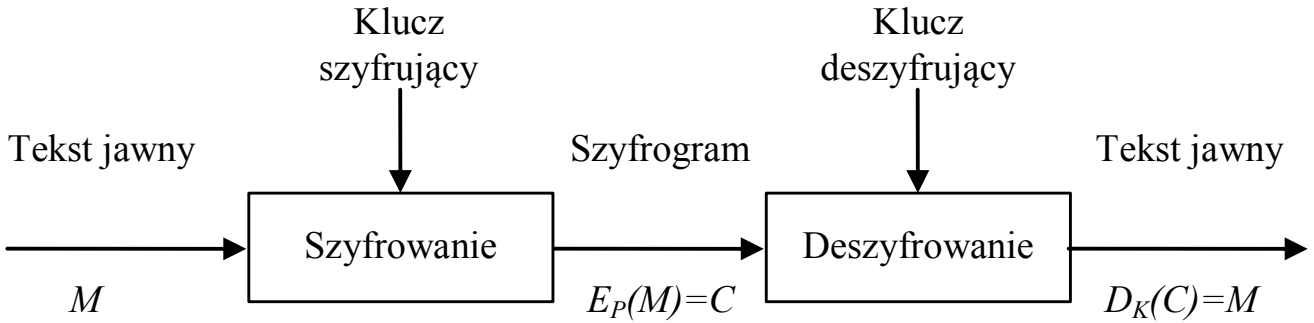
Podstawowe wady szyfrów symetrycznych:

- Klucz musi być przekazywany w sposób tajny.
- Jeżeli każda para korespondujących w sieci dysponuje własnym kluczem, to liczba kluczy szybko rośnie wraz z liczbą kontaktów utrzymywanych przez daną osobę.
- Jeżeli Ewa zdobędzie klucz może deszyfrować wszystko i podszywać się pod Alicję, która o tym nie wie.

System kryptograficzny asymetryczny (z kluczem publicznym)

- Systemy z kluczem publicznym wykorzystują dwa rodzaje kluczy, inny do szyfrowania i deszyfrowania wiadomości. Klucz deszyfrujący nie może być wyznaczony (w rozsądnym czasie) z klucza szyfrującego.
- Algorytmy asymetryczne są nazywane algorytmami **z kluczem publicznym**, ze względu na możliwość ujawnienia klucza szyfrującego, nazywanego kluczem publicznym (*public key*).
- Klucz publiczny jest rozpowszechniany i służy do szyfrowania wiadomości. Wiadomość zaszyfrowana kluczem publicznym może być odszyfrowana tylko przy użyciu odpowiadającego mu klucza prywatnego (*private key*), który jest znany tylko i wyłącznie właścicielowi klucza publicznego.
- Klucz **prywatny** jest czasami nazywany kluczem tajnym, lecz aby nie doszło do niejednoznaczności z systemami symetrycznymi należy zwrócić uwagę podczas używania tego określenia, z jakim systemem mamy do czynienia.

System kryptograficzny asymetryczny (z kluczem publicznym)



$$E_P(M) = C$$
$$D_K(C) = M$$

$P=f(K)$, f –"jednokierunkowa"

Działanie algorytmu asymetrycznego:

- Nadawca i odbiorca uzgadniają algorytm z kluczem publicznym, jaki będą używać.
- Odbiorca przesyła do nadawcy (w sposób jawny) klucz publiczny.
- Nadawca szyfruje wiadomość używając uzgodnionego algorytmu i publicznego klucza odbiorcy uzyskując wiadomość zaszyfrowaną (kryptogram).
- Nadawca przesyła zaszyfrowaną wiadomość do odbiorcy.
- Odbiorca deszyfruje otrzymaną wiadomość wykorzystując klucz prywatny, uzyskując tekst jawny.

System kryptograficzny asymetryczny (z kluczem publicznym)

- W przypadku algorytmów asymetrycznych **nie występuje problem związany z bezpieczną wymianą klucza**, tak jak ma to miejsce w przypadku algorytmów symetrycznych.
- Alicja nie musi przechowywać w sposób bezpieczny wielu kluczy, gdy koresponduje w sieci z wieloma.
- Klucze publiczne mogą być przechowywane na publicznie dostępnym serwerze.

Wady algorytmów asymetrycznych:

- **Znacznie wolniejsze** działanie systemów asymetrycznych w porównaniu z systemami symetrycznymi.
- Brak odporności na atak typu „man in the middle” .

System kryptograficzny mieszany (hybrydowy)

1. Bolek wysyła do Alicji swój klucz publiczny
2. Alicja generuje losowy klucz K dla obecnej sesji, szyfruje go kluczem publicznym Bolka i wysyła kryptogram klucza E_B (K) do Bolka
3. Bolek deszyfruje kryptogram klucza używając swojego klucza prywatnego, D_B (E_B (K))=K, otrzymując klucz K dla obecnej sesji
4. Oboje używają klucza K i symetrycznego algorytmu do szyfrowania i deszyfrowania informacji przesyłanych w czasie tej sesji.

System kryptograficzny mieszany (hybrydowy)

- System hybrydowy jest znacząco szybszy od systemu asymetrycznego, co ma istotne znaczenie przy przesyłaniu dużej ilości danych.
- Klucz K (liczba losowa) jest używany tylko dla danej sesji łączności. Jeżeli Ewa zdobędzie ten klucz, to ma dostęp do danych tylko jednej sesji.
- System nadal nie jest odporny na atak typu „man in the middle”.

Klasyczne techniki szyfrowania

W ogólności szyfry z kluczem tajnym możemy podzielić na:

- Podstawieniowe jednoalfabetowe (monoalfabetyczne) – szyfrowanie polega na zastąpieniu znaku tekstu jawnego innym znakiem,
- Podstawieniowe wieloalfabetowe (polialfabetyczne) – tutaj również zastępujemy znak tekstu jawnego innym znakiem ale uzależniamy wynik od pozycji znaku w tekście.
- Wieloznakowe (wieloliterowe) – zastępujemy sąsiednie *k* znaków tekstu jawnego, *k*>1 innymi *k* znakami w oparciu o różne reguły, np. geometryczne.
- Tanspozycyjne (przestawieniowe) – przestawianie bloku znaków tekstu w oparciu o różne reguły, np. zamiana wierszy na kolumny.
- Iloczynowe, nazywane także kaskadowymi – produkt złożenia dwóch funkcji, np. kaskadowego użycia dwóch algorytmów szyfrujących.

Szyfry jednoalfabetowe

Szyfrowanie polega na zastąpieniu znaku tekstu jawnego innym znakiem. Przykładowo, literę tekstu jawnego zastępujemy literą odległą od danej litery o ustaloną liczbę pozycji:

$$C=E(M)=(M+K) \mod 26$$

$$M=D(C)=(C-K) \mod 26$$

W szyfrze Cezara K=3.

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	R	S	T	U	V	W	X	Y	Z
D	E	F	G	H	I	J	K	L	M	N	O	P	R	S	T	U	V	W	X	Y	Z	A	B	C

tekst jawny $\Longrightarrow$ KRYPTOGRAFIA

kryptogram $\Longrightarrow$ NUBTWSJUDILD

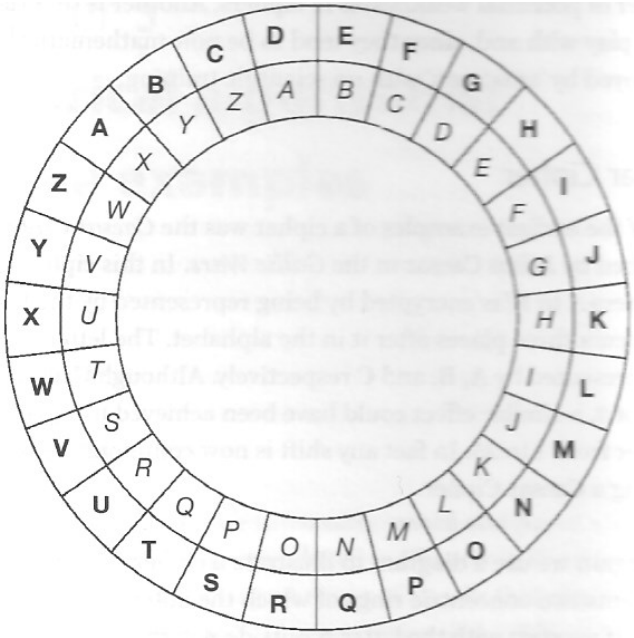
Szyfry jednoalfabetowe

Operację szyfrowania możemy wykonać przyporządkowując literom liczby

A	B	C	D	E	F	G	H	I	J	K	L	M
0	1	2	3	4	5	6	7	8	9	10	11	12

N	O	P	Q	R	S	T	U	V	W	X	Y	Z
13	14	15	16	17	18	19	20	21	22	23	24	25

lub za pomocą specjalnej tablicy („maszyny Cezara”) wykonanej dla konkretnej wartości K



Szyfry jednoalfabetowe

Uogólnieniem szyfru Cezara (tylko 25 kluczy) jest szyfr z kluczem będącym permutacją 26 znaków $(26! > 4 \cdot 10^{26})$.

a b c d e f g h i j k l m n o p q r s t u v w x y z

d e f g h i j k l m n o p q r s t u v w x y z a b c

Tekst jawny: a l a m a k o t a

Tekst zaszyfrowany: d o d p d n r w d

Szyfry wieloalfabetowe

W szyfrach wieloalfabetowych zastępujemy znak tekstu jawnego innym znakiem ale uzależniamy wynik od pozycji znaku w tekście. Przykładem jest szyfr Vigenere'a z 1586 roku (jeden alfabet wejściowy, wiele wyjściowych).

Tablica Vigenere'a

litery tekstu jawnego

litery klucza

	a	b	c	d	e	...	x	y	z
a	b	c	d	e	f	...	y	z	a
b	c	d	e	f	g	...	z	a	b
c	d	e	f	g	h	...	a	b	c
d	e	f	g	h	i	...	b	c	d
e	f	g	h	i	j	...	c	d	e
.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.
x	y	z	a	b	c	...	t	u	w
y	z	a	b	c	d	...	u	w	x
z	a	b	c	d	e	...	w	x	y

Przykład:

k l u c z k l u c z

c o t o z n a c z y

M Z N O Y X L W B X

litery tekstu jawnego

litery klucza

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	R	S	T	U	V	W	X	Y	Z
B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	R	S	T	U	V	W	X	Y	Z	A
C	D	E	F	G	H	I	J	K	L	M	N	O	P	R	S	T	U	V	W	X	Y	Z	A	B
D	E	F	G	H	I	J	K	L	M	N	O	P	R	S	T	U	V	W	X	Y	Z	A	B	C
E	F	G	H	I	J	K	L	M	N	O	P	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	G	H	I	J	K	L	M	N	O	P	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	H	I	J	K	L	M	N	O	P	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	I	J	K	L	M	N	O	P	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	J	K	L	M	N	O	P	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	K	L	M	N	O	P	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	L	M	N	O	P	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	M	N	O	P	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	N	O	P	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	O	P	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	P	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	R
T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	R	S
U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	R	S	T
V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	R	S	T	U
W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	R	S	T	U	V
X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	R	S	T	U	V	W
Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	R	S	T	U	V	W	X
Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	R	S	T	U	V	W	X	Y

Przykład:

klucz $\Longrightarrow$ S Z Y M P A N S S Z Y M

tekst $\Longrightarrow$ K R Y P T O G R A F I A

krypt. $\Longrightarrow$ C P W C I O U I S E G M

Uwaga:

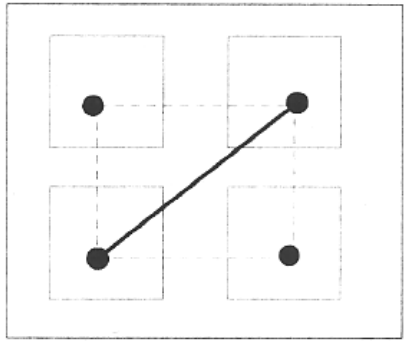
Istnieją dwie wersje tablicy Vignenere'a – z literą Q i bez litery Q.

Szyfry wieloznakowe

W szyfrze wieloznakowym zastępujemy sąsiednie *k* znaków tekstu jawnego, gdzie *k*>1, innymi *k* znakami w oparciu o **różne reguły**, np. geometryczne. Wykorzystuje 25 znaków alfabetu (litera J jest zastąpiona literą I). Kluczem jest układ znaków w tablicy 5x5 wytworzony losowo (przestrzeń klucza 25!). Szyfrujemy bloki liter o długości *k*=2.

Przypuśćmy, że chcemy zaszyfrować litery O i T znajdujące się obok siebie w tekście jawnym. Wynikiem szyfrowania są litery B i Q.

K	I	N	G	D
O	M	A	B	C
F	F	H	L	P
Q	R	S	T	U
V	W	X	Y	Z



Najbardziej znanym przykładem szyfru wieloznakowego jest szyfr Charlsa Playfaira z 1854 wieku. Był on używany aż do roku 1929.

Szyfry tanspozycyjne

Ten rodzaj szyfru polega na przestawieniu bloku znaków tekstu w oparciu o różne reguły, np. zamiana kolumn na wiersze, wierszy na kolumny itp. Najprostszym szyfrem transpozycyjnym jest tzw. *technika płota*.

c r b s j t o

o o i z u r

Komunikat zaszyfrowany: CRBSJTOOOIZUR

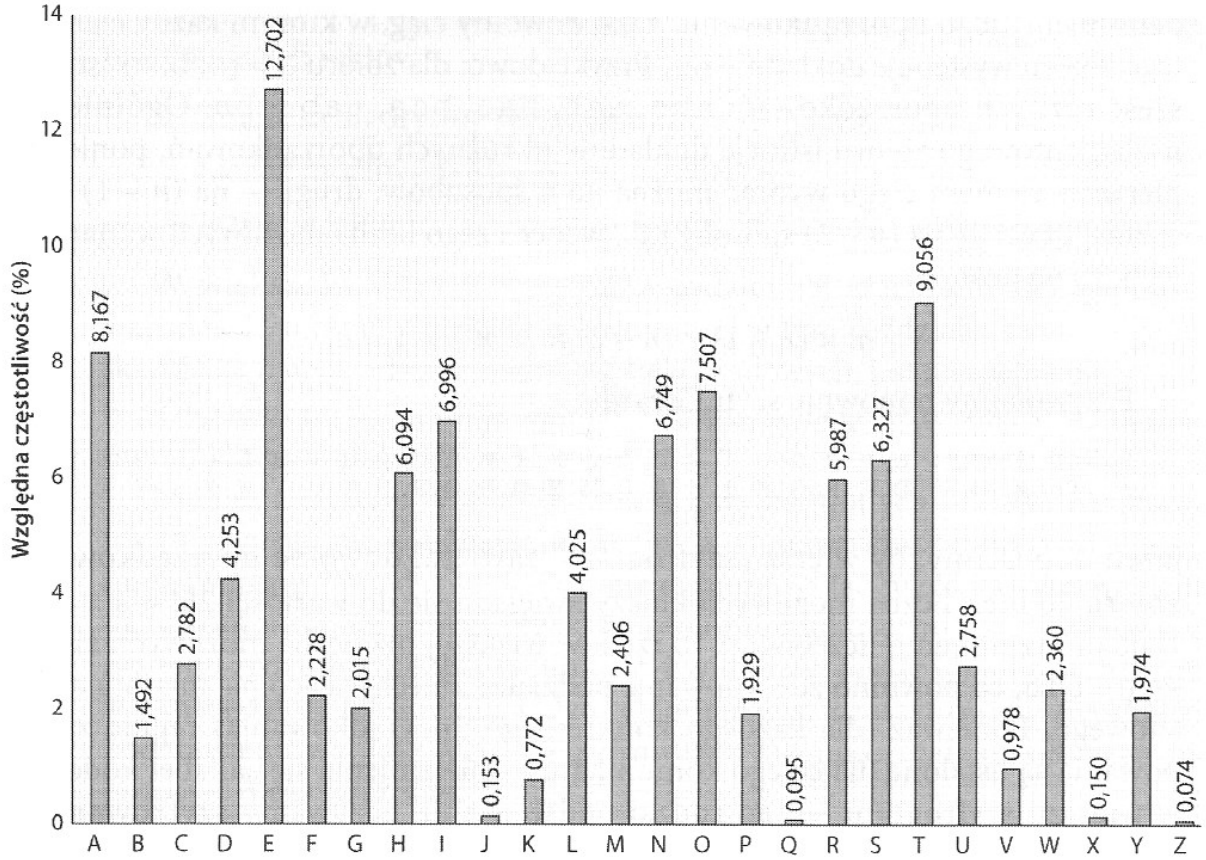
Bardziej skomplikowany szyfr transpozycyjny wykorzystuje zmienną kolejność odczytywania kolumn wyznaczoną przez klucz.

Tekst jawny: nigdzie nie jest lepiej

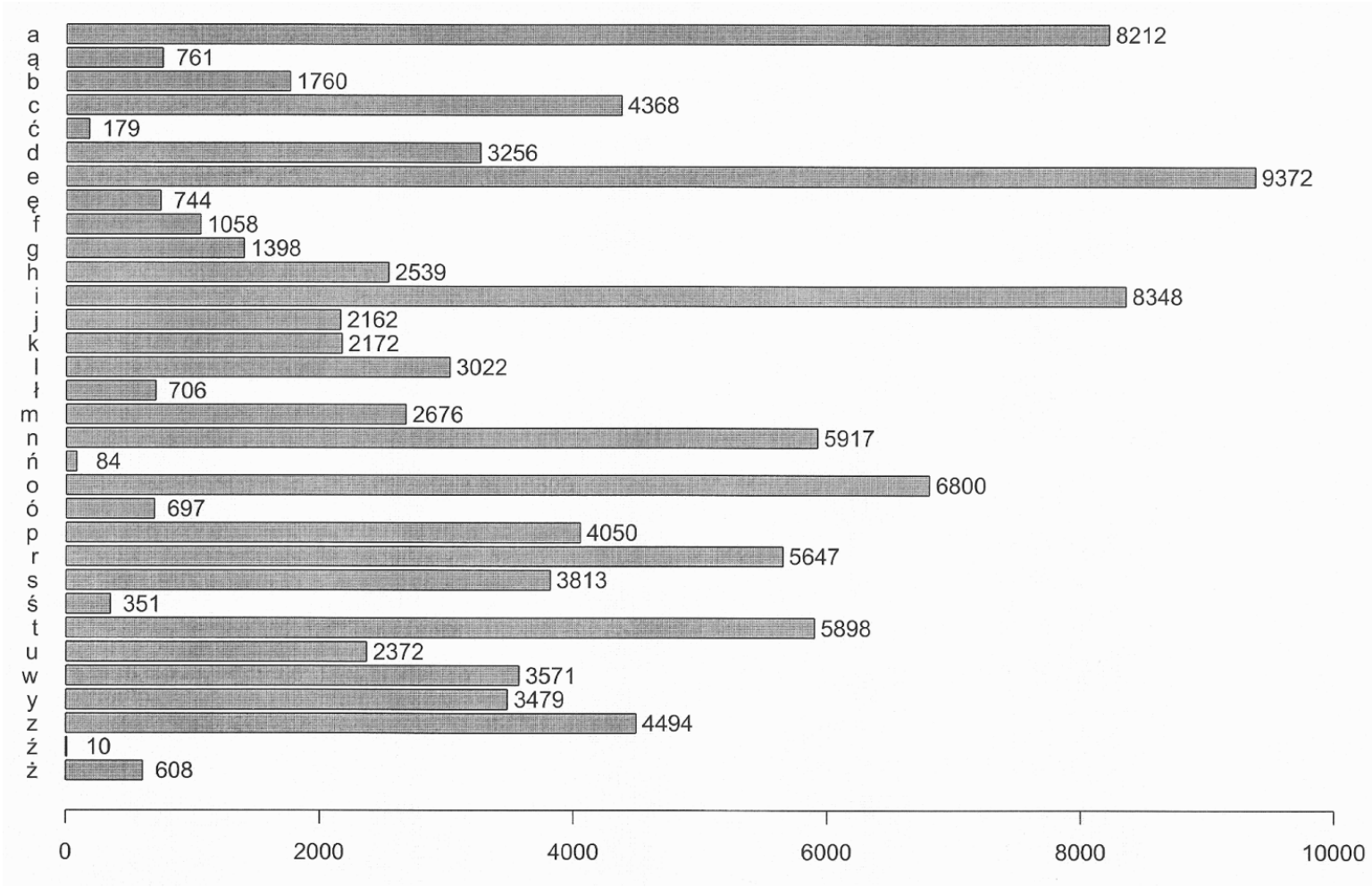
Klucz	2	4	1	3
Tekst jawny	n	i	g	d
	z	i	e	n
	i	e	j	e
	s	t	l	e
	p	i	e	j

Tekst zaszyfrowany:
GEJLENZISPDNEEJIIETI

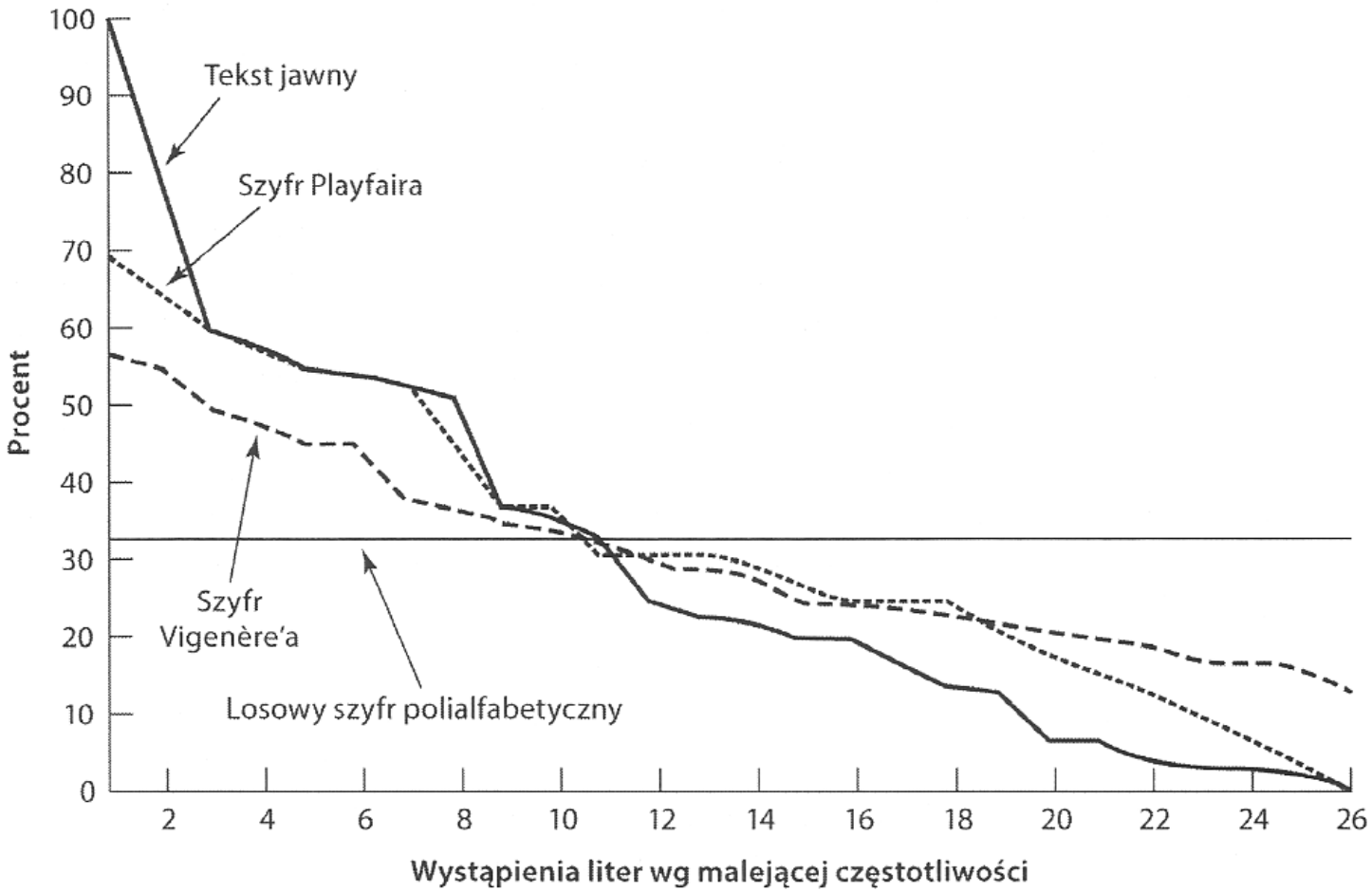
Podstawową wadą prezentowanych szyfrów jest brak ukrywania statystyki języka w szyfrogramie. Znając statystykę liter języka oraz statystykę liter szyfrogramu możemy z dużym prawdopodobieństwem odtworzyć znaczącą część liter szyfrogramu.



Względna częstotliwość występowania liter w języku angielskim



Liczby liter w pracy doktorskiej napisanej w języku polskim (na 100000 znaków)



Względna częstotliwość występowania liter w tekście jawnym i szyfrogramie (język angielski)

Szyfry iloczynowe

W szyfrach iloczynowych stosujemy złożenie (kaskadową realizację) tej samej lub różnych funkcji. Celem jest ukrycie statystyki języka oraz osiągnięcie odpowiednio silnego mieszania. Przykłady to:

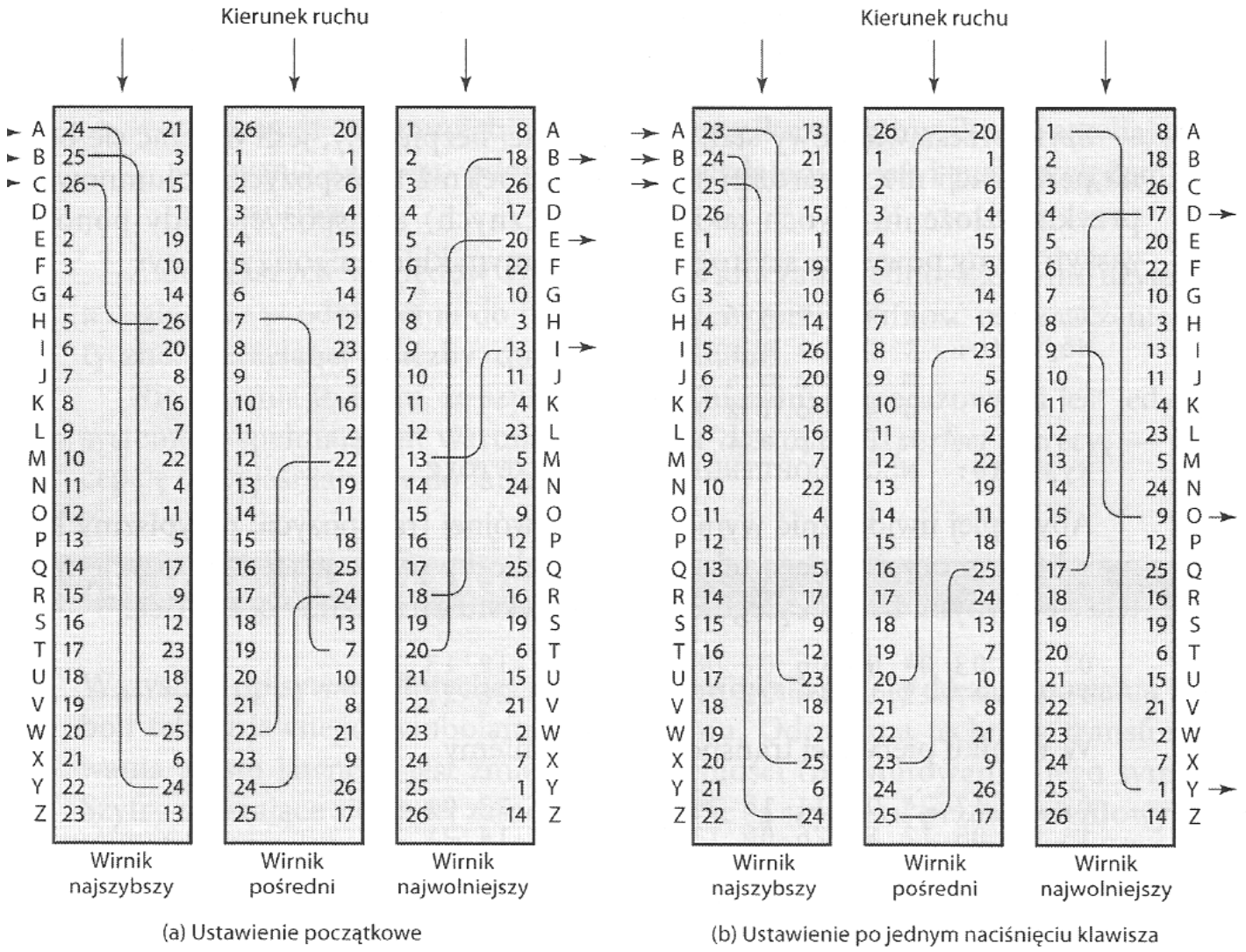
- Enigma
- Lucipher
- DES i 3DES
- IDEA
- Blowfish
- RC5, RC6
- i wiele innych

Szyfry iloczynowe

Jeszcze w wieku XIX zauważono, że wielokrotne stosowanie tej samej funkcji może znacząco utrudnić kryptoanalizę. Takie podejście doprowadziło do koncepcji szyfru iloczynowego oraz stworzyło pojęcia rundy szyfrowania.

W szyfrach iloczynowych stosujemy złożenie (kaskadową realizację) **tej samej lub różnych funkcji**. Celem jest ukrycie statystyki języka oraz osiągnięcie odpowiednio silnego mieszania. Zanim wynaleziono szyfry dla maszyn cyfrowych, idea szyfru iloczynowego była realizowana przez klasę urządzeń, **nazywanych maszynami wirnikowymi**.

Maszyna wirnikowa składa się z kilku niezależnie obracających się cylindrów połączonych elektrycznie. Każdy z cylindrów realizuje połączenie 26 styków wejściowych z 26 stykami wyjściowymi.



Maszyna wirnikowa z trzema cylindrami

Enigma

Automatyzację procesu szyfrowania za pomocą cylindrów zawdzięczamy trzem osobom. Są to:

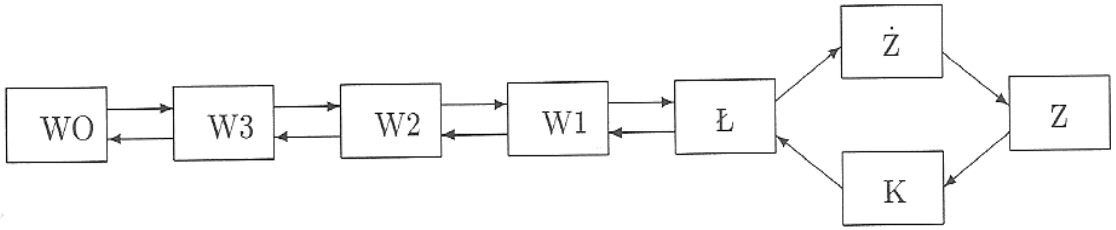
- Eduard Hugo Herben (USA) – dwie połączone elektrycznie maszyny do pisania – kable przebiegają przez wirniki (max 5); 26^5 możliwych połączeń.
- Hugo Aleksander Koch (NL) – maszyna wirnikowa realizująca szyfr Vignere'a, podobna konstrukcja.
- Artur Scherbius (D) – niezależny pomysł; jako pierwszy w 1918 roku zaproponował armii użycie swojego wynalazku; w ofercie handlowej od 1918 roku Scherbius & Ritter.

Lata 20-te to zastosowania w/w maszyn głównie w handlu. Lata 30-te powszechne zastosowanie urządzenia Scherbiusa w armii niemieckiej.
1931 – przekazanie przez Francuzów polskiemu Biuru Szyfrów, utworzonemu w 1930 roku, planów budowy Enigmy – maszyny szyfrującej używanej w armii niemieckiej, zdaniem wywiadu francuskiego „nie do złamania" w realnym czasie.

Enigma (zagadka) rozwiązuje dwa problemy:

- ✓ automatyzuje proces szyfrowania i deszyfrowania,
- ✓ dzięki wielokrotnemu szyfrowaniu ukrywa statystykę języka (szyfr Vigenere'a złamano w wieku XVIII).

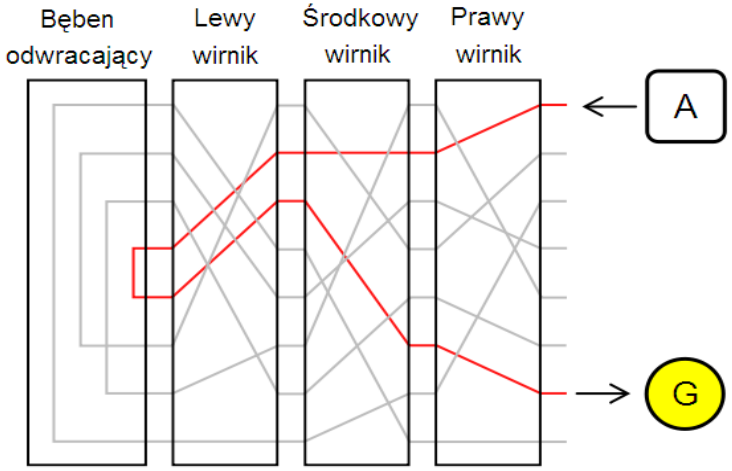
Enigma

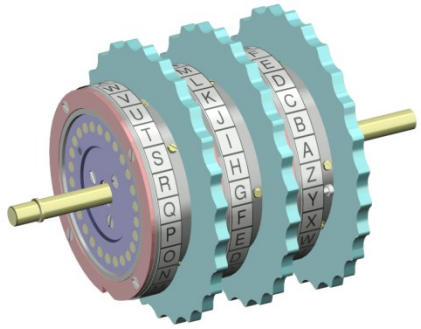




Podstawowe bloki funkcjonalne Enigmy: **Z**-zasilanie, **Ż**-wyświetlacz (lampka podświetlająca wybraną literę i lampka litery kodującej), **K**-klawiatura, **Ł**-łącznica, **W1**, **W2**, **W3** – wirniki (może ich być więcej, w końcowej fazie wojny było 8), **WO** – walec odwracający umożliwiający szyfrowanie i deszyfrowanie na tej samej maszynie.

KLUCZ: kolejność wirników + początkowe ustawienia wirników + połączenia na łącznicy + unikalny dla każdej wiadomości klucz wiadomości (liczba liter równa liczbie wirników).





Enigma

Na liczbę możliwych kluczy składa się:

- Liczba możliwych połączeń łącznicy.
- Liczba znaków na jednym wirniku.
- Liczba niezależnych wirników użytych w maszynie.
- Liczba wirników możliwych do wyboru.
- Liczba możliwych kluczy dziennych.

Ogólnie, liczbę kluczy dla 3 wirnikowej maszyny Enigma można oszacować na:

$$\approx 26! \cdot 26^3 \cdot 10 \cdot 3! > 2^{56}$$

Jest to liczba większa od przestrzeni klucza w algorytmie DES

Enigma

Błędy popełnione przez twórców i użytkowników Enigmy:

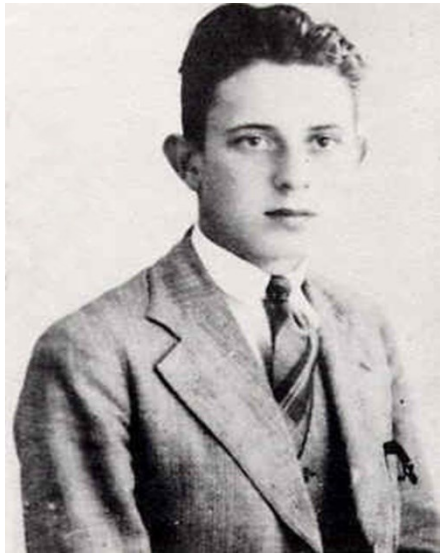
- Powstanie tzw. negatywnego wzorca poprzez dodanie bębna odwracającego (litera alfabetu nie mogła być kodowana w samą siebie).
- Klucz wiadomości przesyłany tym samym kanałem co wiadomość.
- Te same słowa na początku i końcu wiadomości (sformalizowana postać depesz).



Marian Rejewski
16.08.1905 – 13.02.1980



Henryk Zygalski
15.07.1908 – 30.08.1978



Jerzy Różycki
24.07.1909 – 09.01.19942

2. **Bezpieczeństwo systemu kryptograficznego**

2.1. Postulat Kerckhoffsa
2.2. Tajność doskonała
2.3. Bezpieczeństwo obliczeniowe.
2.4. Bezpieczeństwo „udowadnialne”
2.5. Standardowe metody łamania szyfrów

Postulat Kerckhoffsa

Podstawowe zasady tworzenia bezpiecznych systemów szyfrujących zostały sformułowane przez Auguste Kerckhoffsa w książce „La Cryptographie militaire" opublikowanej w 1883 roku.

Kerckhoffs po raz pierwszy dokonał podziału systemu szyfrowania na system ogólny oraz określony klucz. Samo bezpieczeństwo systemu, powinno według niego zależeć wyłącznie od stosowanego klucza (jest to tzw. zasada (postulat) Kerckhoffsa). Zasada ta została powszechnie zaaprobowana w kryptografii. Wynika z niej jawność algorytmów kryptograficznych. Nawet jeśli wróg – tzw. Ewa zna algorytm szyfrowania, to i tak nie powinna odczytać wiadomości bez specjalnego klucza. **Jest to najważniejsza przesłanka współczesnej kryptologii**. Tworzenie szyfru przez ukrywanie zasady jego działania jako sposobu zapewnienia bezpieczeństwa jest obecnie powszechnie odrzucane.

Kerckhoffs zaproponował także stopniowanie metod kryptograficznych zależnie od wagi szyfrowanej wiadomości (obecnie także od tzw. **czasu życia wiadomości** tj. czasu, w którym wiadomość jest istotna dla przeciwnika).
Innym wnioskiem, jaki wysnuł było to, że poziom bezpieczeństwa, jaki oferuje określony kryptosystem może być oszacowany jedynie poprzez kryptoanalizę.

Postulat Kerckhoffsa

Postulat projektowy (Ferguson&Schneier)

Aby uzyskać bezpieczeństwo rzędu *n* bitów, wszystkie wartości używane w kryptografii powinny mieć długość co najmniej *2n* bitów.

Zgodnie z postulatem Kerckhoffsa jedynym tajnym elementem systemu kryptograficznego powinien być klucz. Obecnie wyróżniamy kilka rodzajów klucza:

- **Klucz systemu (*system key*),**
- **Klucz sesji (*session key*),**
- **Klucz bieżący (*running key*),**
- **Litera klucza (*key letter*),**
- **Klucz rundy (*round key*).**

Wymienione rodzaje klucza są wytwarzane na podstawie klucza systemu i ewentualnie dodatkowej jawnej liczby lub jawnych liczb.

Tajność doskonała

Ważną rolę w zdefiniowaniu podstawowych pojęć związanych z oceną bezpieczeństwa systemów kryptograficznych odegrał Claude Elwood Shannon. W dwóch publikacjach: „*A Mathematical Theory of Communication*" i „*Communication Theory of Secrecy Systems*" zajął się strukturą informacji przesyłanej przez kanał. Pierwszy artykuł dał początek teorii informacji. Natomiast drugi dotyczył problemów kryptologii w kategoriach teorii informacji i wprowadzał takie pojęcia jak **tajność doskonała** (ang. *perfect secrecy*) lub inaczej **bezpieczeństwo doskonałe** (ang. *perfect security*), wieloznaczność, czy odległość od jednoznacznego rozwiązania. Model kryptografii Shannona zakłada, że nieprzyjacielski kryptoanalityk ma do dyspozycji **nieograniczony czas i nieskończoną moc obliczeniową**. **Jest on ograniczony jedynie możliwością przeprowadzenia ataku tzw. tekstem zaszyfrowanym** (ang. *ciphertext-only attack*).

Tajność doskonała

Bezpieczeństwo doskonałe, zwane obecnie także bezwarunkowym (*unconditional security*) jest zrealizowane wtedy, jeżeli przechwycony szyfrogram **nie dostarcza kryptoanalitykowi żadnych dodatkowych informacji o tekście jawnym ponad te, które posiadał przed przechwyceniem.**

Definicja
System kryptograficzny nazywamy **bezpiecznym bezwarunkowo**, jeśli nie może być złamany nawet wtedy, gdy dysponujemy nieograniczonymi zasobami.

Warunkiem koniecznym bezpieczeństwa bezwarunkowego systemu kryptograficznego z kluczem tajnym jest to, **aby liczba znaków kodujących klucz szyfrujący wiadomości nie była mniejsza od liczby znaków tej wiadomości**. Obecnie **nie są znane szyfry realizowane fizycznie,** które można uznać za bezpieczne bezwarunkowo. Istnieją natomiast konstrukcje teoretyczne, wprowadzone przed Shannonem, których bezpieczeństwo doskonale można wykazać matematycznie. Przykładem takiego kryptosystemu jest szyfr *one-time-pad*, który stanowi binarną odmianę szyfru Vernama (Gilbert Vernam, 1917).

Tajność doskonała

Modelowi kryptografii Shannona zawdzięczamy dwie metody użyteczne przy tworzeniu szyfrów. Są to metody:

- **dyfuzji**
- **konfuzji.**

Celem stosowania obu metod jest utrudnienie statystycznej analizy szyfrogramu przez wrogiego kryptoanalityka. W metodzie **dyfuzji** zakłada się, że pojedynczy bit (znak) ma wpływ na znaczącą liczbę bitów szyfrogramu (tzw. efekt lawinowy). W metodzie **konfuzji**, przekształcenia są tak skonstruowane, aby utrudnić znalezienie sposobu w jaki statystyka szyfrogramu zależy od statystyki tekstu jawnego. Obu celom służy np. wprowadzenie rund oraz używanie w każdej rundzie operacji mieszających. Wynikiem szyfrowania ciągu bitów wejściowych o dowolnej statystyce i znaczącej długości ma być ciąg, którego nie można odróżnić od ciągu losowego (ang. *distinguishing attack*).

Bezpieczeństwo obliczeniowe

Ponieważ żaden praktycznie realizowalny system kryptograficzny nie jest bezpieczny bezwarunkowo, do określenia poziomu bezpieczeństwa zaczęto używać czasu i kosztu, jaki jest potrzebny do złamania określonego szyfru. Im czas łamania szyfru jest dłuższy lub potrzebujemy większych zasobów do odczytania tekstu jawnego z szyfrogramu, tym poziom bezpieczeństwa oferowany przez ten system jest większy. Wiąże się to z innym pojęciem wprowadzonym przez Shannona – **charakterystyką pracy**. Określa ona przeciętną ilość pracy, jaką musi wykonać kryptoanalityk w celu wyznaczenia wiadomości jawnej na podstawie przechwyconego szyfrogramu.

Definicja

Dowolny system kryptograficzny nazywamy **bezpiecznym obliczeniowo** (ang. *computationally secure*) jeśli najlepszy algorytm łamania tego systemu wymaga co najmniej *N* operacji, gdzie *N* jest bardzo dużą liczbą.

Bezpieczeństwo obliczeniowe

Definicja wprowadza pojęcie bezpieczeństwa obliczeniowego. Zakłada ona, że kryptoanalityk posiada **ograniczone zasoby obliczeniowe**, wyznaczające wartość dla *N*. Jeżeli systematyczna analiza z użyciem dostępnych środków, w „realnym czasie" nie pozwala złamać algorytmu szyfrującego, to system może zostać uznany za bezpieczny obliczeniowo. Ponieważ zasoby kryptoanalityka stale rosną, system bezpieczny obliczeniowo nie może być uznany za bezpieczny w dłuższym okresie czasu.

Uwaga:
Współcześnie wielu autorów posługuje się stwierdzeniem, że określony algorytm szyfrujący jest bezpieczny lub że „dowiedziono bezpieczeństwa szyfru". Takie uproszczenie należy ZAWSZE czytać: „**nie ma podstaw do odrzucenia hipotezy, że określony algorytm szyfrujący jest bezpieczny obliczeniowo"**. Oznacza to, iż wykazano odporność algorytmu na łamanie z użyciem K<N operacji dla wszystkich znanych ataków.

Bezpieczeństwo „udowadnialne”

Odmianą bezpieczeństwa obliczeniowego jest bezpieczeństwo udowadnialne. Istnienie tej kategorii bezpieczeństwa nie jest powszechnie akceptowane. Jej wyróżnienie może być jednak przydatne przy konstruowaniu nowych kryptosystemów.

Definicja
Dowolny system kryptograficzny nazywamy **bezpiecznym „udowadnialnie”**, jeśli bezpieczeństwo tego systemu może być sprowadzone do znanego problemu uważanego za trudny.

Przykładowo, jeżeli bezpieczeństwo pewnego systemu można sprowadzić do problemu logarytmu dyskretnego lub faktoryzacji dużych liczb, to taki system jest bezpieczny „udowadnialnie”. Ponieważ takie bezpieczeństwo jest odmianą bezpieczeństwa obliczeniowego, zatem nie można wykazać bezpieczeństwa takiego systemu. Dowód bezpieczeństwa tego typu jest w istocie redukcją bezpieczeństwa. Bezpieczeństwo systemu zostało zredukowane do **kwestii obliczeniowej** nierozwiązywalności pewnych problemów.

Standardowe metody łamania szyfrów

Z **matematycznego** punktu widzenia celem ataku jest znalezienie klucza używanego do szyfrowania. Z **praktycznego** punktu widzenia celem ataku jest poznanie całości lub chociażby części treści wiadomości zaszyfrowanej. W ogólności metody łamania szyfrów, nazywane także **atakami kryptoanalitycznymi** możemy podzielić na dwie podstawowe grupy:

- metody ogólne – scenariusz ataku nie zależy od właściwości atakowanego systemu kryptograficznego ,
- metody szczególne (kryptoanalitycze) – scenariusz ataku zależy od właściwości atakowanego systemu kryptograficznego, tzn. bazuje na znajomości natury algorytmu i ewentualnie pewnych ogólnych cechach tekstu jawnego bądź par „tekst jawny – szyfrogram”.

Spośród metod ogólnych najbardziej znaną metoda jest tzw. **atak siłowy** (*ang.* brute-force attack). W tej metodzie intruz dokonuje rozszyfrowania szyfrogramu wypróbowując wszystkie możliwe klucze, aż do uzyskania tekstu stwarzającego wrażenia lub pozory wiarygodności.

Standardowe metody łamania szyfrów

Rozmiar klucza	Liczba możliwych kluczy	Czas potrzebny w systemie sprawdzającym jeden klucz w ciągu µs	Czas potrzebny w systemie sprawdzającym milion kluczy w ciągu µs
32 bity	$2^{32} \approx 4{,}3 \times 10^{9}$	2^{31} µs ≈ 35,8 minuty	2,15 milisekundy
56 bitów	$2^{56} \approx 7{,}2 \times 10^{16}$	2^{55} µs ≈ 1142 lata	10,01 godziny
128 bitów	$2^{128} \approx 3{,}4 \times 10^{38}$	2^{127} µs ≈ $5{,}4 \times 10^{24}$ lat	$5{,}4 \times 10^{18}$ lat
168 bitów	$2^{168} \approx 3{,}7 \times 10^{50}$	2^{167} µs ≈ $5{,}9 \times 10^{36}$ lat	$5{,}4 \times 10^{30}$ lat
26 znaków (w dowolnym uporządkowaniu)	$26! \approx 4 \times 10^{26}$	2×10^{26} µs ≈ $6{,}4 \times 10^{12}$ lat	$6{,}4 \times 10^{6}$ lat

Oszacowanie średniego czas łamania szyfru metodą ataku siłowego
(W. Stallings, 2012)

Standardowe metody łamania szyfrów

Do podstawowych ataków kryptoanalitycznych należą:

Rodzaj ataku	Informacja dostępna dla kryptoanalityka
Atak z samym szyfrogramem	• Algorytm szyfrujący • Przechwycony szyfrogram
Atak ze znanym tekstem jawnym	• Algorytm szyfrujący • Przechwycony szyfrogram • Jedna lub więcej par „tekst jawny – szyfrogram" utworzonych przy użyciu tego samego klucza
Atak z wybranym tekstem jawnym	• Algorytm szyfrujący • Przechwycony szyfrogram • Utworzony przez kryptoanalityka tekst jawny wraz z odpowiadającym mu szyfrogramem, utworzonym przy użyciu tajnego klucza
Atak z wybranym szyfrogramem	• Algorytm szyfrujący • Przechwycony szyfrogram • Utworzony przez kryptoanalityka szyfrogram wraz z odpowiadającym mu tekstem jawnym, odtworzonym przy użyciu tajnego klucza
Atak z wybranym tekstem jawnym i wybranym szyfrogramem	• Algorytm szyfrujący • Przechwycony szyfrogram • Utworzony przez kryptoanalityka tekst jawny wraz z odpowiadającym mu szyfrogramem, utworzonym przy użyciu tajnego klucza • Utworzony przez kryptoanalityka szyfrogram wraz z odpowiadającym mu tekstem jawnym, odtworzonym przy użyciu tajnego klucza

PODSTAWY KRYPTOGRAFII

3. Szyfry blokowe.

3.1. Konstrukcja szyfru blokowego.

3.2. Tryby pracy (użycia) szyfru blokowego.

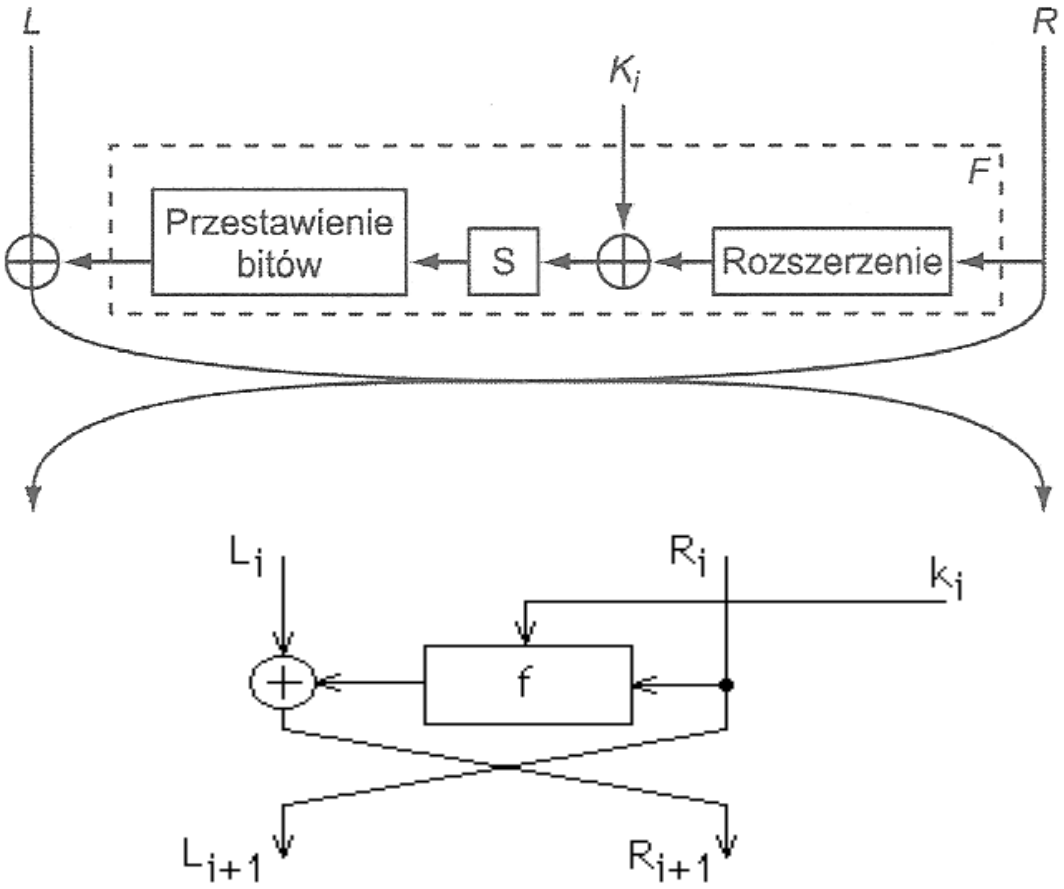
- 3.2.1. Tryb elektronicznej książki kodowej (ECB).
- 3.2.2. Tryb wiązania bloków szyfrogramu (CBC).
- 3.2.3. Tryb sprzężenia zwrotnego szyfrogramu (CFB).
- 3.2.4. Tryb sprzężenia zwrotnego wyjścia (OFB).
- 3.2.5. Tryb licznikowy (CTR)

3.3. Przykłady szyfrów blokowych.

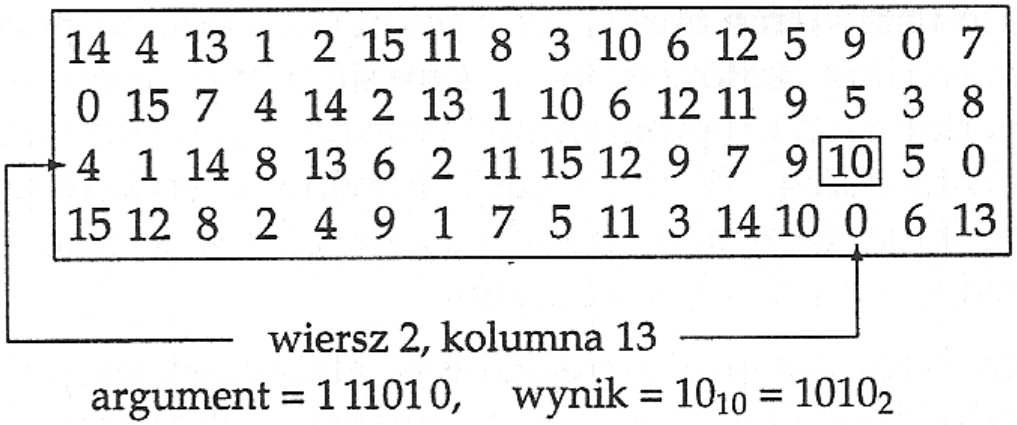
Konstrukcja szyfru blokowego

- Nie istnieje szyfr blokowy idealny do wszystkich zastosowań.
- Istnieje teoretycznie nieskończenie wiele szyfrów blokowych.
- Każdy szyfr musi realizować dyfuzję i konfuzję.
- Ponieważ zadowalającej dyfuzji i konfuzji nie można otrzymać za pomocą jednej lub nawet kilku operacji, powiela się pewien zestaw operacji elementarnych realizujących dyfuzję i konfuzję w możliwie dużym stopniu. Każde takie powielenie nazywamy rundą.
- Danymi wejściowymi do każdej z rund są dane otrzymane w rundzie poprzedniej oraz dodatkowe liczby, określane jako klucze rund. Są to tajne liczby wytworzone na podstawie klucza systemu i ewentualnie dodatkowych liczb dostępnych jawnie.
- Przykładem zestawu operacji elementarnych wykonywanych w jednej rundzie jest sieć Horsta Feistela.

Konstrukcja szyfru blokowego



Konstrukcja szyfru blokowego



S-boks (S-blok, na rysunku S-blok o nazwie S1) definiuje funkcję elementarną, której podstawowym zadaniem jest realizacja w możliwie dużym zakresie dyfuzji i konfuzji. Argumentem S-boksu jest ciąg złożony z 6 bitów a wyjściem ciąg złożony z 4 bitów.

- Pierwszy i ostatni bit S-boksu określa liczbę w systemie dwójkowym wyznaczającą numer *i* wiersza.
- Pozostałe bity (4 środkowe) określają numer *j* kolumny.
- Wyjściem S-boksu jest liczba znajdująca się w *i*-tym wierszu oraz w *j*-tej kolumnie.

Konstrukcja szyfru blokowego

Wybór liczb znajdujących się w danym S-boksie oraz wybór S-boksów **ma fundamentalne znaczenie dla właściwości szyfru**. Dlatego nie był on często podawany do wiadomości publicznej. Możemy natomiast podać kilka reguł, które należy brać pod uwagę.

Kryteria wyboru S-boksów (omawiamy niektóre ważne własności, jakie powinny być spełnione przez dobre S-boksy; każda z nich została sformułowana na zasadzie „jeśli własność nie jest spełniona, to kryptoanaliza może być ułatwiona):

- Każdy wiersz S-boksu zawiera wszystkie liczby całkowite od 0 do 15. Własność ta zapewnia, że sama znajomość wiersza nie daje żadnej informacji o wartości końcowej.
- Funkcja obliczana przez S-boks **nie jest funkcją afiniczną**, tzn. żaden bit wyniku nie da się przedstawić jako kombinację liniową bitów argumentu. Operacje arytmetyczne wykonywane są modulo 2. Gdyby bity wyniku były wyrażalne za pomocą funkcji afinicznych, to kryptoanaliza byłaby znacznie ułatwiona. Po pierwsze, składanie S-boksów nie miałoby sensu - dalej poruszalibyśmy się w obrębie funkcji afinicznych, bo złożenie funkcji afinicznych jest funkcją afiniczną. Po drugie, kryptoanaliza ograniczałaby się do rozwiązywania układów równań liniowych, co nie jest trudne

Konstrukcja szyfru blokowego

- **Zmiana jednego bitu argumentu zmienia co najmniej dwa bity wyniku**. Własność ta ma zapewnić efekt lawinowy w przypadku wielokrotnego zastosowania S-boksów. Jeśli ustalimy wartość jednego bitu argumentu oraz jedną pozycję wyniku, to dla około połowy ustawień pozostałych bitów argumentu na wybranej pozycji wyniku otrzymamy 0. Własność ta gwarantuje, że nawet gdy znamy wartość jednego bitu argumentu, to na dowolnej pozycji wyniku otrzymujemy 0 w przybliżeniu z takimi częstościami, jak w przypadku losowania bitów za pomocą rzutów monetą.
- Należy tak dobrać wartości w tablicy S-boksu, aby jak najbardziej utrudnić tzw. kryptoanalizę różnicową.
- Wskazany jest taki wybór wartości w tablicy S-boksu, by jak najbardziej utrudnione były wszystkie techniki kryptoanalizy, w tym kryptoanaliza liniowa. Ponieważ ta metoda nie była znana w momencie wyboru S-boksów dla standardu DES i 3DES, w związku z tym konstrukcja S-boksów w tych szyfrach zawiera pewne słabości.

Oprócz sformułowania odpowiednich kryteriów, należy znaleźć S-boksy jak najlepiej odpowiadające przyjętym założeniom dla danego szyfru. Nie było i nie jest to łatwe. Nawet poszukiwania prowadzone za pomocą specjalnych programów są bardzo trudne, bowiem liczba potencjalnych S-boksów jest astronomiczna. Próby znalezienia S-boksów o **większych rozmiarach** (to znaczy o większej liczbie bitów na wejściu i wyjściu) poprzez testowanie wszystkich potencjalnych kandydatów **mogą być z tej przyczyny nierealizowalne.**

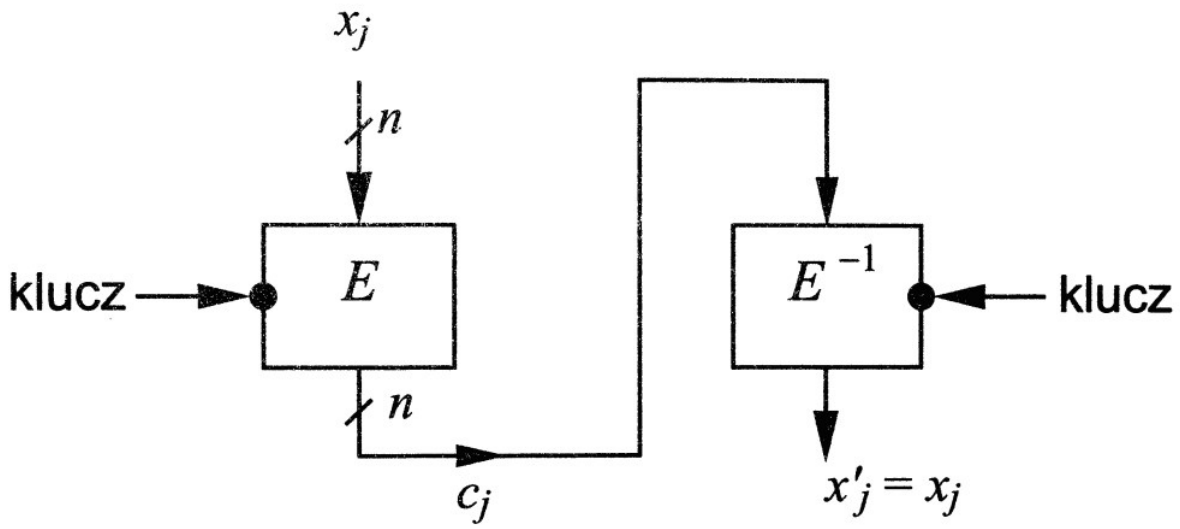
Tryby pracy (użycia) szyfru blokowego

Bezpieczeństwo komunikacji nie zależy wyłącznie od szyfru. Istotny jest także sposób używania szyfru. Podstawowe tryby szyfrowania to:

Tryb	Opis	Typowe zastosowanie
Elektroniczna książka kodowa (ECB)	Każdy *n*-bitowy blok szyfrowany jest niezależnie od pozostałych, przy użyciu tego samego klucza.	• Bezpieczna transmisja pojedynczych obiektów, na przykład kluczy.
Łańcuchowanie bloków szyfrogramu (CBC)	Wejściem dla algorytmu szyfrującego jest suma XOR bieżącego *n*-bitowego bloku tekstu jawnego i poprzedniego *n*-bitowego bloku szyfrogramu.	• Transmisja blokowa ogólnego przeznaczenia. • Uwierzytelnianie.
Sprzężenie zwrotne szyfrogramu (CEB)	Informacja wejściowa przetwarzana jest **bit po bicie**. Poprzedni szyfrogram jest używany w charakterze wejścia do algorytmu szyfrującego, produkującego pseudolosowe wyjście, które zsumowane z bieżącym blokiem tekstu jawnego daje kolejny blok szyfrogramu.	• Transmisja strumieniowa ogólnego przeznaczenia. • Uwierzytelnianie.
Sprzężenie wyjściowe (OFB)	Tryb podobny do CFB, jednakże informacja przetwarzana jest całymi blokami, a wejściem do algorytmu szyfrującego jest wyjście funkcji szyfrującej dla poprzedniego bloku.	• Transmisja strumieniowa przez kanały o wysokim stopniu zakłóceń (na przykład w komunikacji satelitarnej).
Tryb licznikowy (CTR)	Każdy blok teksu jawnego sumowany jest (XOR) z licznikiem o identycznej długości. Licznik ten jest inkrementowany dla kolejnych bloków.	• Transmisja blokowa ogólnego przeznaczenia. • Tryb użyteczny w przypadku zapotrzebowania na dużą szybkość transmisji, np. transmisja ATM, sieci IP.

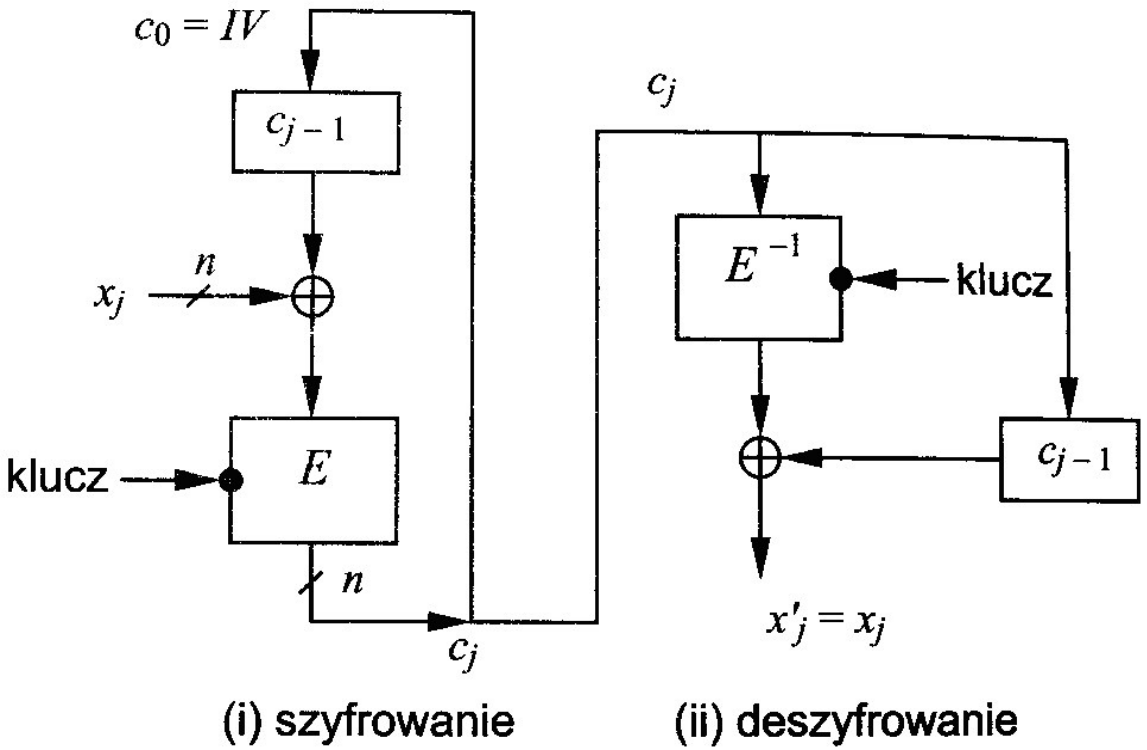
Tryby pracy (użycia) szyfru blokowego

- Tryb elektronicznej książki kodowej ECB (*Electronic Codebook mode*)



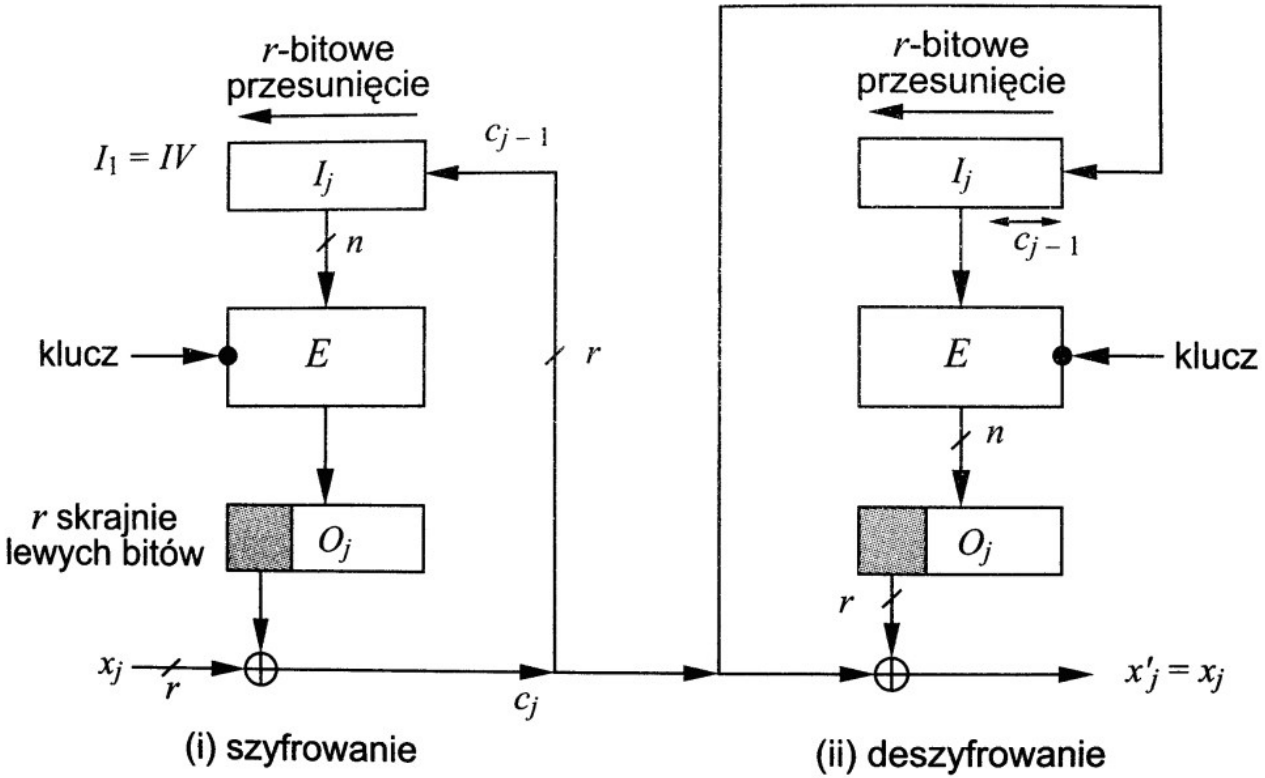
Tryby pracy (użycia) szyfru blokowego

- Tryb wiązania bloków szyfrogramu CBC (*Cipher Block Chaining m*ode)



Tryby pracy (użycia) szyfru blokowego

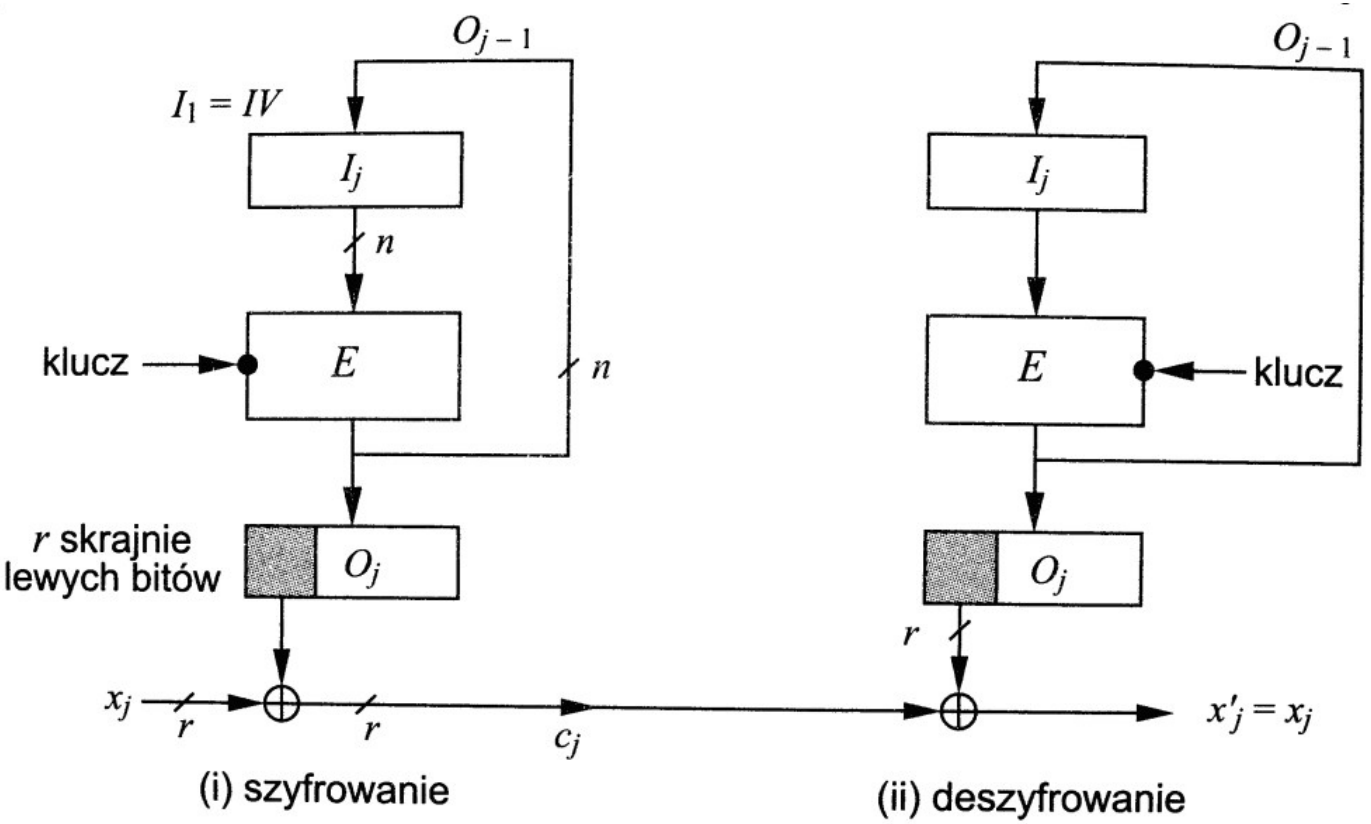
- Tryb sprzężenia zwrotnego szyfrogramu CFB (*Cipher Feedback mode*)



$1 \le r \le n \qquad I_j = 2^r I_{j-1} + c_{j-1} \mod 2^n$

Tryby pracy (użycia) szyfru blokowego

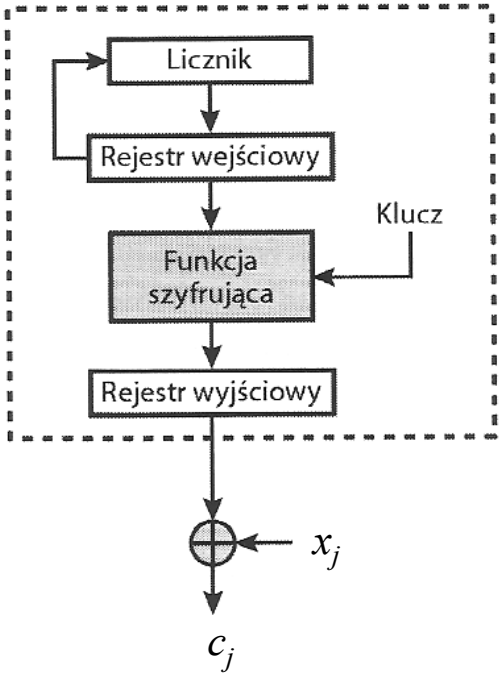
- Tryb sprzężenia zwrotnego wyjścia OFB (*Output Feedback mode*)



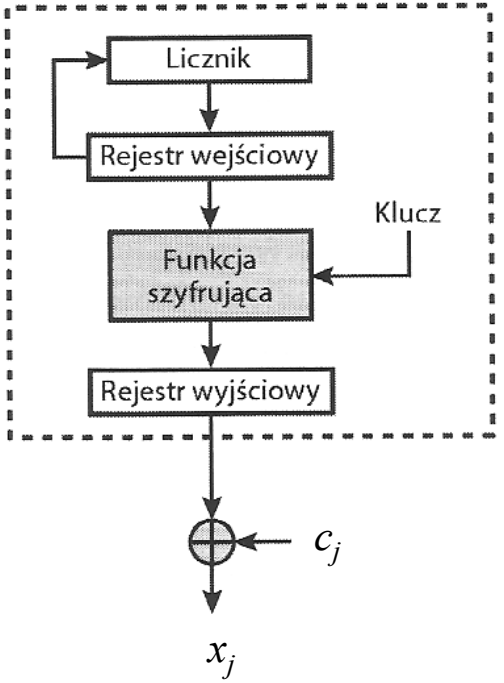
Uwaga: Jeśli ten sam klucz ma być użyty do zaszyfrowania dwóch tekstów otwartych, to należy zmienić wektor początkowy IV.

Tryby pracy (użycia) szyfru blokowego

- Tryb licznikowy CTR (*Counter mode*)



(i) szyfrowanie



(ii) deszyfrowanie

Wartość licznika przyrasta o 1 dla każdego bloku danych, zaczynając od IV. Dla różnych sesji transmisji wartości IV muszą być różne i niepowtarzalne.

Przykłady szyfrów blokowych

- DES (*Data Encryption Standard*)

Algorytm DES jest szeroko używanym amerykańskim standardem szyfrowania danych opisanym w publikacji nr 46 jako Federal Information Processing Standard FIPS 46 datowanej na 15 stycznia 1977 roku. Nowy standard szyfrowania został stworzony przez IBM na podstawie szyfru Lucyfer (*Lucipher*) wywodzącego się od zaprojektowanego w 1973 przez Horsta Feistela szyfru DEMON (skrót od Demonstration). Istniało kilka wersji systemu Lucyfer, najbardziej popularna operowała na blokach 64 bitowych i 128 bitowym kluczu i to z niej wywodzi się algorytm DES. Aby Lucyfer mógł zostać standardem, agencja NSA (*National Security Agency*) z powodów politycznych ograniczyła długość klucza do 64 bitów (w praktyce do 56 bitów, gdyż co ósmy bit jest przeznaczony na bit parzystości). Z powodu słabości tego klucza z biegiem czasu został zastąpiony przez inne algorytmy m.in. przez 3DES, który okazał się znacznie bezpieczniejszy. W **1981 roku DES został przyjęty jako standard do celów cywilnych**.

DES

Podstawowe cechy DES:

- algorytm symetryczny
- pełna jawność algorytmu
- szyfrowanie bloków o długości 64 bity (8 znaków ASCII z bitami parzystości)
- efektywna długość klucza 56 bitów (64 bity minus 8 bitów parzystości
- 16 rund identycznych operacji opisanych tą samą funkcją f, w czasie których dane z prawej połowy są przekształcane z użyciem oddzielnego klucza – klucza rundy K_i
- realizacja koncepcji sieci Feistela – ten sam algorytm używany do szyfrowania oraz deszyfrowani

DES

Łamanie DES:

Liczba możliwych kluczy to 2^{56}.

Średnia liczba bezpiecznych kluczy przy ataku brutalnym (całościowe przeszukiwanie to 2^{54})

Kryptoanaliza różnicowa (możliwość wykonania eksperymentu - przesłanie wiadomości jawnej i odczytania zaszyfrowanej) zmniejsza przestrzeń bezpiecznych kluczy do 2^{47}. Dokonuje się jej przez wprowadzenie dwóch wejść różniących się o ustaloną liczbę bitów i obserwuje wyjście.

Analiza liniowa (również atak przez tekst jawny) pozwala zmniejszyć przestrzeń bezpiecznych kluczy do 2^{43} (obecnie można złamać w kilka godzin). Rozwiązaniem jest częste zmienianie kluczy. Gdyby klucz był 128 - bitowy (2^{128} kluczy) - nie do złamania (teoretycznie).

Wniosek: **DES nie jest bezpieczny**!

19 maja 2005 r. – wycofanie certyfikatu NIST dla DES.

Rozwiązanie: np. potrójny DES - Zaadaptowny w ramach standardu ANSI X9.17, ISO 8732 oraz w ramach PEM (*privacy enhanced mail*).

3DES (triple DES)

Rozszerzenie algorytmu DES, w którym stosuje się dwa klucze K_1 i K_2

- **Szyfrowanie**
  1. wiadomość szyfrowana jest kluczem K_1
  2. wynik kroku 1. deszyfrowany jest kluczem K_2
  3. wynik kroku 2. jest ponownie szyfrowany kluczem K_1
- **Deszyfrowanie**
  1. kryptogram deszyfrowany jest kluczem K_1
  2. wynik kroku 1. szyfrowany jest kluczem K_2
  3. wynik kroku 2. jest powtórnie deszyfrowany kluczem K_1

Łamanie 3DES w wersji z 2 kluczami (K_1 i K_2) metodą brutalną - 2^{112} kluczy; (w kontekście nowych metod kryptoanalizy niewystarczające bezpieczeństwo). Łamanie 3DES w wersji z 3 kluczami (K_1, K_2 i K_3) metodą brutalną - 2^{168} kluczy (bezpieczny).

Przykłady szyfrów blokowych

- IDEA (*International Data Encryption Algorithm*)

Ten algorytm szyfrowania/deszyfrowania został stworzony przez Xueji Lai'a oraz Jamesa Massey'a w 1990. Operuje na 64-bitowych blokach wiadomości i wykorzystuje do szyfrowania 128-bitowy klucz. Początkowo szyfr nazywał się *Proposed Encryption Standard* (PES), jednak po odkryciu kryptoanalizy różnicowej został poprawiony i przemianowany na *Improved Proposed Encryption Standard* (IPES). Ostateczna nazwa algorytmu została przyjęta w 1992.

- Blowfish

Szyfr Blowfish stworzył Bruce Schneier w 1993 roku jako szybka i bezpłatna alternatywa dla istniejących ówcześnie algorytmów. Algorytm operuje na 64-bitowych blokach i używa kluczy od 32 do 448 bitów. Algorytm ma postać sieci Feistela z 16 rundami i S-boksami zależnymi od klucza. Każda zmiana klucza wymaga sporej liczby wstępnych obliczeń, żeby ustalić S-boksy.

Przykłady szyfrów blokowych

- AES (*Advanced Encryption Standard*), znany także jako szyfr Rijndael

W 1997 roku rozpoczęły się oficjalne przygotowania do zastąpienia szyfru DES przez nowocześniejszy. Bezpośrednią przyczyną była zbyt mała odporność DES na kryptoanalizę. Organizacja Electronic Frontier Foundation (EFF) była w stanie złamać w roku 1997 wiadomość zakodowaną DESem w ciągu 3 dni sprzętem o wartości 250 tysięcy dolarów (obecnie można złamać DES-a jeszcze szybciej i taniej). Z tej bezpośredniej przyczyny, w styczniu 1997 *National Institute of Standards and Technology* (NIST) wystąpił z propozycją opracowania nowego standardu szyfrowania blokowego, który miał spełnić pewne postulaty, sformułowane przez NIST. Do najważniejszych należały:

- jawność algorytmu,
- publiczna dostępność (brak ograniczeń np. patentowych jak w algorytmie IDEA),
- realizacja szyfru blokowego symetrycznego (z kluczem tajnym),
- obsługa kluczy 128-, 192- i 256-bitowych,
- prostota implementacji sprzętowej i programowej (szybkość pracy).

Przykłady szyfrów blokowych

Do ostatniej fazy konkursu zakwalifikowano 5 szyfrów:

1. Rijndael (od nazwisk autorów: Joana Daemena i Vincenta Rijmena) – 89 głosów,
2. Serpent (ang. „wąż") (Ross Anderson, Eli Biham, L. Knudsen) – 59 głosów,
3. Twofish (projekt zespolowy pod kierunkiem B. Schneiera) – 31 głosów,
4. RC6 (projekt RSA Laboratories) – 23 głosy,
5. MARS (projekt IBM) – 13 głosów.

W październiku 2000 roku NIST ujawnił, że w głosowaniu poparł szyfr Rijndael, a w 2001 roku szyfr ten stał się oficjalnym standardem rządowym jako *Federal Information Processing Standard* FIPS 197.

Rijndael zapewnia obsługę kluczy o długości 128, 160, 192, 224 i 256 bitów (obecnie przyjmuje się za wystarczające 128 bitów, lecz niektórzy autorzy, np. Bruce Schneier uważają, że aby uzyskać bezpieczeństwo *n*-bitów, wszystkie wartości używane w kryptografii powinny mieć długość co najmniej 2*n* bitów.

Szyfr AES **nie jest zbudowany** wg. konstrukcji Feistela. W odróżnieniu od innych szyfrów jego budowa bazuje na teorii ciał Galois, co umożliwia formalne dowodzenie niektórych cech bezpieczeństwa. Liczba rund zależy od długości klucza; dla 128-bitów wiadomości i 128-bitowych kluczy wynosi ona 10 i rośnie do 14 dla dłuższych kluczy i większych bloków. **W odróżnieniu od DES operacje są wykonywane na całych bajtach,** co ułatwia realizację sprzętową i programową.

Przykłady szyfrów blokowych

- Twofish

Jest to jeden z finalistów konkursu na AES. Operuje na blokach 128 bitowych i kluczach o długości 128, 192 lub 256. Nie można używać dłuższych kluczy. Liczba rund wynosi 16, a sam algorytm jest skomplikowany w porównaniu do Rijndael. Nie ma on prostej reprezentacji algebraicznej. Podobnie jak DES wykorzystuje strukturę sieci Feistela. Najlepszy znany atak obejmuje obecnie 10 na 16 rund. Największą wadą tego szyfru jest to, że zmiana klucza szyfrującego wymaga wielu obliczeń wstępnych (wysoki koszt obliczeniowy)

- Serpent

Podobnie jak Twofish jest jednym z finalistów konkursu na AES. Pod pewnymi względami przypomina AES. Składa się z 32 rund, w których dokonuje się przekształceń XOR względem 128 bitowego klucza rund oraz wykorzystuje się 128 bitową liniową funkcję mieszającą. Ponadto równolegle stosuje się 32 czterobitowe S-boksy. Wszystkie 32 S-boksy są identyczne (podobnie jak w AES), ale do dyspozycji jest osiem różnych S-boksów, których używa się po kolei w następnych rundach. O ile AES największy nacisk kładzie na elegancję i wydajność, Serpent zaprojektowano z myślą o bezpieczeństwie. Najlepszy znany atak obejmuje 14 z 32 rund. Wadą algorytmu jest relatywnie mała szybkość. Jest trzykrotnie wolniejszy od AES, a jego wydajna obliczeniowo implementacja jest skomplikowana, gdyż S-boksy muszą być przekonwertowane na wyrażenia boolowski dopasowane do używanego procesora.

Przykłady szyfrów blokowych

- RC5 (od Rivest Cipher no 5)

Szyfr RC5 został opublikowany w 1994 roku przez Rona Rivesta. Ma wiele cech wspólnych z algorytmami DES oraz IDEA. Umożliwia jednak wybieranie liczby rund *r* (max *256)*, wielkości szyfrowanych bloków 2*w* oraz długości klucza *b* w bajtach (standardowe parametry to *r*=12, *w*=32, *b*=16. Podstawowym problemem są, podobnie jak w algorytmie IDEA, ograniczenia patentowe. Nowsza wersja RC5 nosi nazwę RC6. **Uwaga:** nie mylić z RC4, który jest szyfrem strumieniowym i poza nazwiskiem autora nie ma nic wspólnego z RC5 i RC6.

- CAST

Określenie CAST opisuje schemat zastosowany w rodzinie zbliżonych do DES algorytmów kryptograficznych o zmiennej długości kluczy i bloków. Najpowszechniej znany reprezentant to szyfr CAST-128 opublikowany w 1997. CAST-128 składa się z 16 rund, a długość klucza wynosi 128 bitów. Jest używany np. do szyfrowania dysków wirtualnych w pakiecie PGP (*Pretty Good Privacy*) realizującym podstawowe operacje kryptograficzne na różnych platformach (Unix, Windows itd.), głównie na potrzeby użytkowników prywatnych. Długość bloku danych wynosi 64 bity. Nowszą wersją szyfru CAST-128 jest szyfr CAST-256.

PODSTAWY KRYPTOGRAFII

4. Szyfry strumieniowe

4.1. Konstrukcja szyfru strumieniowego
4.2. One-time pad
4.3. Generatory liczb losowych
4.4. Generatory bezpiecznych liczb pseudolosowych
4.5. Przykłady szyfrów strumieniowych

Konstrukcja szyfru strumieniowego

Szyfry strumieniowe w odróżnieniu od szyfrów blokowych nie szyfrują całych bloków danych, lecz pojedyncze bity (czasem bajty) jawnego tekstu wiadomości. W danej chwili jest szyfrowany tylko jeden bit (bajt) wiadomości, a dla kolejnych bajtów zmieniają się przekształcenia szyfrujące. Ogólnie, **sprzętowe szyfry strumieniowe są szybsze od szyfrów blokowych oraz charakteryzują się mniejsza złożonością sprzętową**. Kolejną przewagą nad szyframi blokowymi jest to, że szyfry strumieniowe są od nich lepsze, w sytuacji, gdy nie można zastosować szyfru blokowego (np. ograniczenia buforowania, konieczność przetwarzania znaków zaraz po ich otrzymaniu).

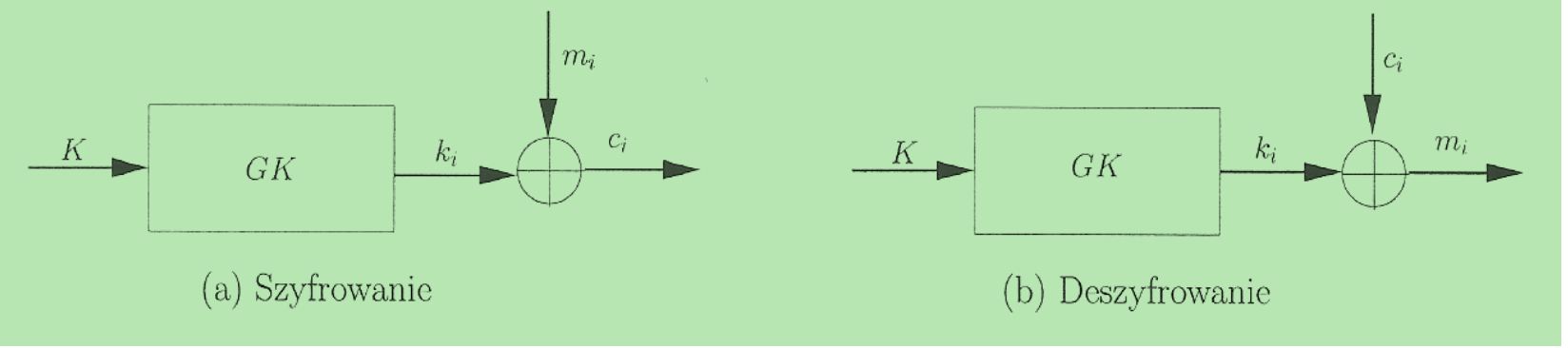
Konstrukcja szyfru strumieniowego

Ze względu na bardzo dużą wiedzę teoretyczną dotyczącą szyfrów strumieniowych zaproponowano różne zasady projektowania tych szyfrów, jednak w literaturze ogólnodostępnej jest ich opisanych relatywnie mało. Powodem tego jest fakt, że większość stosowanych w praktyce szyfrów strumieniowych jest rozwiązaniami prawnie zastrzeżonymi lub poufnymi. Mimo tego szyfry strumieniowe są obecnie stosowane w bardzo różnych dziedzinach, zatem można się spodziewać coraz częstszych wystąpień tego rodzaju szyfrów oraz coraz to nowych propozycji w niedalekiej przyszłości.

Konstrukcja szyfru strumieniowego

Szyfry strumieniowe dzielimy na szyfry strumieniowe **synchroniczne** oraz **asynchroniczne** (samosynchronisujące).

W szyfrze synchronicznym ciąg bitów klucza wytwarzany jest niezależnie od szyfrowanej wiadomości i kryptogramu.

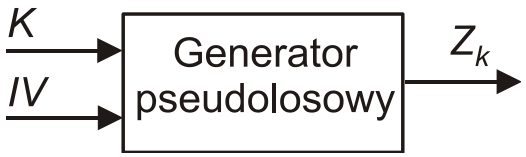


Szyfr strumieniowy synchroniczny z dodawaniem modulo 2 liter klucza i liter wiadomości; GK – generator liter klucza

Konstrukcja szyfru strumieniowego

Cechy szyfru strumieniowego **synchronicznego**:

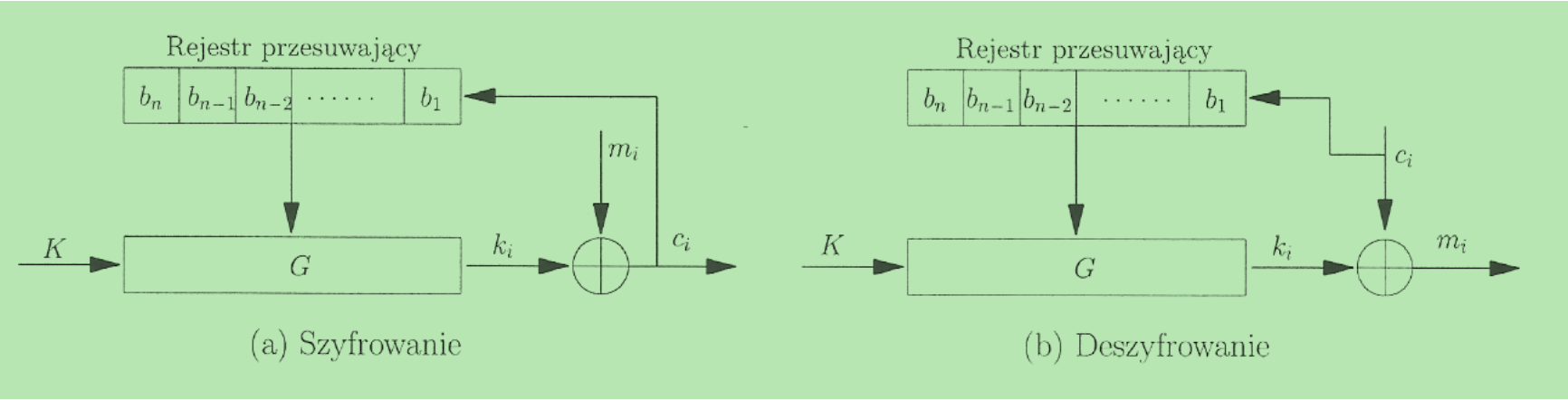
- Wymaga zachowania synchronizacji pomiędzy nadawcą i odbiorcą.
- Zmiana bitu kryptogramu (przekłamanie) nie wpływa na jakość deszyfrowania pozostałych bitów.
- Dodanie lub usuniecie bitu powoduje utratę synchronizacji.
- Istnieje możliwość zmiany wybranych bitów kryptogramu, a co za tym idzie zmiany deszyfrowanej wiadomości.
- Do wytworzenia kluczy bieżących można wykorzystać dowolny szyfr blokowy pracujący w trybie OFB (wyjściowego sprzężenia zwrotnego).
- Strumienie klucza nie mogą się powtarzać. Może to zapewnić np. losowy IV.



Pojedynczy strumień klucza $\{Z_k\}$ o ustalonej długości generowany dla jednej wartości IV nazywamy ramką. Długość ramki nie może być większa od okresu strumienia klucza

Konstrukcja szyfru strumieniowego

W szyfrze asynchronicznym ciąg bitów klucza zależy od szyfrowanej wiadomości i kryptogramu.



Szyfr strumieniowy samosynchronizujący z dodawaniem modulo 2 liter klucza i liter wiadomości; G – generator liter klucza o wartościach zależnych od klucza *K* oraz szyfrowanej wiadomości

Konstrukcja szyfru strumieniowego

Cechy szyfru strumieniowego **asynchronicznego**:

- Realizuje tryb szyfrowego sprzężenia (CFB) z blokiem jednobitowym i wielobitowym IV ($n>>r=1$ w CFB).
- Ograniczona propagacja błędów (przekłamanie 1 bitu wpływa na przekłamanie maksymalnie n bitów).
- Przekłamanie bitu kryptogramu prowadzi do utraty synchronizacji, która odzyskiwana jest automatycznie.
- Wykrycie wrogich zmian w kryptogramie jest trudniejsze (nie wiemy, czy są one spowodowane wrogimi działaniami czy odzyskiwaniem synchronizacji po przekłamaniu w kanale).
- Podobnie jak w szyfrze synchronicznym strumienie klucza nie mogą się powtarzać.

One-time pad

Najsłynniejszym systemem kryptograficznym mającym tajność doskonałą jest szyfr Vernama (1917). Jest to algorytm z kluczem jednorazowym (*one-time pad*). Ma on następującą postać:

$$C_n = M_n + K_n \mod 26$$

gdzie M_n jest pojedynczą literą wiadomości *m*, a K_n jest literą klucza *K (***liczba losowa)** usuwaną natychmiast po użyciu. Na przykład,

$P=N+B \mod 26$, $K=E+F \mod 26$ itd.

Przykładowo, zaszyfrowanie wiadomości ONETIMEPAD kluczem TBFRGFARFM wygeneruje szyfrogram IPKLPSFHGO. Kryptoanalityk bez znajomości liter klucza, które występują z równym prawdopodobieństwem i nie są zależne, nie ma możliwości znalezienia poprawnego tekstu. Obecnie, w użyciu jest wersja binarna algorytmu one-time pad operująca na pojedynczych bitach.

One-time pad

$$\begin{aligned} 0 \oplus 0 &= 0 \\ 0 \oplus 1 &= 1 \\ 1 \oplus 0 &= 1 \\ 1 \oplus 1 &= 0 \end{aligned}$$

- tekst jawny jest ciągiem bitów M=$m_1, m_2, \ldots, m_n$
- wybieramy losowy ciąg bitów K=$k_1, k_2, \ldots, k_n$, który stanowi klucz
- szyfrowanie polega na wykonaniu operacji XOR bit po bicie;

  otrzymujemy w ten sposób losowy ciąg bitów stanowiących kryptogram C=$c_1, c_2, \ldots, c_n$, gdzie $c_i = m_i \oplus k_i$
- operacja ta jest odwracalna;

  ponieważ $a \oplus a = 0$ i $a \oplus b \oplus b = a$, zatem $c_i \oplus k_i = (m_i \oplus k_i) \oplus k_i = m_i$

One-time pad

- kryptogram jest losowym ciągiem n bitów

  Jeśli $k_i = m_i$ to $c_i = 0$, w przeciwnym wypadku $c_i = 1$; prawdopodobieństwo, że $c_i = 0$ jest równe $\frac{1}{2}$ niezależnie od wartości m_i, zatem i-ty bit kryptogramu jest losowy

- szyfr ten jest nie do złamania — **bezpieczeństwo doskonałe** — nie można uzyskać żadnej informacji o tekście jawnym bez znajomości klucza

  Ponieważ $c_i = m_i \oplus k_i$ implikuje $k_i = m_i \oplus c_i$, a kryptogram $c_1, c_2, \ldots, c_n$ odpowiada każdemu możliwemu tekstowi jawnemu z takim samym prawdopodobieństwem, to na podstawie samego kryptogramu nie wiemy nic o tekście jawnym

tekst jawny $\Longrightarrow$	S	Z	Y	F	R
binarnie $\Longrightarrow$	01010011	01011010	01011001	01000110	01010010
klucz $\Longrightarrow$	01110010	01010101	11011100	10110011	00101011
kryptogram $\Longrightarrow$	00100001	00001111	10000101	11110101	01111001

One-time pad

Problemy:

- Klucz musi być wcześniej uzgodniony przez Alicję i Bolka.
- Klucz musi być co najmniej tak długi jak szyfrowany tekst.
- Klucz musi być wybrany naprawdę losowo, co nie jest łatwe.
- Klucz musi być przechowywany w bezpieczny sposób.

UWAGA

Jeśli klucz zostanie użyty do zaszyfrowania wielu tekstów otwartych, szyfr z kluczem jednorazowym traci swoje bezpieczeństwo doskonałe:

$$\left.\begin{matrix} m_1 \oplus k_1 = c_1 \\ m_2 \oplus k_1 = c_2 \end{matrix}\right\} \Rightarrow c_1 \oplus c_2 = m_1 \oplus k_1 \oplus m_2 \oplus k_1 = m_1 \oplus m_2$$

Jeśli dodatkowo przeciwnik zna tekst otwarty *m* i odpowiadający mu kryptogram *c*. Wtedy:

$$m \oplus k = c \quad \Rightarrow \quad k = c \oplus m$$

Generatory liczb losowych

Liczby losowe – elementy ciągu losowego.

Ciąg losowy – z matematycznego punktu widzenia jest to pewna abstrakcja, definiowana przez podanie zbioru oczekiwanych właściwości.

Definicja 1

Ciągiem losowym nazywamy każdy ciąg $\{s_i\}$, gdzie $i = 0, 1, \ldots$, $s_i \in A_\alpha = \{s_0, s_1, \ldots, s_{\alpha-1}\}$, $\alpha \in N$, który spełnia wszystkie znane testy losowości sprawdzające, na ile właściwości ciągu $\{s_i\}$ są zgodne z właściwościami, których oczekujemy od ciągu losowego.

Z punktu widzenia kryptografii można wyróżnić dwie kategorie ciągów losowych, wynikające z zastosowanego źródła losowości:

- Ciągi wykorzystujące zjawiska postrzegane jako losowe.
- Ciągi wykorzystujące zjawiska fizycznie losowe.

Ciągi wytworzone w oparciu o fizyczne źródła losowości (źródła o dodatniej entropii) są nazywane ciągami **rzeczywiście losowymi**, a źródła je wytwarzające, źródłami liczb rzeczywiście losowych (ang. *True Random Number Generator*).

Generatory liczb losowych

Zjawiska postrzegane jako losowe, wykorzystywane do wytworzenia ciągu losowego to na przykład:

- ruchy wykonywane myszą przez użytkownika (kierunek i zwrot ruchu),
- aktualna pozycja myszy,
- zmienność czasu dostępu do pamięci,
- obciążenie procesora,
- parametry operacji dyskowych (numer sektora, czas wyszukania, zawartość obszaru dysku itp.),
- operacje wykonywane przez karty graficzne, szczególnie wielordzeniowe,
- zgłoszenia obsługi różnych urządzeń peryferyjnych,
- parametry rejestrowanego obrazu (jasność składowych RGB, szumy kamery internetowej itp.),
- wiele innych…

Ponieważ ciągi pozyskane w opisany sposób nie spełniają oczekiwań, dlatego na elementach ciągu wykonywane są dodatkowe operacje zwane **postprocessingiem**, których celem jest eliminacja skażeń ciągu (np. zapewnienie zbliżonej liczby zer i jedynek).

Generatory liczb losowych

Fizyczne źródła losowości wykorzystywane do wytworzenia ciągu losowego to na przykład:

- Rozpad promieniotwórczy.
- Zjawiska kwantowe w systemach optycznych
- Różnego rodzaju szumy elementów elektronicznych, np. diod lawinowych.
- Szumy fazowe generatorów pierścieniowych (*Ring Oscillators* - RO).
- Stany metastabilne.

UWAGA

1. O jakości ciągu wyjściowego oprócz wartości entropii źródła losowości decyduje sposób pozyskania losowości.
2. Podobnie jak dla zjawisk postrzeganych jako losowe niezbędne są dodatkowe operacje na elementach wytwarzanego ciągu tj. postprocessing.

Generatory bezpiecznych liczb pseudolosowych

Jednym z powszechnych zastosowań generatora TRNG jest inicjacja układu deterministycznego generującego ciągi „udające" ciągi losowe. Ponieważ elementy ciągu wyjściowego źródła deterministycznego można przewidzieć, znając jego strukturę, parametry i stan początkowy, co nie jest możliwe w przypadku źródła fizycznego, ciągi wytwarzane w układach deterministycznych są nazywane **ciągami pseudolosowymi**. Nazwa „ciągi pseudolosowe" przysługuje jednak tylko tym ciągom wyjściowym źródła deterministycznego, które spełniają zestaw testów statystycznych istotnych dla danego zastosowania.

Generatory bezpiecznych liczb pseudolosowych

Każdy znany generator pseudolosowy można zdefiniować przez podanie pięciu wielkości niezbędnych do jednoznacznego określenia jego właściwości. Są to: **zbiór stanów *S*** o skończonej liczbie elementów, **funkcja przejścia** (ang. *transition function*) $f : S \to S$ przekształcająca elementy zbioru *S* w elementy tego samego zbioru, **zbiór wyjściowy** *U*, **funkcja wyjścia** (ang. *output function*) $g : S \to U$ przekształcająca elementy zbioru *S* w elementy zbioru *U*, oraz **rozkład prawdopodobieństwa** *F*, użyty do wyboru elementu początkowego dla funkcji przejścia *f* i należącego do zbioru *S*. Element ten jest nazywany punktem początkowym, zarodkiem lub **ziarnem** (ang. *seed*) generatora pseudolosowego.

Generatory bezpiecznych liczb pseudolosowych

W zastosowaniach kryptograficznych potrzebujemy ciągów, których elementów nie można przewidzieć na podstawie elementów już wygenerowanych. Ten warunek spełniają ciągi losowe generowane przez fizyczne źródła losowe. Ponieważ w zdecydowanej większości zastosowań żądamy, aby te same ciągi można było wytwarzać w **przestrzennie oddalonych miejscach** bez jednoczesnego przesyłania dodatkowego sygnału, ciągi są w istocie generowane przez generatory pseudolosowe.

Definicja

Ciąg pseudolosowy nazywamy **ciągiem bezpiecznym w przód**, jeśli w realnym czasie i na podstawie dotychczas wygenerowanych elementów ciągu nie można przewidzieć elementów następnych, pomimo że znany jest algorytm lub sprzęt użyty do wytworzenia ciągu.

UWAGA

Nie utożsamiać znajomości algorytmu lub sprzętu ze znajomością wartości zmiennych lub zawartością rejestrów, buforów, pamięci itp.

Generatory bezpiecznych liczb pseudolosowych

Definicja
Ciąg pseudolosowy nazywamy **ciągiem bezpiecznym wstecz**, jeśli w realnym czasie i na podstawie dotychczas wytworzonych elementów ciągu nie można przewidzieć parametrów generatora pseudolosowego (np. ziarna, wartości zmiennych itp.) użytego do wytworzenia tego ciągu pomimo że znany jest algorytm lub sprzęt użyty do wytworzenia ciągu.

W dalszej części pisząc o ciągach bezpiecznych będziemy mieli na uwadze ciągi które są bezpieczne zarówno w przód jak i wstecz.

Jeżeli ciągi bezpieczne są wytwarzane dla wszystkich stanów początkowych generatora ciągów pseudolosowych, to taki generator nazywamy **bezpiecznym generatorem ciągów pseudolosowych**.

Generatory bezpiecznych liczb pseudolosowych

Do wytworzenia bezpiecznego ciągu pseudolosowego możemy użyć:

- Szyfru blokowego lub funkcji skrótu.
- Funkcji nieliniowej.
- Zestawu niezależnych generatorów pseudolosowych z naprzemiennym taktowaniem
- Problemu obliczeniowego uważanego za trudny, np. logarytm dyskretny.
- Uciętego generatora pseudolosowego.
- Kombinacji w/w rozwiązań.

Decydujące znaczenie ma **bezpieczeństwo** wytwarzanych ciągów. Złożoność obliczeniowa, wymagane zasoby itp. są ważne ale mają znaczenie drugorzędne, zaraz po bezpieczeństwie i szybkości wytwarzania ciągu.

PODSTAWY KRYPTOGRAFII

5. Szyfry z kluczem publicznym
   - 5.1. Schemat wymiany kluczy Diffiego-Hellmana
   - 5.2. Szyfr RSA
   - 5.3. Szyfr Rabina
   - 5.4. Szyfr ElGamala
   - 5.5. Wady i zalety szyfrowania z kluczem tajnym i publicznym

Schemat wymiany kluczy Diffiego-Hellmana

Algorytm Diffiego-Hellmana jest **pierwszym z algorytmów z kluczem publicznym**. Został opublikowany w roku 1976. **Nie jest to algorytm szyfrowania**, lecz algorytm bezpiecznej wymiany kluczy, które w dalszej kolejności można wykorzystać w procesie szyfrowania. Bezpieczeństwo algorytmu wynika z trudności obliczenia logarytmu w ciele skończonym. Jest to tzw. problem logarytmu dyskretnego. Algorytm Diffiego-Hellmana do swego działania wymaga ustalenia przez strony komunikujące się dwóch dużych liczb pierwszych p i q takich, że $1<p<q$. Algorytm jest następujący:

Schemat wymiany kluczy Diffiego-Hellmana

Algorytm D-H ma następująca postać:

1. Alicja A wybiera losowo dużą liczbę pierwszą *x* i oblicza $X = p^x \bmod q$ (do obliczenia X wykorzystujemy metodę square-and-multiply, tj. podnoszenie-do-kwadratu-i-mnożenie).
2. Bolek B wybiera losowo dużą liczbę pierwszą *y* i oblicza $Y = p^y \bmod q$ (do obliczenia *Y* wykorzystujemy metodę *square-and-multiply*, tj. podnoszenie-do-kwadratu-i-mnożenie).
3. A wysyła *X* do B, a B wysyła *Y* do A. Liczby *x*, *y* strony zachowują w tajemnicy.
4. Alicja A oblicza $k = Y^x \bmod q = (p^y \bmod q)^x \bmod q = p^{yx} \bmod q$
5. Bolek B oblicza $k' = X^y \bmod q = (p^x \bmod q)^y \bmod q = p^{xy} \bmod q$

Schemat wymiany kluczy Diffiego-Hellmana

Alicja i Bolek otrzymują te same liczby, **które traktują jak tajny klucz**. Nikt inny nie może obliczyć tej wartości na podstawie jawnych *p* i *q*. **Wybór liczb *p* i *q* ma jednakże istotny wpływ na bezpieczeństwo algorytmu**. Moduł *q* musi być dużą liczbą pierwszą (co najmniej 1024 bity; obecnie niektórzy autorzy uważają, że *p* i *q* powinny być kodowane co najmniej 2048 bitami), a liczba (*q*-1)/2 także powinna być liczbą pierwszą. Dodatkowo *p* powinno być elementem **pierwotnym** w GF(*q*) (pierwiastkiem wielomianu pierwotnego).

Algorytm D-H **nie jest odporny na atak typu *Man in the Middle***, zwany czasem atakiem z wnętrza systemu. Polega on na przechwytywaniu na bieżąco informacji przekazywanych pomiędzy komunikującymi się stronami i najczęściej modyfikowanie ich treści. W praktyce może to oznaczać pośredniczenie osoby trzeciej w procesie uzgodnienia klucza. Rozwiązaniem tego problemu jest potwierdzenie tożsamości właściciela danego p^x lub p^y, **certyfikatem** wydanym przez niezależną instytucję.

Szyfr RSA

Szyfr RSA opracowany został w 1977 roku (wkrótce po powstaniu schematu wymiany kluczy D-H) przez trzech matematyków, profesorów z MIT (*Massachussets Institute of Technology*) L. R. Rivesta, A. Shamira i L. M. Adlemana i nazywany jest w skrócie od nazwisk autorów: szyfrem RSA. **Wśród algorytmów asymetrycznych odegrał podobną rolę jak DES wśród algorytmów symetrycznych**. Inaczej jednak niż dla DES-a, w przypadku RSA długość klucza może być **dowolnie ustalana przez użytkownika**. Inną ważna zaletą tego szyfru jest **symetria** tzn. wiadomość może być szyfrowana zarówno kluczem publicznym jak i prywatnym, a to ostatnie ma istotne znaczenie w **podpisie cyfrowym**.

Szyfr RSA

Bezpieczeństwo RSA oparte jest na **trudności rozkładu dużych liczb na czynniki pierwsze**. Aby dokonać złamania szyfru RSA należy rozbić klucz publiczny na dwie liczby pierwsze będące jego dzielnikami. Takie rozbicie liczb pozwala **rozszyfrować każdą informację zakodowaną kluczem prywatnym i publicznym**. Nie jest to jednak tak proste do zaimplementowania, ponieważ nie ma prostej metody rozbijania dużych liczb na czynniki pierwsze. **Nie istnieje żaden wzór, do którego podstawiamy daną liczbę i w wyniku otrzymujemy wartości jej czynników pierwszych**. Należy je znaleźć testując podzielność kolejnych liczb. Obecnie takie operacje zajmują bardzo dużo czasu, jednak na skutek postępu w zakresie rozkładania liczb na czynniki pierwsze, należy od czasu do czasu sprawdzać jaka długość kluczy jest uznawana (najlepiej przez osoby specjalizujące się w tej dziedzinie) w danym momencie za bezpieczną. Od chwili powstania RSA następowały **liczne próby** złamania szyfru tą metodą. Konsekwencją tego było zwiększenie długości kluczy używanych przez RSA.

Szyfr RSA

Algorytm RSA składa się z następujących kroków:

Wybór klucza:

1. Wybieramy losowo dwie duże (różne od siebie) liczby pierwsze *p, q*, każda w przybliżeniu tej samej wielkości.
2. Obliczamy iloczyn
$$n = p \cdot q$$
3. Wybieramy losowo **klucz szyfrujący** ***e*** taki, żeby liczby *e* oraz $\varphi = \varphi(n) = (p-1)\cdot(q-1)$ były tego samego rzędu co *n* oraz NWD(e, φ)=1. W tym celu wystarczy wybrać dowolną liczbę *e* i za pomocą algorytmu Euklidesa sprawdzić, czy NWD(*e*, ϕ)=1. Liczba ta powinna również spełniać nierówność 1 < *e* < *n*. Nie musi ona być pierwsza, lecz musi być nieparzysta. Po obliczeniu φ liczby pierwsze *p* i *q* **usuwamy**, aby nie wpadły w niepowołane ręce.
4. Wyznaczamy **klucz deszyfrujący** $d = e^{-1} \bmod \varphi$ będący odwrotnością *e* modulo φ, tj. spełniający warunek.
$$(e \cdot d) \bmod \varphi = 1$$
Do wyznaczenia *d* możemy zastosować rozszerzony algorytm Euklidesa.
5. **Klucz publiczny tworzy para liczb (*e,n*)**, można go przekazywać wszystkim zainteresowanym. **Kluczem prywatnym jest liczba *d***. Klucz ten należy przechowywać pod ścisłym nadzorem.

Szyfr RSA

Szyfrowanie:

1. Otrzymujemy od adresata autentyczny klucz publiczny w postaci pary liczb (*e,n*).
2. Wiadomość do zaszyfrowania zamieniamy na liczby całkowite *m* należące do przedziału $0 < m < n$. Możemy tutaj skorzystać np. z łączenia kodów znaków. Oczywiście adresat musi znać użyty przez nas sposób przekształcenia tekstu w liczbę, aby mógł on później odtworzyć otrzymaną wiadomość.
3. Na otrzymanych w ten sposób liczbach wykonujemy operację szyfrowania. Otrzymujemy liczby
$$c = m^e \bmod n .$$
4. Liczby *c* są zaszyfrowaną postacią liczb *m,* które następnie przekazuje się adresatowi wiadomości.

Deszyfrowanie:

W celu odszyfrowania wiadomości, podnosimy liczbę *c* do potęgi *d* modulo *n*:

$$D(c) = c^d \bmod n = m^{e \cdot d} \bmod n = m^1 \bmod n = m$$

Szyfr RSA

Przykład:

Wybór klucza:

1. Wybieramy losowo dwie liczby pierwsze *p*=11, *q*=17.
2. Wyliczamy $n = p \cdot q = 11 \cdot 17 = 187$.
3. Obliczamy $\varphi = (p-1) \cdot (q-1) = 10 \cdot 16 = 160$. Wybieramy klucz publiczny *e* (szyfrujący) taki, że *e* i $\varphi = 160$ są liczbami względnie pierwszymi, np. $e = 7$.
4. Wyliczamy klucz prywatny taki, że $(e \cdot d) \bmod \varphi = 1$ tj. $(7 \cdot d) \bmod 160 = 1$. Jest to liczba $d = 23$ $(7 \cdot d = 10 \cdot 16 + 1)$.

Otrzymaliśmy klucz publiczny składający się z pary (7, 187) oraz klucz prywatny 23.

Szyfrowanie:

Załóżmy, że chcemy przesłać adresatowi zaszyfrowaną liczbę ***m* = 65**. W tym celu musimy obliczyć wartość wyrażenia:

$$c = m^e \bmod n = 65^7 \bmod 187 = 142$$

Liczbę $c = 142$, która szyfruje liczbę 65, przesyłamy do adresata

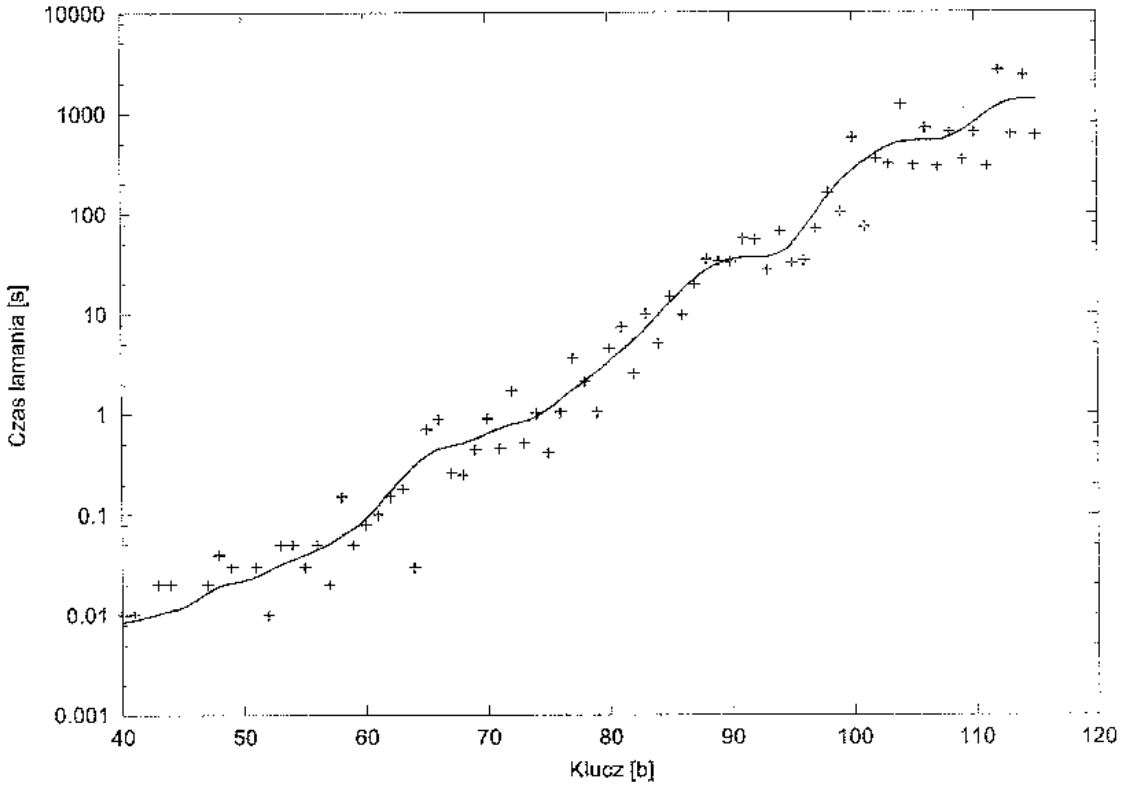
Deszyfrowanie:

W celu dokonania dekodowania otrzymanej liczby $c = 142$ adresat oblicza:

$$m = c^d \bmod n = 142^{23} \bmod 187 = 65$$

Szyfr RSA

Główną wadą szyfru RSA jest jego **znacząco większa złożoność obliczeniowa w porównaniu z innymi szyframi** na przykład z DES-em.
Szyfr RSA jak każdy szyfr bezpieczny obliczeniowo można złamać.



Czas łamania szyfru RSA w zależności od długości klucza *L* (oś Y – skala log). Linia falista, to aproksymacja otrzymanych wyników.

Szyfr RSA

Średni czas łamania szyfru RSA można określić w przybliżeniu wzorem

$$t \approx s \cdot \frac{2^{L/4}}{2 \cdot 10^5} \quad [s]$$

Dla klucza o długości 100 bitów wynosi, średnio 2 minuty 47 sekund. Dla 128 bitów prawie 6 godzin, a dla 200 bitów 178 lat. Dla 300 bitów otrzymujemy już 6 mld lat. Podany wzór otrzymano dla komputera z procesorem Intel Celeron 433 MHz (s=1), a więc relatywnie wolnego. Użycie szybszego procesora nie zmienia zasadniczo wzoru, lecz wprowadza do niego współczynnik skali s mniejszy od jedności. Należy jednak zaznaczyć, że wzór wyprowadzono tylko dla jednej funkcji faktoryzującej, związanej z biblioteką języka C. Dla innych funkcji wyniki mogą być lepsze lub gorsze.

Uwaga: W przypadku RSA można dowieść, że obliczenie ostatniego bitu tekstu jawnego jest w przybliżeniu tak samo trudne jak odtworzenie całego kryptogramu (Kut. p. 69)

Szyfr Rabina

Bezpieczeństwo szyfru Rabina (1979) opiera się również na trudności faktoryzacji dużych liczb. Różnica jest taka, że w przypadku szyfru Rabina można pokazać, że każdy kto może efektywnie złamać system Rabina, może również efektywnie rozkładać liczby całkowite na czynniki pierwsze.

Wybór klucza:

Alicja losowo wybiera dwie duże liczby pierwsze takie, że

$$p \equiv q \equiv 3 \bmod 4$$

Liczba $n=pq$ jest kluczem publicznym Alicji, a para $\{p,q\}$ kluczem prywatnym.

Szyfrowanie:

Aby zaszyfrować tekst otwarty $m<n$-1 Bolek używa klucza publicznego n Alicji i oblicza

$$c = m^2 \bmod n$$

Deszyfrowanie:

$$D(c) = c^{1/2} \bmod n$$

Aby obliczyć tekst jawny musimy znaleźć pierwiastki równania

$$m^2 = c \bmod n$$

Szyfr Rabina

Ponieważ $n=pq$, powyższe równanie można zapisać w postaci układu równań:

$$\begin{cases} m_p^2 = c \bmod p \\ m_q^2 = c \bmod q \end{cases}$$

Równanie spełniają dwie pary liczb: $\pm m_p$ i $\pm m_q$. Można je rozwiązać numerycznie korzystając z rozszerzonego algorytmu Euklidesa oraz Chińskiego twierdzenia o resztach.

Wniosek:

Produktem deszyfrowania są cztery możliwe wiadomości. Alicja musi dokonać selekcji i wybrać właściwą wiadomość od Bolka.

Zaletą szyfru Rabina jest **znacząco większa szybkość szyfrowania** niż w szyfrze RSA. Deszyfrowanie wymaga podobnego nakładu obliczeniowego.

Szyfr ElGamala

W 1985 roku na łamach IEEE *Transactions on Information Theory* w artykule „A public key cryptosystem and a signature scheme based on discrete logarithms" Taher ElGamal zaproponował inne podejście do systemów klucza publicznego. W pracy opisał **system szyfrowania wiadomości** wykorzystujący koncepcję klucza publicznego, który był ściśle powiązany z wymianą kluczy Diffiego-Hellmana. Bezpieczeństwo systemu, podobnie jak bezpieczeństwo schematu wymiany kluczy D-H jest oparte na trudności związanej z obliczeniem logarytmu dyskretnego. Kolejne kroki algorytmu są następujące:

Wybór klucza:

1. Losowo wybieramy dużą liczbę pierwszą p taką, że obliczenie logarytmu dyskretnego jest niewykonalne w rozsądnym czasie (obecnie co najmniej 2048 bitów).
2. Wybieramy losowo liczbę g taką, że $1<g<p-1$.
3. Wybieramy dużą liczbę a taką, że $0<a<p-1$, która będzie naszym **kluczem prywatnym**.
4. Obliczamy $A=g^a \bmod p$. Trójka $\{A, g, p\}$ jest naszym **kluczem publicznym**.

Szyfr ElGamala

Szyfrujemy wiadomość *m* kluczem publicznym (*A, g, p*). Aby tego dokonać wybieramy losowo liczbę *k*, która jest względnie pierwsza z *p*-1, i obliczamy:

$$\begin{cases} c_1 = g^k \bmod p \\ c_2 = m \cdot A^k \bmod p \end{cases}$$

Szyfrowane mogą być liczby $m<p$. Para liczb c_1 i c_2 tworzy kryptogram, który jest dwa razy dłuższy od *m*.

Deszyfrowanie:
Deszyfrowanie polega na obliczeniu wartości wyrażenia:

$$D(C) = c_2 \cdot \left((c_1)^a\right)^{-1} \bmod p$$

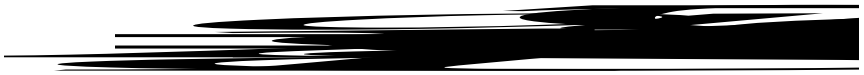
Sprawdzenie:

$$c_2 \cdot \left((c_1)^a\right)^{-1} = m \cdot A^k \cdot (g^{ak})^{-1} = m \cdot g^{ak} (g^{ak})^{-1} = m$$

Szyfr ElGamala

Przykład:

Wybierzmy *p*=229, *g*=6, *a*=70. Wtedy:



Klucz publiczny tworzą liczby {97,6,229}, które są dostępne dla każdego, kto chce nam przesłać wiadomość. Niech Bolek chce do nas wysłać liczbę *m*=130. W tym celu wybiera losowe *k*=127 takie, że NWD(127,228)=1 i oblicza:

$$\begin{cases} c_1 = 6^{127} \bmod 229 = 155 \\ c_2 = 130 \cdot 97^{127} \bmod 229 = 169 \end{cases}$$

Bolek przesyła do nas kryptogram {155,169}. Aby odszyfrować wiadomość musimy obliczyć:

$$D(C) = 169 \cdot \left((155)^{70}\right)^{-1} \bmod 229 = 169 \cdot 36 \bmod 229 = 130$$

Zwróćmy uwagę, że deszyfrujący nie musi znać liczby *k* użytej przez szyfrującego wiadomość.

Wady i zalety szyfrowania z kluczem tajnym i publicznym

Zalety kryptografii z kluczem symetrycznym:

1. Szyfry z kluczem symetrycznym mogą być projektowane z myślą o systemach o dużej przepływności danych. Znane są realizacje sprzętowe osiągające szybkość szyfrowania wielkości gigabajtów na sekundę, natomiast systemy programowe mogą osiągnąć przepustowość rzędu kilkdziesięciu megabitów na sekundę.
2. Klucze w szyfrach z kluczem symetrycznym są dość krótkie.
3. Szyfry z kluczem symetrycznym mogą być stosowane jako pierwiastki pierwotne, wchodzące w skład różnych mechanizmów kryptograficznych, w tym do efektywnych schematów podpisów cyfrowych.
4. Szyfry z kluczem symetrycznym w połączeniu ze sobą mogą tworzyć silniejsze szyfry. Proste podstawienia, które są przewidywalne i łatwe do rozwiązania, mogą być skumulowane w celu budowy silnych szyfrów iloczynowych.
5. Powszechnie przyjęte jest, że historia szyfrowania z kluczem symetrycznym ma dość dalekie początki, mimo to odkrycia bliższe obecnym czasom mają znaczący wpływ na rozwój kolejnych, nowych systemów i technologii. Przykładem takiego następstwa jest wynalezienie komputera cyfrowego, które pociągnęło za sobą stworzenie projektu standardu szyfrowania danych DES we wczesnych latach siedemdziesiątych XX wieku.

Wady i zalety szyfrowania z kluczem tajnym i publicznym

Wady kryptografii z kluczem symetrycznym:

1. W przypadku dwustronnej wymiany danych klucz musi pozostać utajniony po obu stronach.
2. W dużych sieciach musimy zarządzać dużą liczbą par kluczy.
3. W przypadku obustronnej wymiany wiadomości pomiędzy podmiotami A i B zgodnie z doświadczeniami kryptograficznymi zaleca się zmieniać klucz, przynajmniej dla każdej sesji komunikacyjnej.
4. Mechanizmy podpisów cyfrowych oparte na szyfrowaniu z kluczem symetrycznym wymagają alternatywy w postaci dłuższych kluczy dla publicznych funkcji weryfikujących lub zastosowania złożonych procedur dodatkowych.

Wady i zalety szyfrowania z kluczem tajnym i publicznym

Zalety kryptografii z kluczem publicznym:

1. W tajemnicy musi być trzymany jedynie klucz prywatny (autentyczność klucza publicznego musi być zagwarantowana).
2. W zależności od zastosowania, zarówno klucz prywatny jak i publiczny nie wymagają zmiany przez dłuższy czas - przez wiele sesji a nawet przez kilka lat.
3. Wiele schematów z kluczem publicznym zapewnia efektywne mechanizmy podpisów cyfrowych. Klucz używany do opisu publicznej funkcji weryfikującej jest zwykle o wiele krótszy niż jego symetryczny odpowiednik.
4. W dużych sieciach liczba kluczy może być znacząco mniejsza niż w wypadku systemu z kluczem symetrycznym.

Wady i zalety szyfrowania z kluczem tajnym i publicznym

Wady szyfrowania z kluczem publicznym:

1. Szybkości przetwarzania dla najpopularniejszych metod szyfrowania z kluczem publicznym są o kilka rzędów wielkości mniejsze w stosunku do najlepszych schematów z kluczem symetrycznym.
2. Długości klucza są zwykle o wiele większe niż te, które są stosowane w przypadku szyfrowania z kluczem symetrycznym. Długość podpisów opartych na kryptografii z kluczem publicznym jest większa niż dla znaczników zapewniających uwierzytelnianie źródła danych tworzonych w oparciu o metody z kluczem symetrycznym.
3. W stosunku do żadnego schematu z kluczem publicznym, podobnie jak miało to miejsce w przypadku szyfrów blokowych, nie dowiedziono, że schemat szyfrowania jest bezpieczny bezwarunkowo. Bezpieczeństwo najlepszego ze znanych schematów szyfrowania z kluczem publicznym opiera się na złożoności niewielkiego zbioru problemów z teorii liczb.
4. Kryptografia z kluczem publicznym, odkryta w połowie lat siedemdziesiątych XX wieku, nie ma tak długiej historii jak szyfrowanie z kluczem symetrycznym.

Porównanie podstawowych właściwości kryptografii z kluczem tajnym (symetrycznej) i prywatnym (asymetrycznej)

Kryptografia symetryczna	Kryptografia asymetryczna
wysoka efektywność przetwarzania	niska efektywność przetwarzania (o kilka rzędów wielkości mniejsza w stosunku do kryptografii symetrycznej)
stosunkowo krótkie klucze (64,128,256 bitów)	długie klucze (zaleca się długości > 1024 bitów)
konieczność zarządzania znaczną liczbą par kluczy w dużych sieciach	znacząco mniejsza liczba kluczy
szeroki zakres zastosowań przy konstrukcji różnorodnych zastosowań kryptograficznych	konieczność utrzymania w sekrecie wyłącznie klucza prywatnego
bardzo wysoki stopień ochrony	wymagane zapewnienie autentyczności klucza publicznego przez zaufaną trzecią stronę
konieczność stworzenia bezpiecznego kanału wymiany kluczy	formalnie nie udowodnione bezpieczeństwo (nie ma dowodu, że nie istnieje efektywny algorytm pozwalający na złamanie szyfru)
w praktyce wymagane częste zmiany kluczy, nawet podczas otwierania każdej nowej sesji wymiany danych	pary kluczy mogą być używane przez dłuższy okres czasu, co oznacza możliwość korzystania z wielu sesji jednocześnie

Wady i zalety szyfrowania z kluczem tajnym i publicznym

Podsumowanie

Na obecną chwilę **efektywność obliczeniowa szyfrowania z kluczem publicznym jest mniejsza w stosunku do efektywności szyfrowania z kluczem symetrycznym**. Nie jest to jednak regułą. Najważniejszymi praktycznymi komponentami są:

- Kryptografia z kluczem publicznym realizująca **podpisywanie** (ze szczególnym naciskiem na niezaprzeczalność) oraz **zarządzanie kluczami**.
- Kryptografia z kluczem symetrycznym jest skuteczna w przypadku **szyfrowania oraz do pewnych działań w przypadku integralności danych**.

Dokonując liczbowego porównania, stwierdza się, że klucze prywatne w schematach z kluczem publicznym muszą być dłuższe (np. 1024 lub 2048 bitów dla RSA) niż klucze symetryczne w systemach z kluczami symetrycznymi (np. 128 bitów lub 256 bitów). W przypadku bezpiecznych algorytmów dla systemów z kluczem symetrycznym **najskuteczniejszą ofensywą jest przeszukanie całej przestrzeni kluczy**. Natomiast systemy z kluczem publicznym, uległe są na ataki **„skrótem"** (np. na faktoryzację), które są bardziej efektywne niż przeszukiwanie przestrzeni kluczy. W celu osiągnięcia porównywalnej ochrony, liczba bitów kluczy symetrycznych może być znacznie mniejsza w stosunku do liczby bitow kluczy prywatnych w systemach z kluczem publicznym (przynajmniej 10-krotnie).

PODSTAWY KRYPTOGRAFII

6. Kryptograficzna funkcja skrótu
   - 6.1. Funkcja skrótu
   - 6.2. Ataki przeciwko funkcji skrótu
   - 6.3. Przykłady funkcji skrótu
   - 6.4. Kody uwierzytelniające wiadomość (MAC)

.

Kryptograficzna funkcja skrótu

Jednym z fundamentalnych elementów współczesnej kryptografii jest kryptograficzna funkcja skrótu (ang. *hash function*), potocznie nazywana jednokierunkową funkcją skrótu. W ogólności funkcja skrótu jest obliczeniowo efektywną funkcją odwzorowującą ciągi o wybranej długości na ciągi o pewnej mniejszej i ustalonej długości, zwane wartościami skrótu.

Definicja

Funkcję h nazywamy funkcją skrótu (hashującą), jeśli ma co najmniej trzy właściwości:

- właściwość kompresji – h przekształca wejście x o dowolnej długości, liczonej w bitach, na wyjście $h(x)$ o ustalonej, mniejszej długości,
- właściwość łatwości obliczeń – dla każdego x łatwo obliczyć $h(x)$ (w czasie wielomianowym),
- jednokierunkowość – dla zadanego $y=h(x)$ znalezienie takiego x, że $h(x)=y$ jest obliczeniowo trudne,

Kryptograficzna funkcja skrótu

Zastosowania funkcji skrótu:

- Podpis cyfrowy – można uwierzytelnić wiadomość bez potrzeby szyfrowania, co znacząco skraca proces podpisywania wiadomości.
- Sprawdzanie integralności danych transmitowanych lub przechowywanych w sposób jawny lub tajny. W przypadku transmisji porównywane są skróty otrzymane w nadajniku i odbiorniku. Dla danych przechowywanych obliczamy skróty w pewnych odstępach czasu, a następnie je porównujemy.
- Przechowywanie haseł – obliczamy i przechowujemy skrót obliczony dla danych złożonych z hasła i liczby losowej zamiast przechowywać hasła.
- Detekcja błędów - dołączenie przed transmisją do wiadomości wartości skrótu pozwala na wykrycie modyfikacji spowodowanych przyczynami technicznymi (skuteczniejsza detekcja błędów niż np. za pomocą kodu BIP-N).

Kryptograficzna funkcja skrótu

Funkcje skrótu są **zwykle publicznie znane i nie zawierają kluczy tajnych**. Jeżeli służą jako test wykrywalności zmian treści wiadomości wejściowych, to mamy do czynienia z kodami wykrywającymi modyfikacje (ang. *Modification Detection Codes* - MDC).

Jeżeli użytkownikowi zależy na uwierzytelnieniu nadawcy (np. wytwórcy produktu), to może zastosować funkcje skrótu bazujące na **kluczu tajnym**, gwarantujące uwierzytelnianie źródła danych. Kody te, w specjalistycznej terminologii figurują pod nazwą kodów uwierzytelniania wiadomości (ang. *Message Authentication Codes* - MAC). Tajny klucz (hasło) można np. dołączyć do tekstu, który ulega hashowaniu.

Kryptograficzna funkcja skrótu

Niezależnie od tego, czy korzystamy z MDC czy z MAC głównym wymaganiem bezpieczeństwa funkcji skrótu **jest niemożność odnalezienia dwóch wiadomości o takiej samej wartości skrótu (brak tzw. kolizji)**. Jest to poważna reguła, bez której podpis na jednej wiadomości mógłby być taki sam jak na innej wiadomości. Zapewniałoby to podmiotowi nadawcy możliwość podpisania jednej wiadomości, a następnie podmiot ten mógłby twierdzić, że podpisał inną wiadomość. W tym kontekście wyróżniamy dwie klasy funkcji skrótu:

Kryptograficzna funkcja skrótu

Definicja

Funkcja skrótu *h* jest słabo bezkonfliktowa, jeśli dla danego x nie jest praktycznie możliwe wyznaczenie elementu *x'* różnego od *x* takiego, że $h(x)=h(x')$.

Definicja

Funkcja skrótu *h* jest silnie bezkonfliktowa, jeśli nie jest praktycznie możliwe wyznaczenie elementów *x* oraz *x'* , gdzie $x \neq x'$ takich, że $h(x)=h(x')$.

Kryptograficzną funkcję skrótu powinny cechować następujące właściwości:

- praktycznie brak możliwości otrzymania tych samych wartości skrótu dla różnych danych wejściowych (silna bezkonfliktowość),
- brak możliwości wnioskowania o treści wiadomości wejściowej na podstawie wartości skrótu (jednokierunkowość),
- jakakolwiek, nawet najmniejsza zmiana informacji wejściowej powoduje zmianę wartości skrótu.

Ataki przeciwko funkcji skrótu

Podstawowe ataki przeciwko funkcjom skrótu to:

- atak słownikowy,
- atak urodzinowy,
- ataki specjalistyczne, wykorzystujące słabości konkretnej funkcji skrótu.

Atak słownikowy

Jest najprostszą formą łamania haseł w systemach komputerowych, w których hasła użytkowników są zapamiętywane poprzez wartość funkcji skrótu. Celem ataku nie jest znalezienie hasła konkretnego użytkownika, ale **jakiegokolwiek hasła używanego w systemie.** Są to tzw. słabe hasła. Atak wykorzystuje fakt, że wielu użytkowników, zwłaszcza początkujących, wybiera słabe hasła, np. daty urodzin, nazwiska, przezwiska, nieprzyzwoite wyrazy itp. Scenariusz ataku jest następujący:

Ataki przeciwko funkcji skrótu

Krok 1

Obliczamy wartość funkcji skrótu używanej w systemie dla haseł (słów) z dużego słownika. Słownik powinien zawierać słowa z języków, z którymi mógł się zetknąć użytkownik atakowanego systemu (np. angielskie, polskie, niemieckie itd.). Słownik powinien zawierać nazwy geograficzne i inne nazwy własne (np. nazwiska i imiona bohaterów seriali, książek, komiksów itp.). Dobrze jest uwzględnić także słowa zapisane z drobnymi zmianami np. zaba zamiast żaba. Zazwyczaj takie słowniki już istnieją i są dostępne np. w Internecie, na płycie CD/DVD itp.

Krok 2

Kopiujemy plik z wartościami funkcji skrótu użytkowników na nośnik wymienny, lub bezpośrednio do naszego komputera. Pliki ze skrótami w większości systemów operacyjnych nie są chronione lub ochrona jest słaba (np. ukrycie w systemie plików widocznym tylko dla administratora).

Krok 3

W domu, na własnym komputerze poszukujemy wartości skrótu, który jest zgodny ze skrótem otrzymanym dla jakiegokolwiek hasła z posiadanego słownika.

Ataki przeciwko funkcji skrótu

Ochrona przed atakiem słownikowym:

1. Prewencyjne ataki słownikowe na hasła zaproponowane przez użytkowników.
2. Instalowanie programów wykrywających słabe hasła na etapie ich ustanawiania.
3. Wytwarzanie silnych haseł przez system bez możliwości zmiany przez użytkownika.
4. Stosowanie dodatkowych zabezpieczeń dla wprowadzanych haseł.

Przykładowo, dla każdego użytkownika tworzony jest rekord zawierający dodatkowy ciąg losowo wybranych bitów, np. 24 bitowy, oraz wartość skrótu obliczoną łącznie dla tego ciągu i hasła ustanowionego przez użytkownika. Chociaż ciąg losowy może być odczytany to jest różny dla każdego hasła używanego w systemie, a kojarzenie wielu 24-bitowych ciągów z bardzo dużą liczbą haseł z pozyskanego słownika znacząco wydłuża czas skutecznego ataku.

Ataki przeciwko funkcji skrótu

Atak urodzinowy

Atak urodzinowy wykorzystuje tzw. paradoks urodzinowy polegający na tym, że prawdopodobieństwo iż w grupie losowo N wybranych osób są dwie osoby o tej samej dacie urodzin jest znacząco większe od prawdopodobieństwa, że jest tam osoba o tej samej dacie urodzin co dana osoba. To ostatnie prawdopodobieństwo wynosi N/365. Natomiast prawdopodobieństwo, że w grupie N osób są dwie osoby o tej samej dacie urodzin wynosi:

$$1 - \frac{364}{365} \cdot \frac{363}{365} \cdot \ldots \cdot \frac{365 - N + 1}{365}$$

Przykładowo, dla *N*=20 otrzymujemy 0,44, podczas gdy prawdopodobieństwa, że jest tam osoba o tej samej dacie urodzin co dana osoba wynosi tylko 0,06. Powyższe możemy wykorzystać do wykonania następującego ataku (urodzinowego).

Ataki przeciwko funkcji skrótu

Krok 1

Alicja przygotowuje dwie wersje umowy: wersję A1 i wersję A2, która przynosi dodatkowe korzyści Alicji.

Krok 2

Alicja dokonując niewielkich zmian w plikach A1 i A2, np. zmiana odstępu, sposób łamania wierszy czy nawet celowe „literówki" tak długo, aż otrzyma identyczne skróty dla obu umów. Wynikiem są dwie umowy, nazwijmy jest B1 i B2, które cechuje ta sama wartość skrótu.

Krok 3

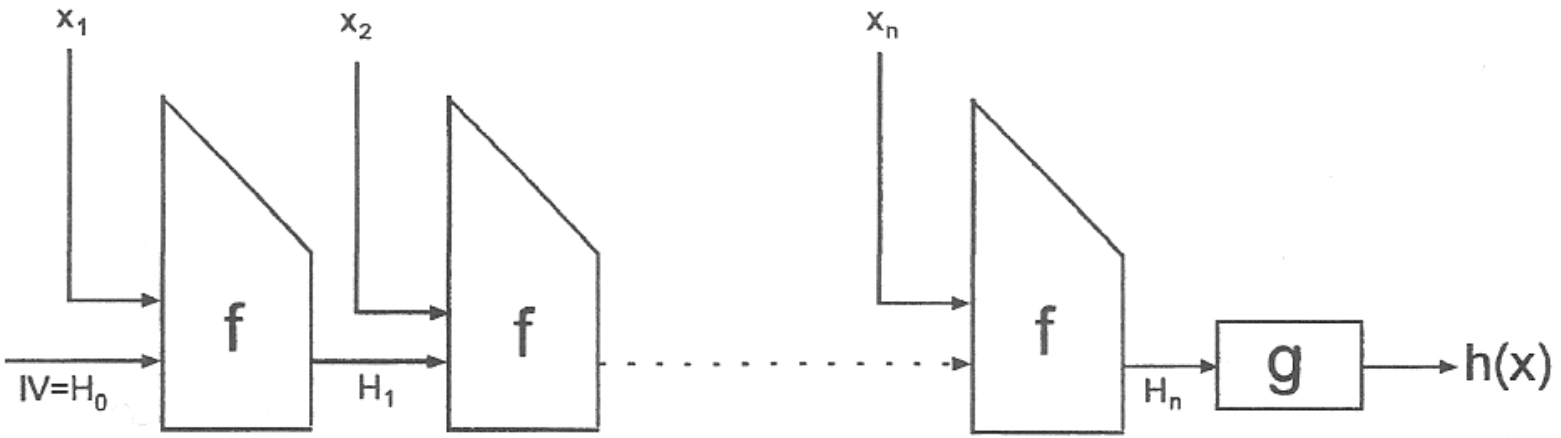
Bolek w dobrej wierze podpisuje wartość funkcji skrótu obliczoną dla B1.

Krok 4

Po pewnym czasie Alicja oskarża Bolka o złamanie umowy. Przedstawia umowę B2 ze skrótem podpisanym przez Bolka. Chociaż Bolek może przedstawić umowę B1, nie dowodzi to, kto oszukuje, Alicja czy Bolek.

Przykłady funkcji skrótu

Ogólny schemat konstrukcji funkcji skrótu to wielokrotne iteracje funkcji elementarnej odpornej na kolizje (R. C. Merkle i niezależnie I. B. Damgard, CRYPTO'89).



f – funkcja elementarna
$x_1, x_2, \ldots, x_n$ – kolejne bloki wiadomości
IV – wektor inicjujący
$g(H_n)$ – dodatkowe przekształcenie, np. nieliniowe, nie zawsze używane

Przykłady funkcji skrótu

MD5 (Message Digest Algorithm 5)

Algorytm MD5 został zaproponowany przez Rona Rivesta, współtwórcę algorytmu RSA, w 1991 roku. Jest następcą algorytmu MD4, który w wyniku analizy przeprowadzonej przez Hansa Dobbertina okazał się zbyt mało bezpieczny.

Algorytm MD5 jest funkcją skrótu przekształcającą **512-bitowe bloki danych w bloki 128-bitowe**. W pierwszym etapie tekst jawny uzupełniany jest ciągiem bitów w taki sposób, aby był o 64 bity krótszy od wielokrotności 512 i następnie dzielony jest na bloki 512-bitowe. Uzupełnienie polega na dołączeniu na końcu wiadomości jedynki i odpowiedniej liczby zer. Na pozostałych 64 bitach zapisywany jest rozmiar wiadomości.

Algorytm MD5 wykorzystuje 128-bitowy wektor początkowy IV podzielony na cztery 32-bitowe zmienne A, B, C, D przypisane następnie zmiennym a,b,c,d. Podział na słowa 32-bitowe pozwala na wydajną implementację na procesorach 32-bitowych.

Stan początkowy inicjowany jest wartością:IV=0123456789abcdeffedcba9876543210.

Przykłady funkcji skrótu

Dla większości zastosowań 128-bitowy obszar mieszania nie jest wystarczający. W związku ze skutecznymi atakami na MD5 przeprowadzonymi w latach 2004-2006 i udowodnieniu braku odporności na kolizję, metoda ta, podobnie jak MD4, nie jest uznawana za bezpieczną. W 2006 udało się doprowadzić do kolizji w przeciągu jednej minuty, korzystając z pojedynczego komputera i dedytkowanej metody (tunelowanie). Za następcę MD5 uznano najpierw algorytm SHA-1, od 2002 roku SHA-2, a od 2012 roku także SHA-3 (Keccak).

Przykłady funkcji skrótu

SHA-1 (Secure Hash Algorithm version 1)

Algorytm SHA-1 jest funkcją skrótu powszechnie stosowaną w narzędziach kryptograficznych, m.in. w algorytmach podpisów cyfrowych (np. DSA, Digital Signature Algorithm) oraz w protokole SSL (ang. Secure Socket Layer) w latach dziewięćdziesiątych oraz na początku XXI wieku. Standard został opracowany w **1993 roku przez NSA** (National Security Agency) jako FIPS 180-1. **Struktura SHA oparta jest na MD5**, co implikuje wiele cech wspólnych pomiędzy algorytmami. Algorytm SHA-1 przetwarza dane wejściowe w blokach 512-bitowych, generując **160-bitową wartość skrótu**. Proces przekształcenia wstępnego (podział na bloki i procedura uzupełnienia) przebiega tak samo jak w algorytmie MD5. Zdefiniowane jest 5 inicjujących zmiennych stanu *A,B,C,D,E* przypisywanych w trakcie wykonywania pętli głównej zmiennym a,b,c,d,e (wektor IV). Wartości początkowe zmiennych stanu wynoszą:

A=0x677452301, B=0xefcdab89, C=0x98badcfe, D=0x10325476, E=0xc3d2e1f0.

Przykłady funkcji skrótu

SHA-2 (Secure Hash Algorithm version 2)

Pod nazwą SHA-2 kryje się zestaw kryptograficznych funkcji skrótu SHA-224, SHA-256, SHA-384 oraz SHA-512, generujących skrót o długości bitowej zgodnej z ich nazwą. Funkcje te zostały zaproponowane przez NSA w 2002 (FIPS 180-2). Pomimo pewnego podobieństwa do SHA-1, skuteczne ataki przeprowadzone na SHA-1 nie zostały rozszerzone na funkcje SHA-2. Standard SHA-2 można uważać za ulepszoną wersję SHA-1.

Porównanie SHA-1 i SHA-2

Algorytm		Długość skrótu	Rozmiar bloku danych	Max. rozmiar wiadomości	Rundy	Relatywna szybkość przetwarzania
SHA1		160	512	$2^{64}-1$	80	1.0
SHA-2	**SHA-224/SHA-256**	224/256	512	$2^{64}-1$	64	1.38
	SHA-384/SHA-512	384/512	1024	$2^{128}-1$	80	1.56

Algorytm SHA-2 jest do dzisiaj algorytmem bezpiecznym.

Przykłady funkcji skrótu

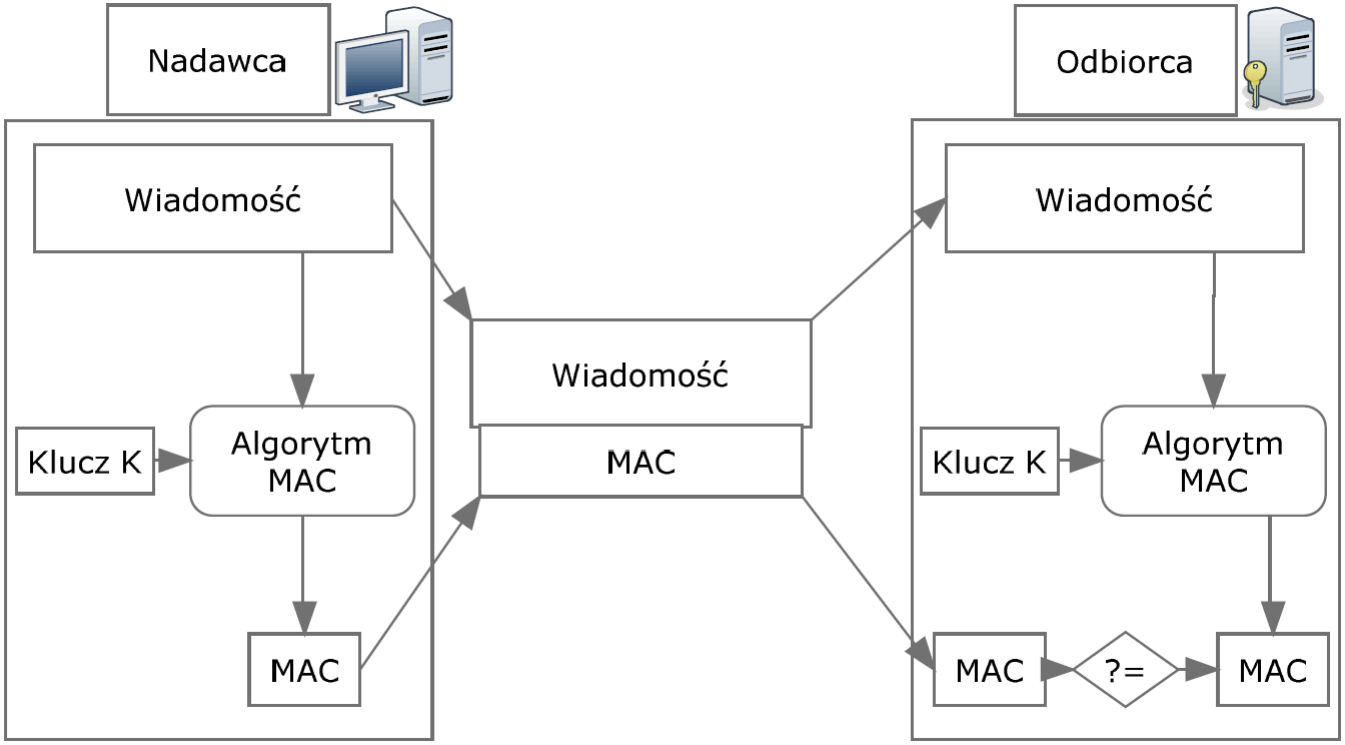
SHA-3 (Secure Hash Algorithm version 3)

W roku 2007 NIST ogłosił konkurs na nowy standard w zakresie tworzenia skrótów. Ponieważ SHA-2 jest nadal algorytmem bezpiecznym, SHA-3 jest uważany **nie** za następcę SHA-2 ale jako **alternatywa**. Konkurs zakończył się w roku 2012 i wygrał go algorytm Keccak opracowany przez G. Bertoni, J. Daemen, M. Peetersa oraz G.V. Assche. Od roku 2012 jest nazywany także SHA-3 (**FIPS 202** z 12.08.2015 definiuje SHA3-224, SHA3-256, SHA3-384, SHA3-512). Algorytm oferuje wybór szybkość/bezpieczeństwo.

SHA-3 nie jest modyfikacją SHA-2 lecz nowym rozwiązaniem. Keccak wykorzystuje tzw. konstrukcję gąbkową (ang. *spong*). Bloki danych są stopniowo „wchłaniane" i mieszane z tablicą stanu (jej zawartość jest inicjowana samymi zerami), bloki wyjściowe natomiast – stopniowo „wyciskane" z tablicy. Po każdym wchłanianiu lub wyciskaniu wykonywane są operacje w ramach tzw. funkcji rundy *f*[*b*], gdzie *b* może przyjmować jedna z liczb ze zbioru {25, 50,100, 200, 400, 800, 1600}. Liczby te określają zakres permutacji i związaną z nimi długość ***w*** tablicy stanu funkcji *f,* przy czym ***w=b/25***. Długość tablicy stanu może zatem wynosić *w*={1, 2, 4, 8, 16, 32, 64}. Stan SHA-3 zawarty jest w **tablicy trójwymiarowej 5x5x*w* (*w* tablic o wymiarach 5x5)**.

Kody uwierzytelniające wiadomość (MAC)

MAC (Message Authentication Code) jest także funkcją skrótu ale jej wartość zależy nie tylko od wiadomości ale również od **tajnego klucza**. MAC jest używany do uwierzytelnienia i weryfikacji integralności wiadomości. Aby dokonać uwierzytelnienia Nadawca i Odbiorca muszą dysponować **tym samym tajnym kluczem**.



Podstawy kryptografii

7. Przykłady użycia metod kryptograficznych w praktyce

7.1. Uwierzytelnianie
7.2. System GSM
7.3. System UMTS
7.4. Protokoły SSL i TLS
7.5. Protokół SSH

Uwierzytelnianie

Kluczowym zagadnieniem dla bezpieczeństwa systemów telekomunikacyjnych i informatycznych jest zapewnienie dostępu do systemu i zasobów tylko osobom do tego uprawnionym. W systemie musi więc być wbudowany mechanizm sprawdzania, czy użytkownik podający się za Alicję, naprawdę nią jest. Do tego celu służy **uwierzytelnianie** (*ang.* authentication) nazywane czasami identyfikacją użytkownika, źródła itp.

Uwierzytelniania nie należy mylić z **autoryzacją** (*ang.* authorization), której celem jest sprawdzenie, czy zidentyfikowany użytkownik ma *uprawnienia* do skorzystania z określonych zasobów lub usług.

W systemach teleinformatycznych procesy autoryzacji i uwierzytelniania są często połączone, tj. w tej samej procedurze następuje sprawdzenie tożsamości oraz uprawnień. Część autorów taki łączny proces nie do końca poprawnie nazywa autoryzacją.

Warto także zwrócić uwagę na to, że terminy **„uwierzytelnianie komunikatów”** czy **„uwierzytelnianie wiadomości”** mają znaczenie szersze niż termin „uwierzytelnianie”, gdyż obejmują także procedury sprawdzania **integralności** komunikatu/wiadomości.

Uwierzytelnianie

Podstawowe metody uwierzytelniania to:

- Hasła
- Osobiste numery identyfikacyjne PIN (*ang*. Personal Identification Number)
- Protokoły challenge-response
  - z kluczem tajnym
  - z kluczem publicznym
- Kody MAC (kryptograficzne sumy kontrolne)
- Podpisy cyfrowe
- Dowody z wiedzą zerową
- Funkcje fizycznie nieklonowalne PUF (*ang.* Physically Unclonable Function)

Uwierzytelnianie

Hasła

Hasła nie są przechowywane w sposób jawny lecz w postaci zaszyfrowanej, najczęściej jako skrót obliczony dla hasła i dołączonej liczby losowej (w rzeczywistości postrzeganej jako losowa). Proces dodawania dodatkowej informacji, która ma utrudnić skuteczny atak jest nazywany soleniem (*ang.* salting), a sama dodatkowa informacja solą (*ang.* salt).

PIN

Odmianą hasła jest PIN używany w przypadku kart kredytowych, bankowych, czy tzw. tokenów. Jest to zwykle liczba czterocyfrowa (czasem ośmiocyfrowa), która ma zabezpieczać przed użyciem karty przez osoby niepowołane, np. złodzieja.

Uwierzytelnianie

Protokoły challenge-response

Idea tego sposobu identyfikacji polega na odpowiedzi Alicji na wyzwanie przesłane przez Bolka, która przekona Bolka, że ma do czynienie rzeczywiście z Alicją. Przesyłanym wyzwaniem jest tutaj liczba losowa (postrzegana jako losowa) lub pseudolosowa.

Protokół challenge-response z tajnym kluczem

1. Alicja i Bolek dysponują takim samym tajnym kluczem *K* (algorytm symetryczny) oraz umówili się jakiej funkcji skrótu *H* będą używać.

2. Alicja komunikuje się z Bolkiem przedstawiając się jako Alicja

3. Bolek generuje liczbę losową R_B i wysyła ją Alicji

4. Alicja oblicza $H(K, R_B)$ i przesyła wynik Bolkowi

5. Bolek także oblicza $H(K, R_B)$ i jeśli wynik zgadza się z wynikiem przysłanym przez Alicję to tożsamość Alicji zostaje potwierdzona.

Uwierzytelnianie

Protokół challenge-response z kluczem publicznym

1. Alicja komunikuje się z Bolkiem przedstawiając się jako Alicja

3. Bolek generuje liczbę losową R_B i wysyła ją Alicji

4. Alicja szyfruje liczbę R_B używając swojego klucza prywatnego, a wynik przesyła Bolkowi

5. Bolek deszyfruje kryptogram otrzymany od Alicji używając jej klucza publicznego i jeśli wynikiem jest liczba R_B to tożsamość Alicji zostaje potwierdzona.

Uwierzytelnianie

Kody MAC (kryptograficzne sumy kontrolne)

Uwierzytelnianie za pomocą kodu MAC polega na wykorzystaniu tajnego klucza *K* do wytworzenia niewielkiego bloku danych zależnego od tajnego klucza oraz wiadomości wytwarzanej przez uwierzytelniane źródło. Otrzymany blok danych **nazywamy kodem MAC lub kryptograficzną sumą kontrolną.** MAC razem z oryginalną wiadomością jest przesyłany do odbiorcy. Zakładamy przy tym, że klucz *K* znany jest odbiorcy. Do uzgodnienia klucza K można wykorzystać mechanizmy kryptografii asymetrycznej. Jeśli odbiorca na podstawie odebranej wiadomości otrzyma taki sam MAC co odebrany, to uznajemy źródło za uwierzytelnione. Funkcja użyta do obliczania jest podobna do funkcji szyfrującej ale nie musi być odwracalna.

Uwierzytelnianie

Kody MAC (kryptograficzne sumy kontrolne) cd.

Ponieważ zakładamy, że nikt inny poza nadawcą i odbiorcą nie zna klucza *K*, to zgodność kodu MAC obliczonego przez adresata i otrzymanego oznacza, że wiadomość pochodzi od oczekiwanego nadawcy, ponieważ tylko osoba znająca klucz *K* mogła wyprodukować poprawny kod MAC dla otrzymanego komunikatu. Ta metoda uwierzytelniania nadawcy zapewnia także **uwierzytelnienie wiadomości**: integralność danych nie została naruszona, ponieważ intruz nie znający klucza *K* nie byłby w stanie wyprodukować poprawnego kodu MAC dla zmienionego komunikatu.

Jeśli w strukturze komunikatu znajduje się numer sekwencyjny, za pomocą którego protokoły weryfikują właściwą kolejność bloków wiadomości, co ma miejsce np. w przypadku protokołów X.25 i TCP, adresat może być pewien, że kolejność ta nie została zmieniona przez intruza.

Uwierzytelnianie

Kody MAC (kryptograficzne sumy kontrolne) cd.

Obecnie istnieją protokoły MAC bazujące zarówno na szyfrach, np. DAA – Data Authentication (wykorzystanie DES w trybie CBC, FIPS 113), CMAC – Cipher-based Message Authentication, (wykorzystanie 3DES lub AES, FIPS 800-38B) jak i skrótach, np. **HMAC** (hash MAC) opisany w dokumencie RFC 2104. Stanowi on obowiązkowy komponent bezpieczeństwa protokołu **IP Security**. Wykorzystywany jest także przez inne protokoły internetowe, m.in., przez **SSL** (FIPS 198). Cechą charakterystyczna HMAC jest łatwość wykorzystania różnych dostępnych funkcji haszujących, także SHA-3.

Podpisy cyfrowe

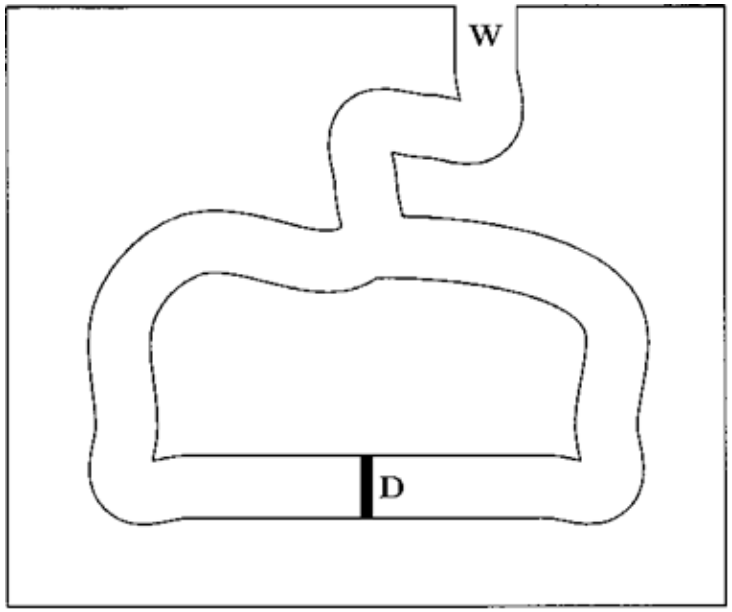
Podpisy cyfrowe należą do najbardziej oczywistych sposobów uwierzytelnienia. W procedurach uwierzytelniania stosowanych w sieciach teleinformatycznych są jednak używane rzadko ze względu na złożoność i czas wykonywania/weryfikacji podpisu. Dominują w **usługach** związanych z obrotem elektronicznym, dla realizacji których wykorzystuje się sieci teleinformatyczne.

Uwierzytelnianie

Dowody z wiedzą zerową

Jedną z metod uwierzytelniania są tzw. dowody z wiedzą zerową. Należą one do kategorii metod probabilistycznych, a więc takich w których tożsamość Alicji **jest potwierdzana z pewnym prawdopodobieństwem ale akceptowalnym.**

Przypuśćmy, że Alicja chce przekonać Bolka, że zna pewien sekret, ale nie chce zdradzić samego sekretu. Alicja twierdzi, że potrafi otworzyć drzwi zamykające przejście w jaskini. Bolek stoi przy wejściu do jaskini



Uwierzytelnianie

- Alicja wchodzi do jaskini i idzie albo w lewo albo w prawo dochodząc do drzwi zamykających przejście
- Bolek dochodzi do rozwidlenia korytarza, rzuca monetą i w zależności od wyniku rzutu krzyczy, nakazując Alicji wyjść albo z lewego korytarza albo z prawego
- Alicja wykonuje polecenie Bolka, otwierając drzwi jeśli to konieczne
- Doświadczenie takie **powtarzają n-krotnie**. Jeśli *n* jest dostatecznie duże, to prawdopodobieństwo tego, że Alicja wykona polecenia Bolka nie potrafiąc otworzyć drzwi jest znikomo małe ($1/2^n$).

W dowodach z wiedzą zerową dowodzący dowodzi, że zna pewną tajemnicę, której weryfikujący nie zna, np. umie obliczać pierwiastki kwadratowe, wartość logarytmu dyskretnego itp., bez ujawniania tajemnicy. W czasie dowodzenia weryfikujący nie dowiaduje się niczego poza tym, że dowodzący zna tę tajemnicę. Nie otrzymuje żadnych informacji o tej tajemnicy.

Uwierzytelnianie

Przykład. Protokół Fiata-Shamira

Bezpieczeństwo tego protokołu opiera się na trudności obliczeniowej pierwiastków kwadratowych modulo *n*, gdzie *n* jest iloczynem dwóch liczb pierwszych (uważa się, że znajdywanie pierwiastków kwadratowych jest wówczas trudne obliczeniowo). Alicja chce przekonać Bolka, że umie obliczać pierwiastki kwadratowe bez ujawniania metody. W tym celu odpowiada na zapytania Bolka.

- Bolek wybiera dwie duże i różne liczby pierwsze *p i q,* oblicza ich iloczyn $n = pq$
- Alicja po otrzymaniu od Bolka *n* wybiera losową liczbę *s* (tajną), względnie pierwszą z *n* i oblicza liczbę $v = s^2 \pmod{n}$. Wynik przekazuje Bolkowi
- Liczby *n* i *v* są powszechnie dostępne jako identyfikatory tożsamości Alicji (klucz publiczny Alicji). Kluczem tajnym jest liczba s.
- Alicja wybiera losowo liczbę *r* względnie pierwszą z *n*, oblicza $x = r^2 \pmod{n}$ i wysyła *x* Bolkowi
- Bolek wysyła Alicji losowy bit *b*

Uwierzytelnianie

Protokół Fiata-Shamira cd.

- Alicja wysyła Bolkowi
  r jeśli $b = 0$
  $y = r \bullet s \pmod{n}$ jeśli $b = 1$
- Bolek sprawdza czy
  $x = r^2 \pmod{n}$ jeśli $b = 0$
  $y^2 = x \bullet v \pmod{n}$ jeśli $b=1$

Pierwsza równość dowodzi, że Alicja zna pierwiastek kwadratowy z x (r dostarczone przez Alicję dla b=0 oraz przesłane do Bolka x dają równość), druga natomiast dowodzi, że Alicja zna też tajne s – *uwierzytelnienie Alicji* (y otrzymane od Alicji dla b=1 i podstawione do wzoru daje tę sama resztę z dzielenia co iloczyn liczb x i v również otrzymanych od Alicji). Protokół ten powtarza się t razy (wystarczy zmieniać r i b, a s pozostawić b/z), wtedy prawdopodobieństwo oszustwa przez Alicję wynosi $\mathbf{1/2^t}$.

Uwierzytelnianie

Funkcje fizycznie nieklonowalne (*ang.* Physically Unclonable Funtions)

Funkcja fizycznie nieklonowalna to taka funkcja, która daje tę samą odpowiedź (*response*) na to samo pobudzenie (*challange*), **ale tylko w jednym układzie**. Sama odpowiedź ma być nieprzewidywalna. Oczekujemy więc nieprzewidywalnej, ale powtarzalnej reakcji tylko na to samo pobudzenie w tym samym układzie oraz innej odpowiedzi na to samo lub inne pobudzenie, w dowolnym innym układzie z zaimplementowaną tą samą funkcją.

Obecnie nieklonowalność funkcji osiąga się na etapie procesu technologicznego, a sama funkcja może być wykorzystana do wytwarzania tego samego ciągu losowego w odpowiedzi na to samo pobudzenie ale tylko w tym samym układzie.

W praktyce wykorzystuje się:

- specjalne materiały (*ang.* cover-based PUF),
- opóźnienia, a ściślej nieklonowalność opóźnień (*ang*. delay-based PUF),
- właściwości zaimplementowanych pamięci (*ang*. memory-based PUF).

Spośród wymienionych podejść największe bezpieczeństwo oferuje wykorzystanie specjalnych materiałów. Jest jednocześnie najbardziej skomplikowane i kosztowne.

Uwierzytelnianie

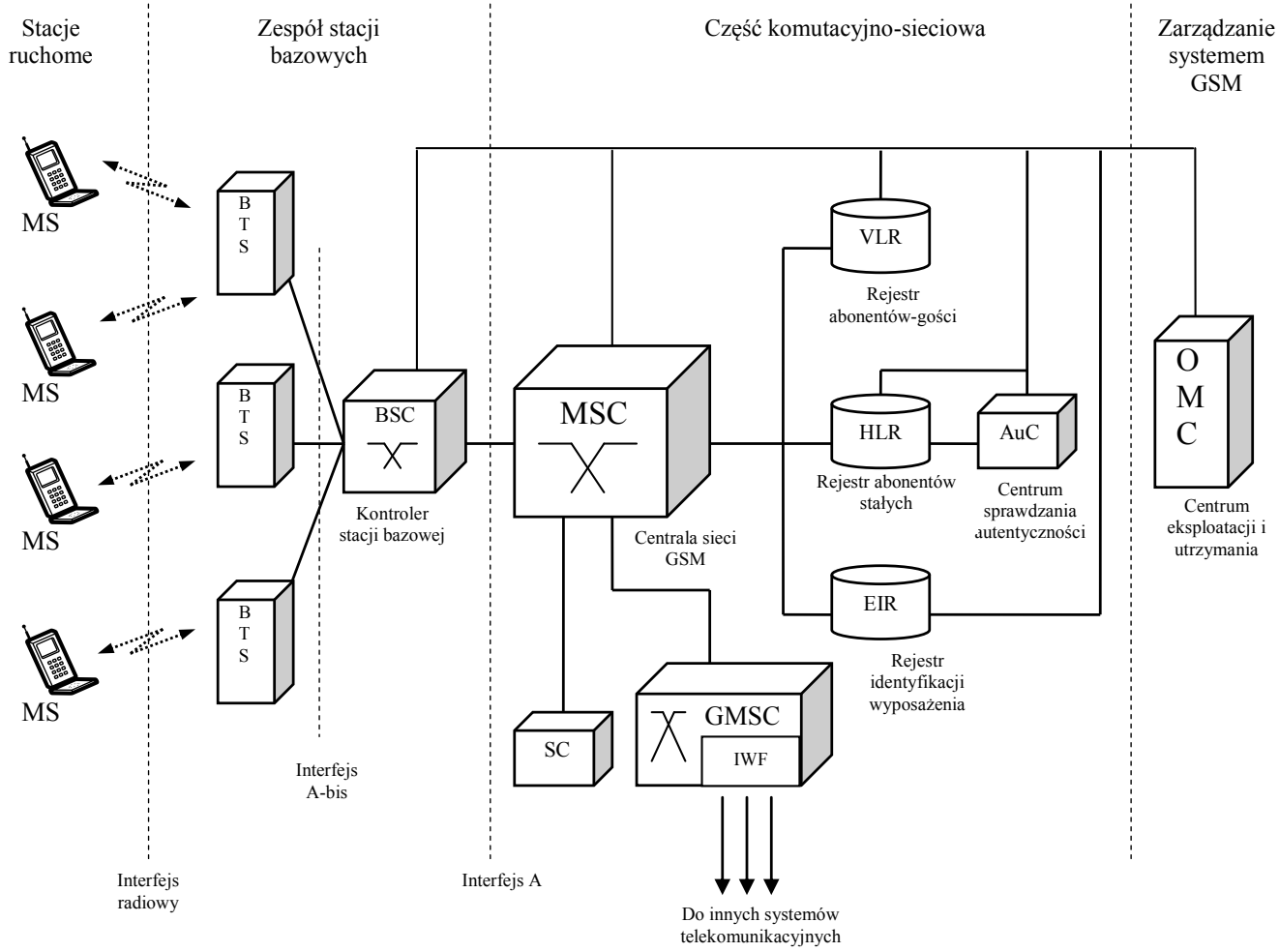
Funkcje fizycznie nieklonowalne (*ang.* Physically Unclonable Funtions)

Obecnie nie potrafimy skonstruować poza procesem technologicznym funkcji PUF, która wytworzy tylko dla konkretnej konstrukcji układowej nieprzewidywalną ale powtarzalną odpowiedź **długoterminową** na to samo pobudzenie (*ang.* circuit-level PUF). Otrzymywanie powtarzalnych **krótkich** odpowiedzi, np. 64 bitowych, na to samo pobudzenie z jednoczesną nieklonowalnością jest tym łatwiejsze im mniej bitów chcemy otrzymać.

Konstruowanie funkcji PUF z dostępnych elementów, tj. poza procesem technologicznym (przełamanie monopolu kilku firm technologicznych), **jest jednym z najważniejszych zadań współczesnej kryptografii**.

Zastosowanie funkcji PUF jest bardzo szerokie, zaczynając od identyfikacji pojedynczych układów, urządzeń, uwierzytelniania osób lub sprzętu, poprzez bankowość elektroniczną, a kończąc na specjalistycznych metodach ochrony informacji, łącznie z wytwarzaniem liczb losowych.

System GSM



Architektura systemu GSM (fr. *Groupe Speciale Mobile*)

System GSM

Dane na karcie SIM:

- **stałe** – zapisywane podczas tworzenia karty i opisują konto użytkownika, np. klucz autoryzacji K_i, IMSI (ang. *International Mobile Subscriber Identity*), klasa dostępu;
- **tymczasowe** – zapisywane na karcie podczas normalnej pracy telefonu w sieci, np. TMSI (ang. *Temporary Mobile Subscriber Identity*), LAI (ang. *Local Area Identity*), klucz szyfrujący K_c (obliczany wg. liczb RAND i K_i), lista zabronionych sieci ruchomych (bez umów roamingowych), krótkie wiadomości tekstowe SMS odebrane podczas nieobecności abonenta, lista skróconych telefonów;
- **związane z usługami** – np. preferencje językowe.

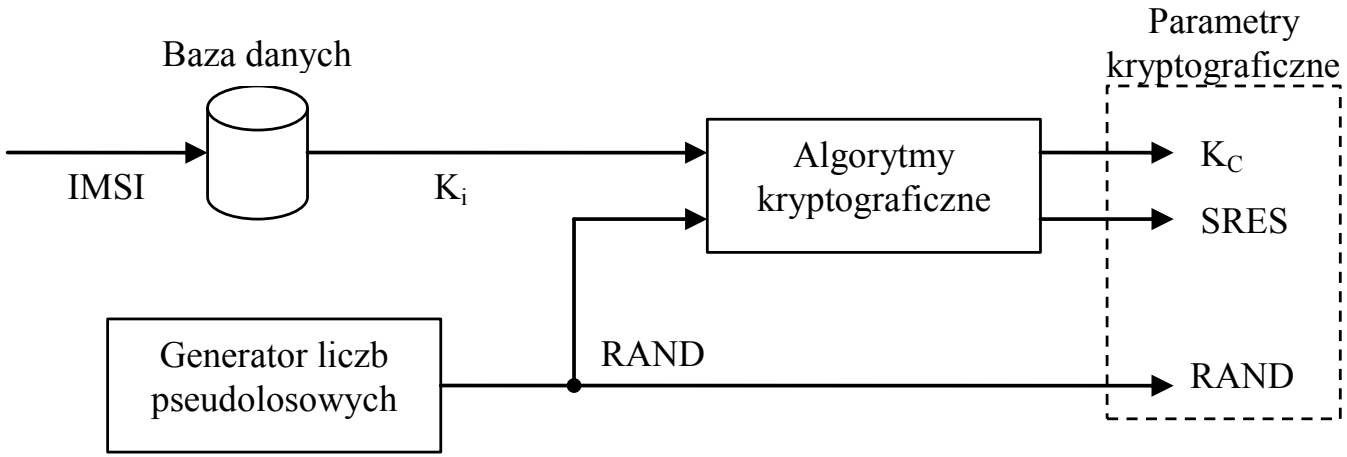
Z punktu widzenia bezpieczeństwa kluczowe dane to:

- międzynarodowy numer abonenta ruchomego **IMSI**,
- klucz identyfikacyjny $\boldsymbol{K_i}$ abonenta (długość nie podlega standaryzacji),

Obie liczby są przechowywane **tylko na karcie SIM oraz w AuC**. Głównym zadaniem AuC jest wygenerowanie za pomocą klucza K_i tzw. trypletu: Liczby **RAND**, liczby **SRES** (ang. *Signed RESponce*) oraz klucza szyfrującego $\boldsymbol{K_c}$.

System GSM

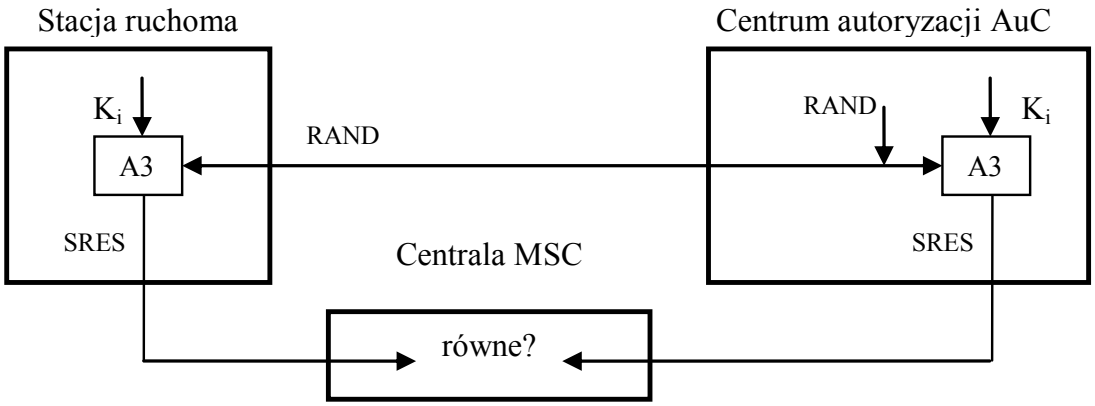
Wytwarzanie trypletu przez AuC:



Tryplet: RAND (128 bitów), SRES (32 bity) oraz K_c (długość klucza nie jest standaryzowana) przechowywany jest w rekordzie abonenta w rejestrze HLR, skąd na żądanie przesyła się je do rejestru VLR centrali MSC, w której obszarze aktualnie znajduje się dany abonent. **Dla każdego połączenia, AuC generuje nowy zestaw parametrów (RAND, K_c, SRES).**

System GSM

Identyfikacja abonenta (Algorytm A3 – nie podlega standaryzacji):

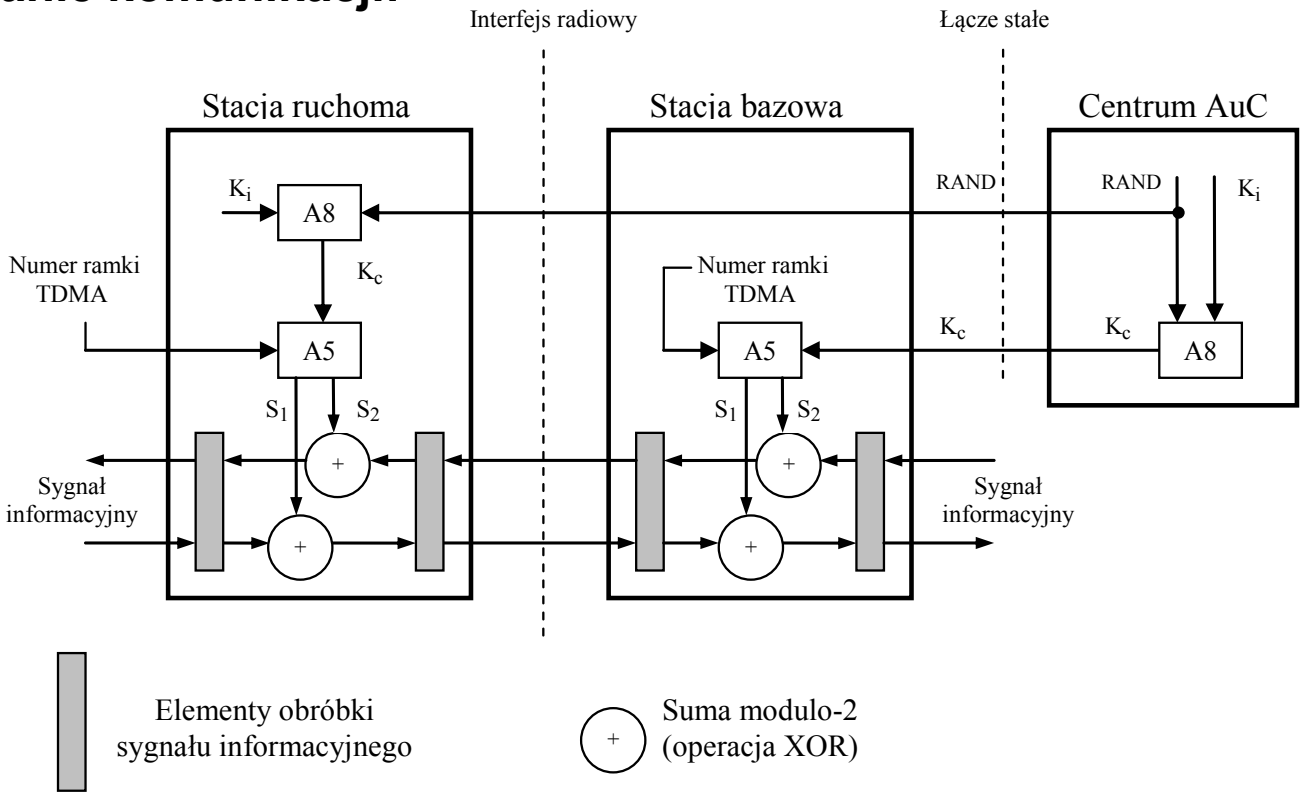


Proces identyfikacji abonenta

Przyczyn braku zestandaryzowania algorytmu A3 jest wiele. Przede wszystkim algorytm A3 jest kluczowym algorytmem do zabezpieczenia całego systemu, gdyż stanowi podstawę zabezpieczenia dostępu do usług. Tak więc obawiano się, że podczas przekazywania tego algorytmu operatorom w różnych krajach może dojść do przecieków informacji.

System GSM

Szyfrowanie komunikacji:



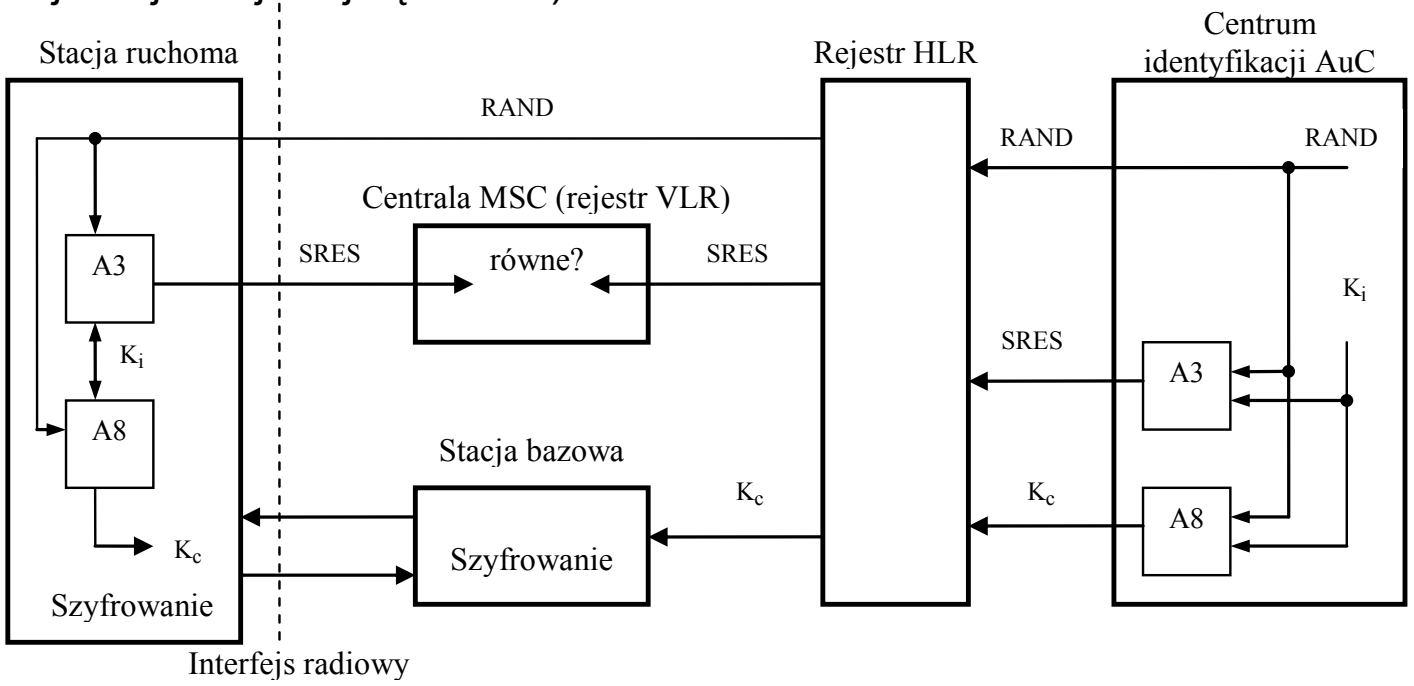
Algorytm A8 – wytworzenie klucza szyfrującego K_c (standardowa długość to 64 bity) na podstawie klucza K_i z karty SIM)

Algorytm A5 – wytwarzanie sekwencji szyfrującej S_1 („w górę”) oraz S_2 („w dół”); korzystamy z klucza K_c oraz numeru ramki TDMA;

Szyfrowanie – XOR 114 bitów informacyjnych i 114 bitów (liter) klucza bieżącego

System GSM

Podobnie jak A3, algorytm A8 nie podlega standaryzacji. Możliwa jest nawet wspólna realizacja A3 i A8 (**często jest to wspólny algorytm COMP128, którego produktem jest liczba 128 bitowa; 32 najstarsze bity tej liczby to SRES, a 54 najmłodsze bity, (uzupełnione do 64 bitów dziesięcioma zerami), to klucz K_c.**). Nowsze algorytmy to COMP128-2, COMP128-3, COMP128-4 (oparty na algorytmie MILANGE z UMTS).
Okres powtarzania numeru ramki (22 bity) to ok. 3,5 godziny (czas „bezpiecznej" komunikacji dla jednej sesji łączności).



Procedury kryptograficzne w systemie GSM - podsumowanie

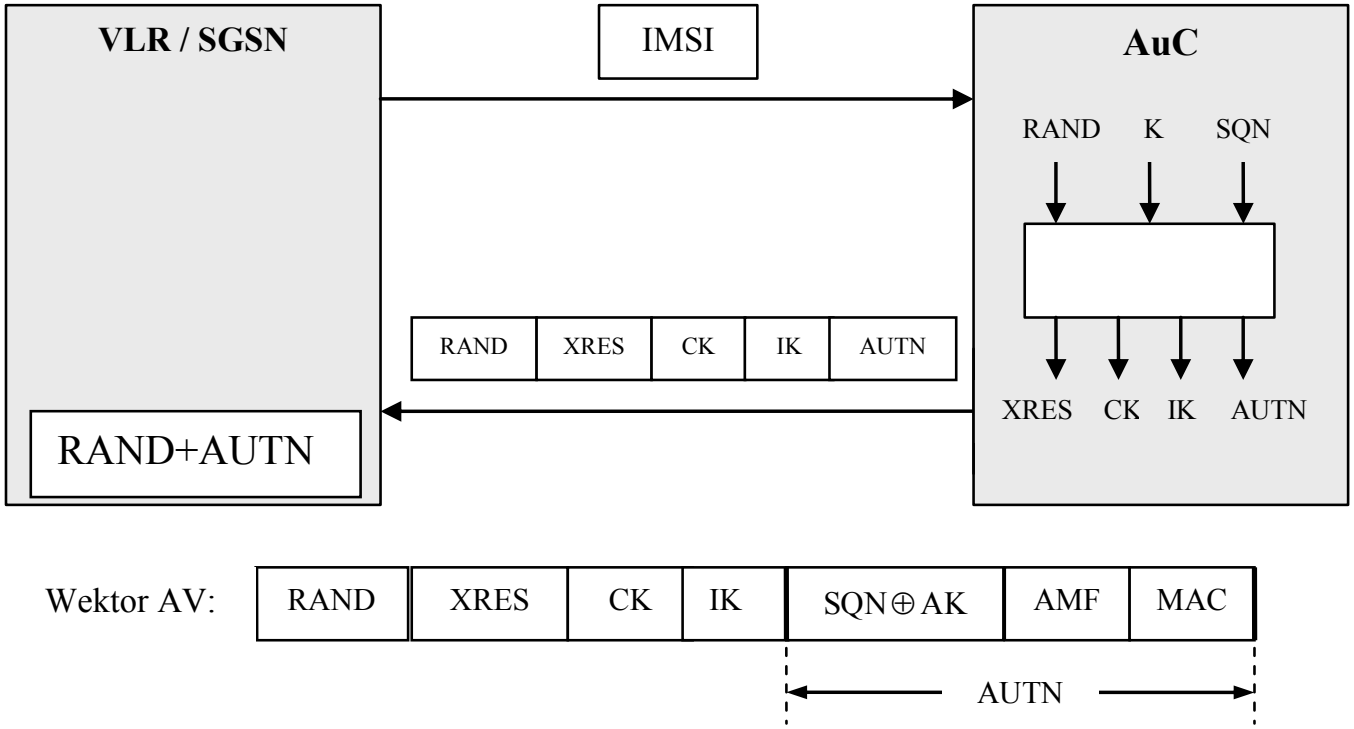
System UMTS

W systemie UMTS (ang. *Universal Mobile Telecommunications System*) wprowadzono procedurę **wzajemnego uwierzytelnienia** stacji ruchomej oraz sieci. Idea wzajemnego uwierzytelnienia polega na tym, że sieć obsługująca SN (ang. *Serving Network*) sprawdza tożsamość abonenta (tak jak w systemie GSM) za pomocą techniki „żądania i odpowiedzi", a w tym czasie stacja ruchoma upewnia się czy **dana sieć obsługująca przeszła pozytywnie uwierzytelnienie wobec sieci rodzimej HE** (ang. *Home Environment*) danego użytkownika. Taka procedura wykonuje się **zawsze podczas logowania się stacji ruchomej do nowej sieci oraz przy aktualizacji informacji o położeniu abonenta**. Druga faza uwierzytelnienia, podczas której stacja ruchoma sprawdza czy jest podłączona z legalną siecią **jest unikatowa dla systemu UMTS**.

System UMTS

Za wzajemne uwierzytelnienie oraz uzgadnianie kluczy kryptograficznych w systemie UMTS odpowiedzialny jest **mechanizm AKA (ang. *Authentication and Key Agreement*). Podstawowym elementem tej procedury jest 128-bitowy tajny klucz *K***, który jest współdzielony między centrum AuC i modułem USIM danego abonenta. **Klucz ten nigdy nie jest transmitowany z tych dwóch lokalizacji**. Dodatkowo każdy moduł USIM (ang. *UMTS Subscriber Identity Module*) ma licznik sekwencji SQN (ang. *Sequence Number*), który numeruje kolejne zaakceptowane wektory uwierzytelniające wygenerowane dla danego abonenta. W centrum AuC przechowywana jest aktualna wartość tego licznika. Licznik sekwencji SQN zapobiega użyciu starych wektorów uwierzytelniających, które mogłyby być wykorzystane do ataku na system zabezpieczeń

System UMTS



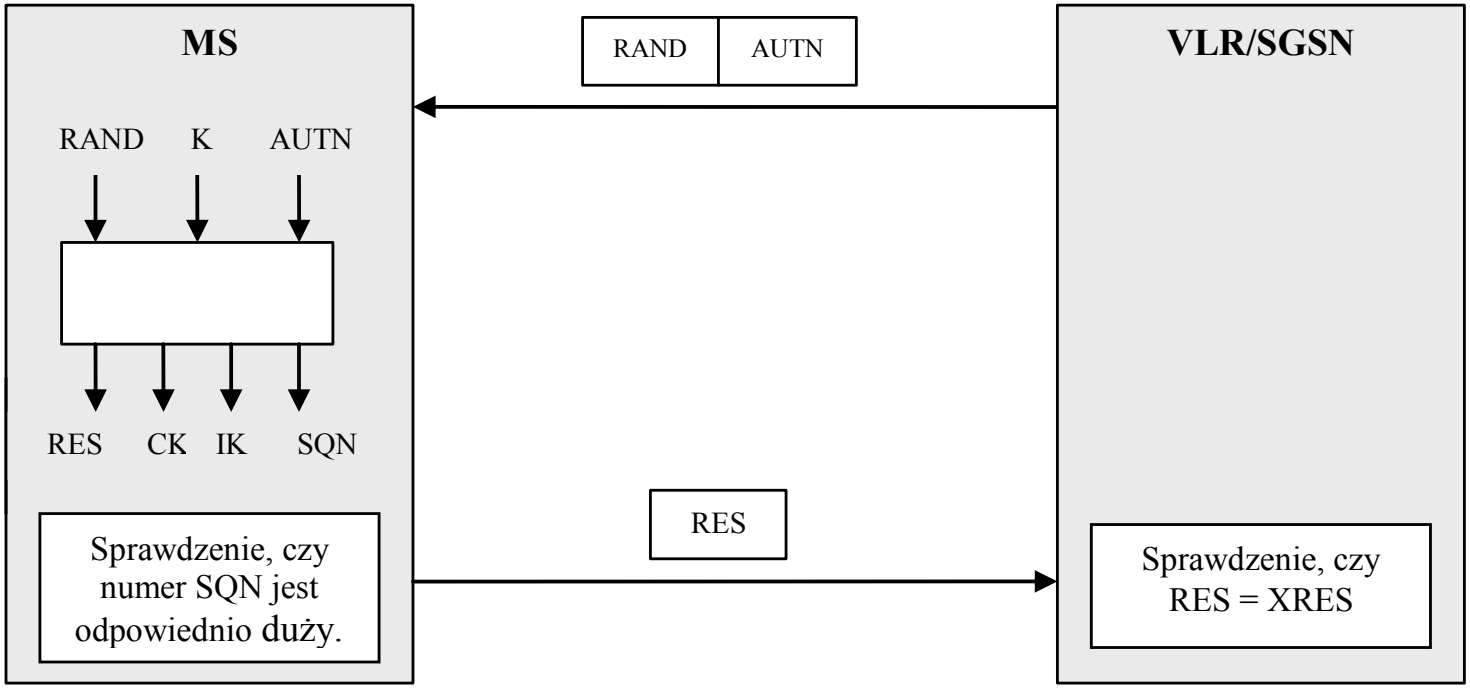
Pierwsza faza procesu wzajemnego uwierzytelnienia w systemie UMTS: wytworzenie wektora uwierzytelniającego AV (SGSN-*Serving GPRS Support Network*) przez sieć rodzimą HE. Przesłanie AV do SN i dalej przesłanie przez SN do terminala RAND i tokena AUT niezbędnych do uwierzytelnienia sieci: XRES – spodziewana odpowiedź (32-128 bitów), CK – klucz szyfrujący (128 bitów), IK – klucz integralności danych (128 bitów), AK – klucz anonimowości (64 bity), AMF – pole zarządzania uwierzytelnianiem, MAC – kod uwierzytelnienia wiadomości (64 bity).

System UMTS

Stacja ruchoma na podstawie otrzymanych danych wytwarza wektor AV, który powinien być kopią wektora wytworzonego w AuC.

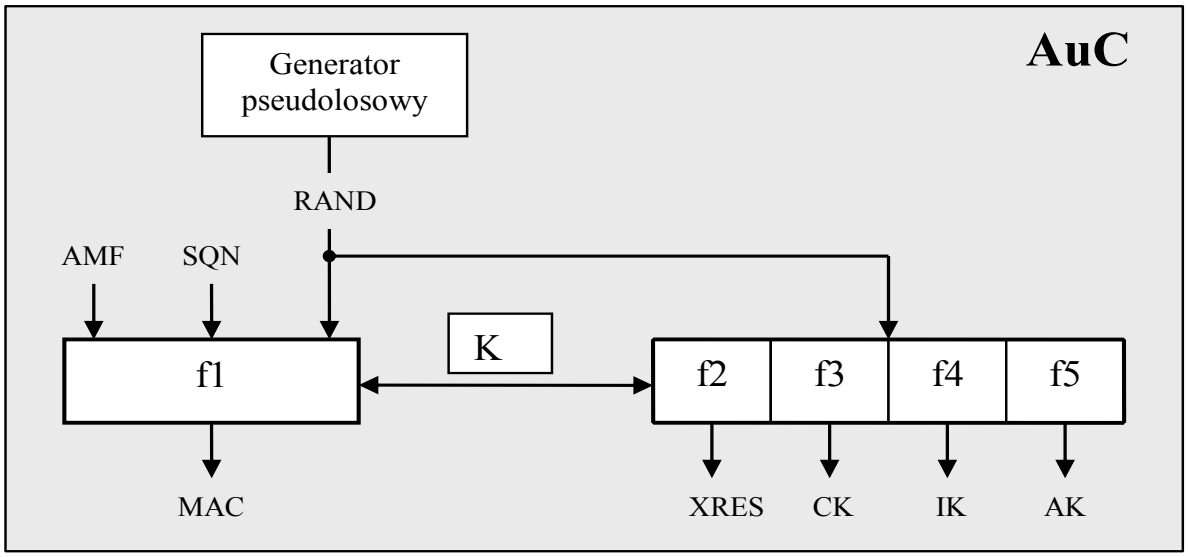
Wykorzystując **odpowiednie algorytmy zapisane na karcie USIM stacja ruchoma może sprawdzić, czy parametr AUTN był faktycznie wytworzony w centrum AuC (porównanie MAC obliczonego przez terminal (tzw. XMAC) z MAC otrzymanym z HE) oraz czy nie został już wcześniej do niej przesłany, czyli aktualny SQN jest większy od poprzedniego**. Jeżeli aktualny wektor uwierzytelniający AV przeszedł pozytywnie wszystkie testy, wówczas stacja ruchoma oblicza parametr RES (ang. *Response*) i przesyła go do VLR/SGSN jako odpowiedź autoryzacyjną użytkownika.

System UMTS



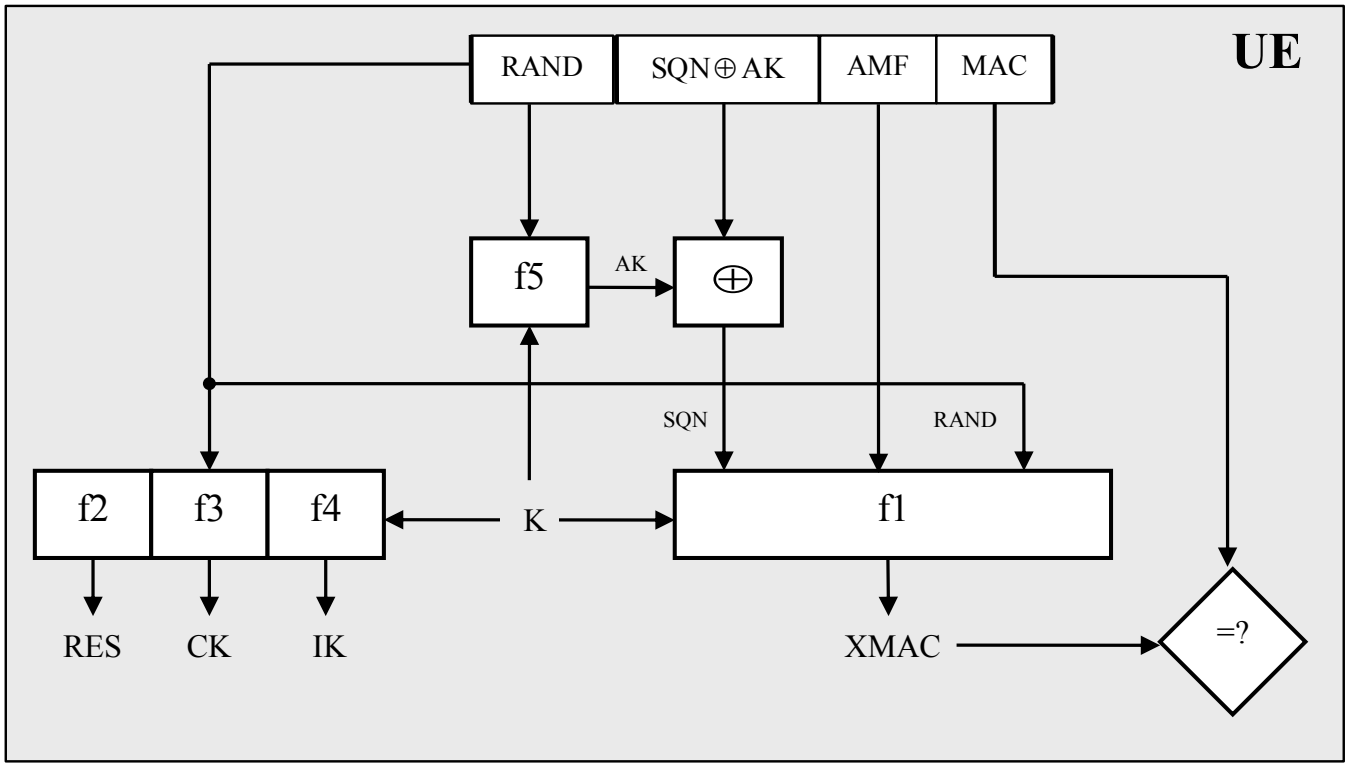
Druga faza wzajemnego uwierzytelnienia: uwierzytelnienie stacji ruchomej. Porównanie przez sieć obsługującą SN liczby RES otrzymanej od terminala i liczby XRES z sieci rodzimej HE.

System UMTS



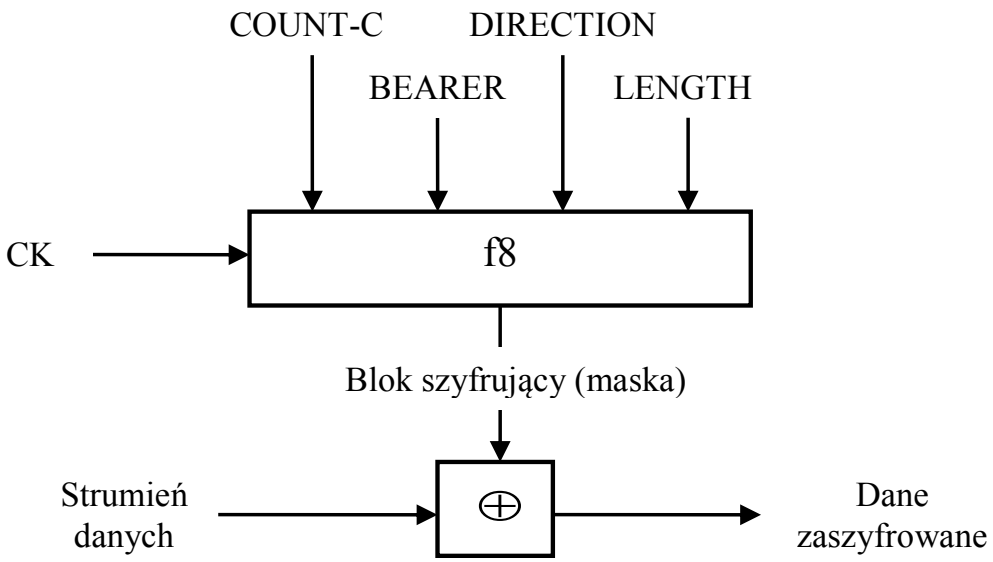
Wytwarzanie wektorów AV po stronie sieci rodzimej HE (f1, f2, f3, f4, f5 – funkcje jednokierunkowe)

System UMTS



Mechanizm AKA po stronie stacji ruchomej

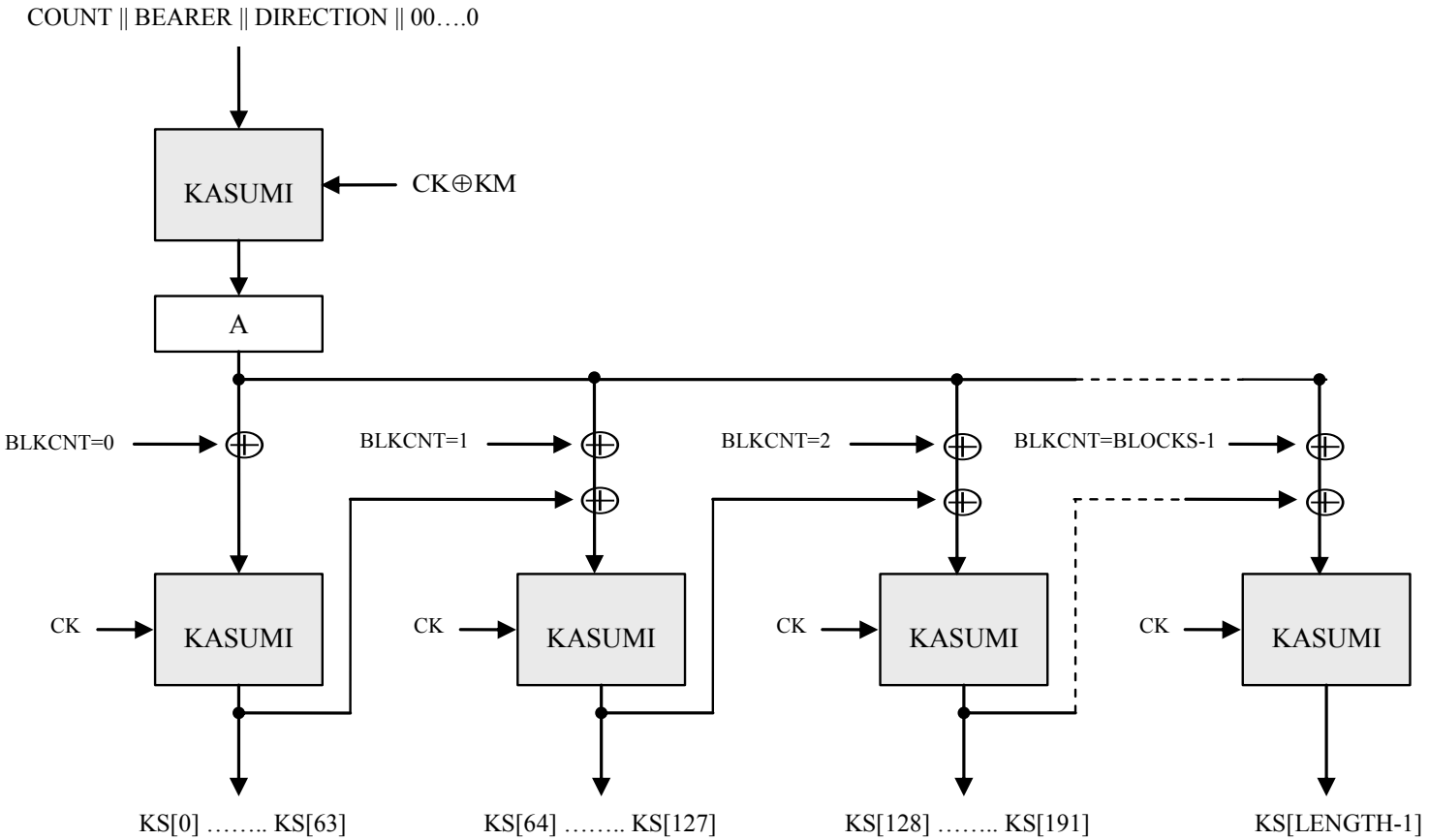
System UMTS



Szyfrowanie danych w podsystemie UTRAN (część radiowa).

CK – klucz szyfrujący. COUNT-C – 32-bitowy licznik oparty na numerze hiperramki HFN (ang. *Hyper Frame Number*), zmienia się dla każdego wygenerowanego bloku szyfrującego. BEARER – 5-bitowy identyfikator transmisji, który wyklucza sytuację ponownego wygenerowania tej samej sekwencji przy identycznych warunkach na wejściu algorytmu szyfrowania. DIRECTION – bit określający kierunek transmisji („w górę" czy „w dół"). LENGTH – parametr definiujący długość sekwencji szyfrującej.

System UMTS



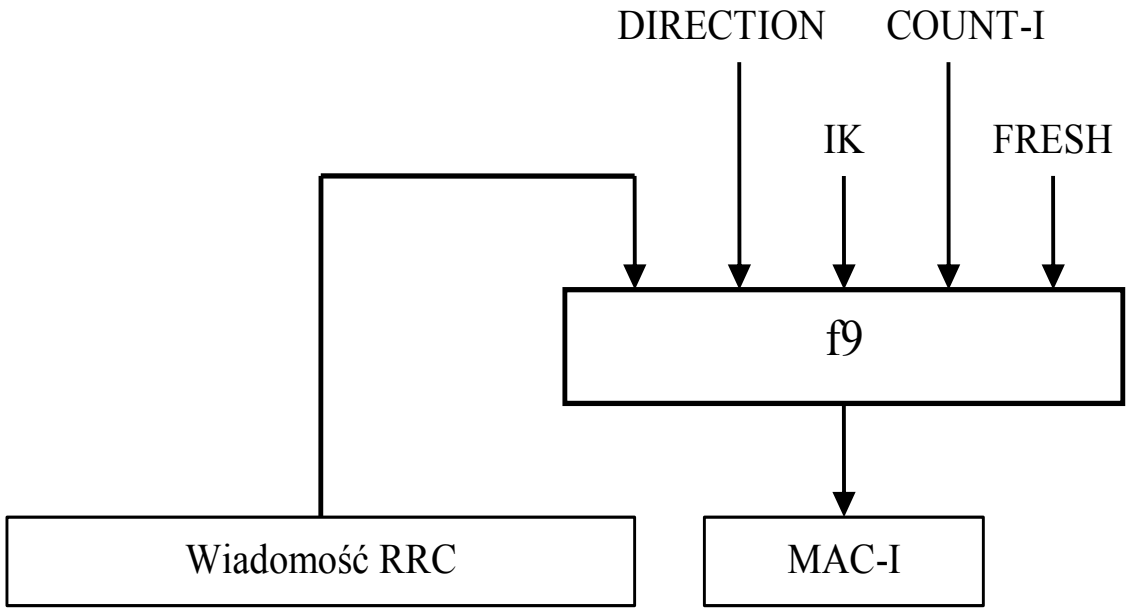
Funkcja szyfrująca f8. Szyfr blokowy KASUMI pracujący w trybie OFB. Klucz – 128 bitów, wyjście 64 bity, długość bloku danych od 1 do 20000 bitów zależnie od wartości parametru LENGTH (wartość IV określa 32 bitowy licznik + 5 bitowy parametr BEARER + 1 bit DIRECTION + 26 bitów zerowych).

System UMTS

Kontrola integralności informacji

Odbywa się między stacją ruchomą a kontrolerem RNC (droga radiowa). Służy do tego **jednokierunkowa funkcjia f9**, która wykorzystuje 128-bitowy klucz IK, wygenerowany podczas procedury AKA. Oprócz klucza IK, na wejście funkcji f9 podawana jest: przesyłana wiadomość (*RRC Message*), bit informujący o kierunku transmisji (DIRECTION), 32-bitowa wartość licznika opartego na numerze hiperramki RRC i kolejnym numerze ramki RRC (COUNT-I) oraz 32-bitowa liczba pseudolosowa generowana dla każdego abonenta na czas pojedynczego połączenia (FRESH). Dla każdej wiadomości RRC generowany jest 32-bitowy kod MAC-I (ang. *Message Authentication Code - Integrity*). Różnorodność wiadomości MAC-I zapewniona jest dzięki parametrowi FRESH, który za każdym razem jest inny.

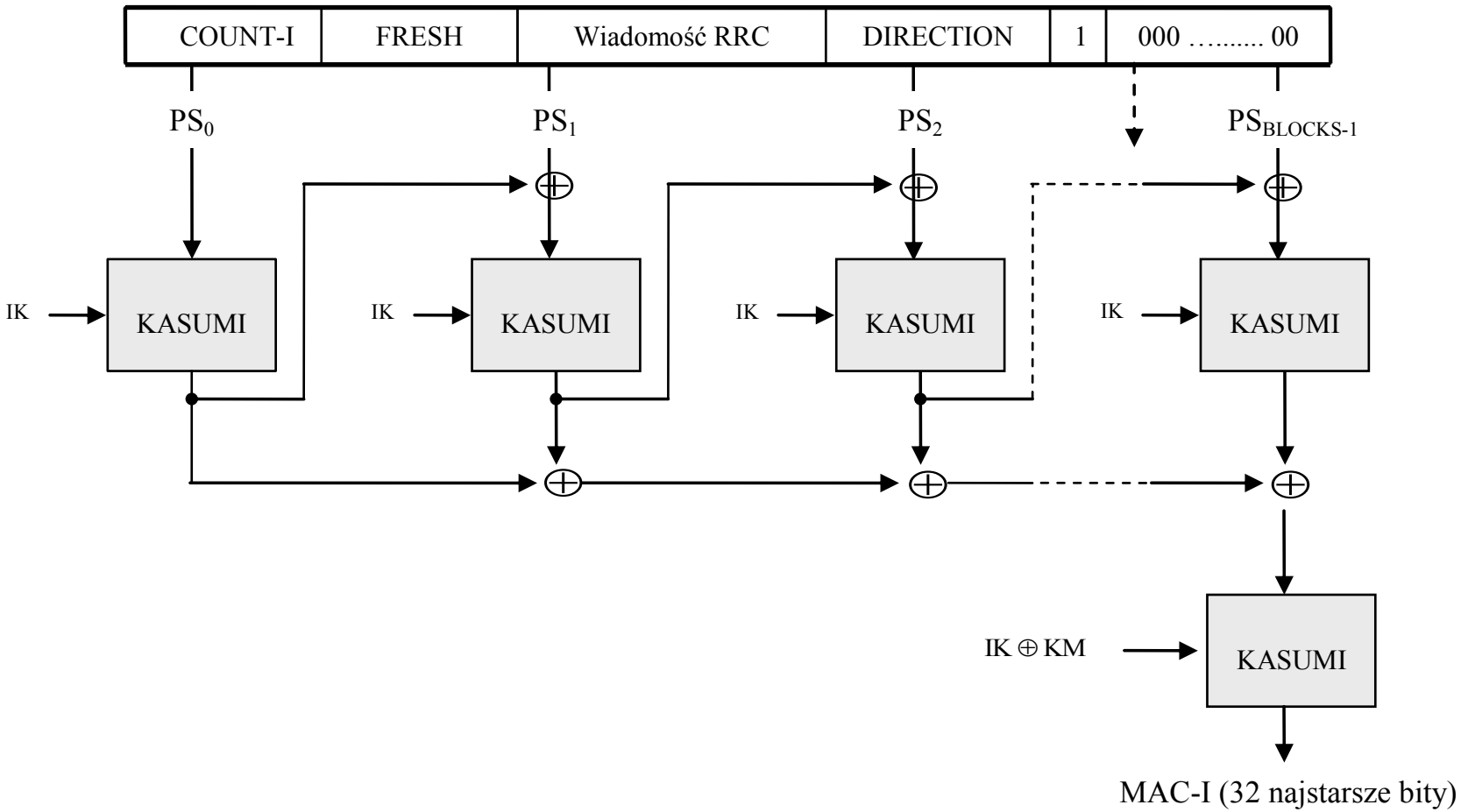
System UMTS



Obliczanie kodu MAC-I

Po stronie odbiorczej na podstawie odebranej wiadomości RRC wytwarzany jest kod MAC-I'. Jeśli MAC-I' i MAC-I są równe oznacza to, że podczas transmisji wiadomość RRC nie została zmieniona (celowo lub przypadkowo).

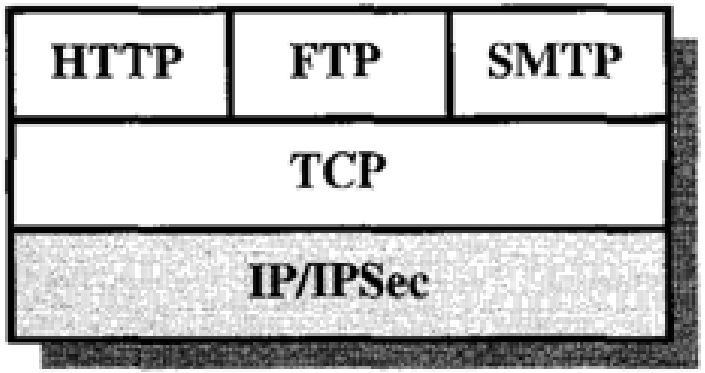
System UMTS



Funkcja kontroli integralności f9. Bity zerowe dodajemy po to, aby całkowita liczba bitów była wielokrotnością liczby 64. KM – stały klucz (128 bitowy) równy $16x(10101010)_2$.

Protokoły SSL i TLS

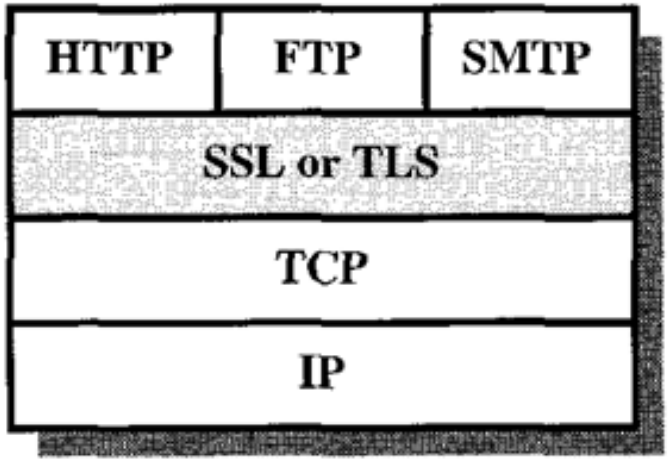
Bezpieczeństwo komunikacji w sieci Internet można zapewnić na wiele sposobów, np. w warstwie sieci, warstwie transportowej lub w warstwie aplikacji.



Usytuowanie protokołów związanych z bezpieczeństwem **w warstwie sieci. Za bezpieczeństwo odpowiada protokół IPSec** (Internet Protocol Security, IP Security) będący zbiorem protokołów służących implementacji bezpiecznych połączeń (encapsulacja pakietu IP);

HTTP – Hypertext Transfer Protocol (znormalizowany sposób komunikacji w sieci WWW, przesyłanie żądania udostępniania plików), FTP – File Transfer Protocol (dwukierunkowy transfer plików w układzie serwer FTP – klient FTP), SMTP – Simple Mail Transfer Protocol (znormalizowany protokół komunikacyjny dla przekazywania poczty elektronicznej w Internecie).

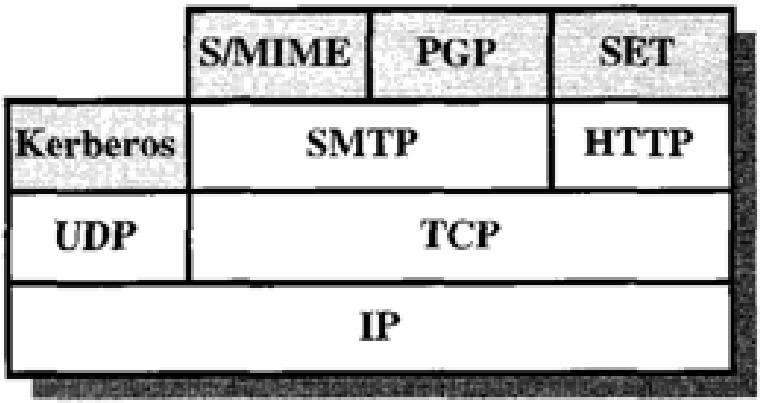
Protokoły SSL i TLS



Usytuowanie protokołów związanych z bezpieczeństwem **w warstwie transportowej (powyżej TCP). Za bezpieczeństwo odpowiada protokół SSL** (*Secure Socket Layer*) **lub TLS** (*Transport Layer Security).* Każdy z nich jest zbiorem protokołów służących bezpiecznej **komunikacji w sieci.**

HTTP – Hypertext Transfer Protocol (znormalizowany sposób komunikacji w sieci WWW, przesyłanie żądania udostępniania plików), FTP – File Transfer Protocol (dwukierunkowy transfer plików w układzie serwer FTP – klient FTP), SMTP – Simple Mail Transfer Protocol (znormalizowany protokół komunikacyjny dla przekazywania poczty elektronicznej w Internecie).

Protokoły SSL i TLS



Usytuowanie protokołów związanych z bezpieczeństwem **w warstwie aplikacji Za bezpieczeństwo odpowiada konkretna aplikacja np.** S/MIME (Secure /Multipurpose Internet Mail Extensions), PGP (Pretty Good Privacy), SET (Secure Electronic Transaction), oferująca uwierzytelnianie, niezaprzeczalność, integralność danych oraz szyfrowanie.

UDP – User Datagram Protocol (protokół pakietów użytkownika stosowany w warstwie transportowej modelu OSI), Kerberos – Protokół dedykowany uwierzytelnianiu i autoryzacji w sieci komputerowej wykorzystujący Centrum Dystrybucji Kluczy (wymaga ciągłego dostępu do centralnego serwera nazywanego Kerberos), SMTP – Simple Mail Transfer Protocol (znormalizowany protokół komunikacyjny dla przekazywania poczty elektronicznej w Internecie), HTTP – Hypertext Transfer Protocol (znormalizowany sposób komunikacji w sieci WWW, przesyłanie żądania udostępniania plików).

Protokoły SSL i TLS

Ulokowanie protokołów SSL i TLS powyżej TCP powoduje, że realizują one wiele celów związanych z bezpieczną komunikacją w sieciach komputerowych.

- Protokół SSL został opracowany przez firmę Netscape. Głównym zastosowaniem miała być bezpieczna komunikacja przeglądarki internetowej z serwerem (HTTPS).
- 1996 – SSL 3.0, publiczny standard Netscape, stosowany do dzisiaj.
- 1999 – TLS 1.0, oparty o SSL 3.0 standard IETF (RFC 2246) RFC, nazywany także SSL 3.1.
- 2006 – TLS 1.1 (RFC 4346).
- 2008 – TLS 1.2 (RFC 5246), rozwinięcie TLS 1.1.
- W 2009 roku odkryto podatność na atak typu „*man in the middle*”; ponieważ problem dotyczył samego protokołu a nie implementacji, problem rozwiązano poprzez usunięcie z protokołu możliwości renegocjacji.
- Obecnie klucze 128 bitowe (poprzednio 40 bitowe – postulat NSA).

RFC (ang. *Request for Comments/Contributions*) – zbiór technicznych i organizacyjnych dokumentów o charakterze memorandum. Idea dokumentów pojawiła się w roku 1969 wraz z koncepcją sieci ARPANET. Obecnie dokumenty wydaje **IETF** (ang. Internet Engineering Task Force).

Protokoły SSL i TLS

Protokoły SSL i TLS realizują:

- Uwierzytelnianie,
- Szyfrowanie,
- Integralność.

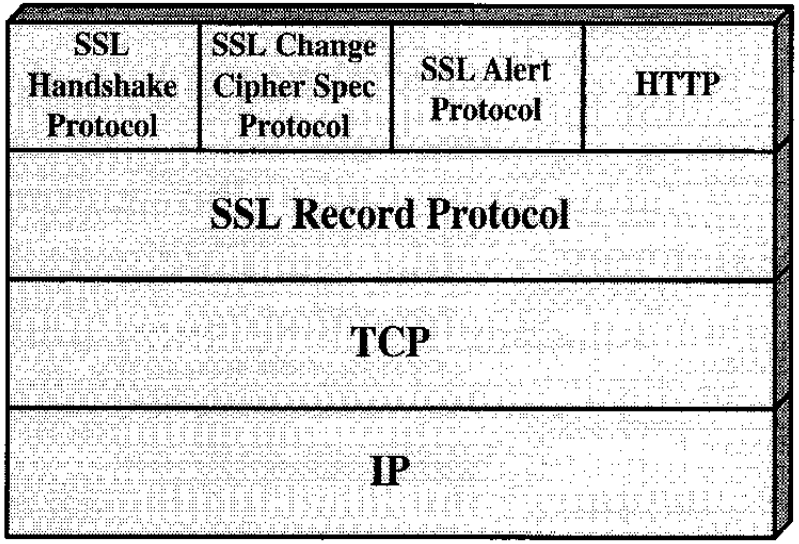
Oba protokoły współpracują z zaporami sieciowymi i połączeniami tunelowanymi np. opartymi o TCP. Używane są na etapie budowy VPN.

TLS cechuje **łatwość rozszerzania** o nowe elementy pierwotne kryptografii.

Protokoły SSL i TCP są niezależne od warstwy aplikacji i zabezpieczają wiele protokołów np. HTTP (HTTPS), Telnet, SMTP. POP, IMAP, FTP itp.

Biblioteki do obsługi SSL i TLS są ogólnie dostępne (np. OpenSSL).

Protokoły SSL i TLS



SSL i TLS formalnie należą do warstwy prezentacji (6) modelu OSI ale ich działanie jest inicjowane w warstwie sesji (5). Z punktu widzenia stosu protokołów TCP/IP, SSL i TLS leżą pomiędzy warstwą aplikacji zawierającą HTTP (HTTPS), Telnet, SMTP, POP, IMAP, FTP itd. a warstwą transportową, która zawiera TCP (warstwa fizyczna, np. Ethernet, xDSL itd. ---> warstwa Internet IP ---> warstwa transportowa TCP ---> warstwa aplikacji HTTP (HTTPS).

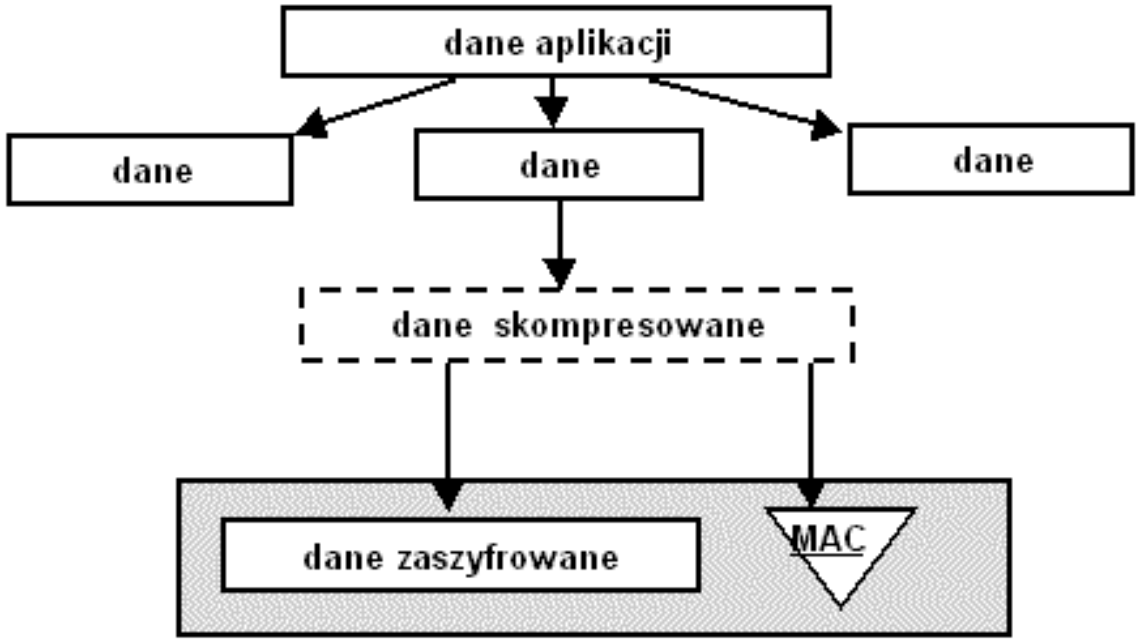
Protokoły SSL i TLS

Protokół SSL jest właściwie zestawem protokołów:

- ***Record Protocol*** - używany do bezpiecznego przesyłania wiadomości zorganizowane w postaci rekordów,
- ***Handshake Protocol*** - używany do negocjowania parametrów bezpiecznego połączenia np. metody szyfrowania i kluczy.
- ***Change Cipher Spec Protocol*** - używany do zmiany parametrów kryptograficznych (zmodyfikowany po ataku z 2009 roku),
- ***Alert Protocol*** - używany do powiadamiania o błędach.

Protokół rekordu służy do zapewnienia bezpieczeństwa i integralności danych i w tych celach jest stosowany przez "wyższe" protokoły wchodzące w skład SSL. Celem protokołu rekordu jest podzielenie danych przesyłanych przez aplikację, opakowanie ich w odpowiednie nagłówki i stworzenie obiektu zwanego właśnie rekordem, który zostaje zaszyfrowany i może zostać przekazany do przesłania poprzez protokół TCP.

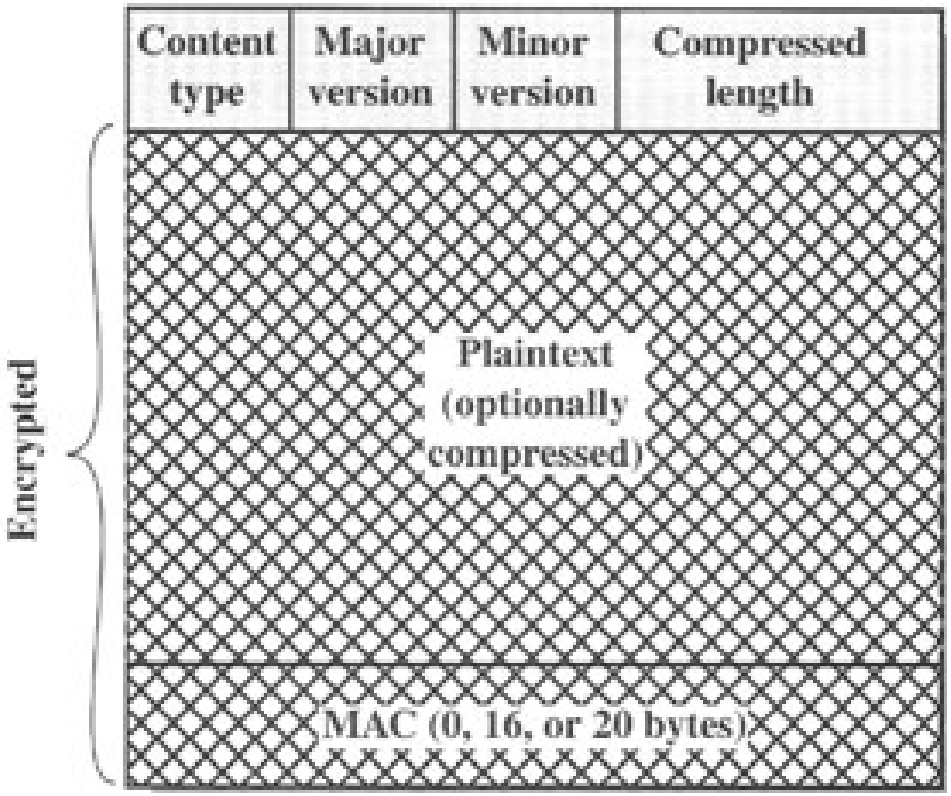
Protokoły SSL i TLS



Schemat procedury realizowanej przez *SSL Record Protocol*

Dane aplikacji są dzielone na bloki, a każdy blok jest szyfrowany i opatrzony indywidualnym kodem MAC (integralność danych). Rekord (zacienione pole) składa się z nagłówka, ewentualnych informacji uzupełniających do wymaganego rozmiaru danych oraz znacznika MAC. Zastosowane parametry kryptograficzne są ustalane na etapie inicjalizacji (Handshake Protocol).

Protokoły SSL i TLS



Postać Rekordu w *SSL*

Protokoły SSL i TLS

Protokół *ChangeCipher Spec* obecnie składa się tylko z jednego komunikatu, w którym przekazywana jest jedynie wartość 1. Celem przesłania tego komunikatu jest spowodowanie, aby ustanawiany właśnie stan sesji został uznany za ustalony. Komunikat taki musi zostać przesłany przez klienta do serwera oraz przez serwer do klienta. Po wymianie tych komunikatów stan sesji uznaje się za ustalony.

Protokół alarmu (*Alert Protocol)* służy do przesyłania komunikatów związanych z wymianą danych i działaniem protokołu. Każdy komunikat wysyłany przez protokół alarmu składa się **z dwóch bajtów**. Pierwszy wskazuje stopień ważności przesyłanego komunikatu. Drugi zawiera jeden ze zdefiniowanych kodów błędu, które mogą wystąpić w trakcie komunikacji SSL.

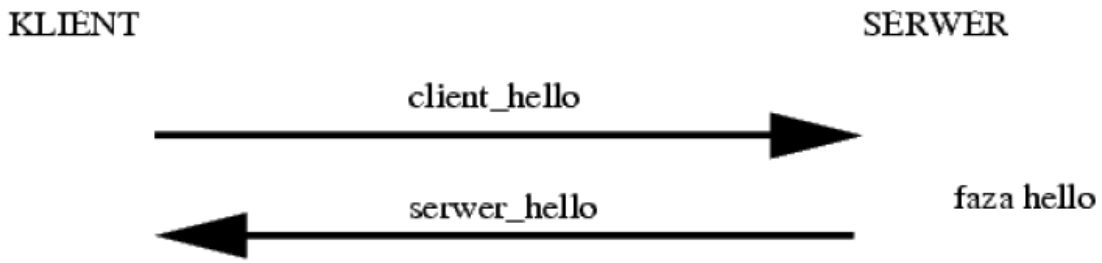
Protokół wymiany *(Handshake Protocol)* jest najbardziej skomplikowaną częścią protokołu SSL. Służy on do nawiązania sesji pomiędzy serwerem a klientem. Umożliwia ustalenie takich elementów, jak algorytmy i klucze używane do szyfrowania danych, funkcje skrótu itp. Umożliwia też uwierzytelnienie stron połączenia i wynegocjowanie odpowiednich parametrów sesji.

Protokoły SSL i TLS

Schemat działania Handshake Protocol (**S** – serwer, **K** – klient):

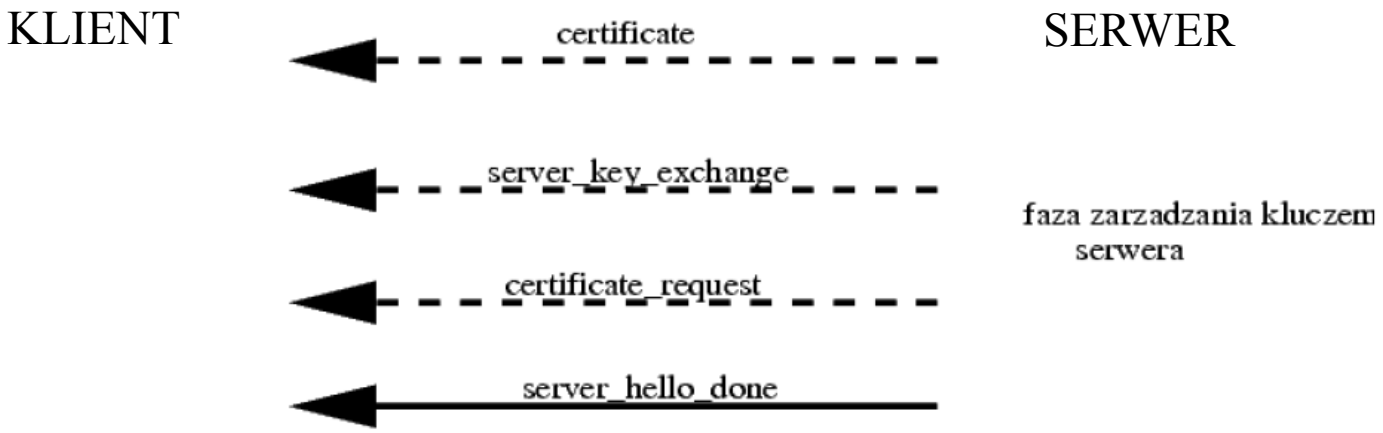
Faza **Hello**

- **S** czeka, np. na porcie TCP 443
- **K** → **S ClientHello.** Klient wysyła do serwera zgłoszenie zawierające m.in. obsługiwaną wersję protokołu SSL (najwyższy numer), dozwolone sposoby szyfrowania i kompresji danych oraz identyfikator sesji. Komunikat ten zawiera również liczbę losową używaną potem przy generowaniu kluczy.
- **K** ← **S ServerHello.** Serwer odpowiada podobnym komunikatem, w którym zwraca klientowi wybrane parametry połączenia: wersję protokołu SSL, rodzaj szyfrowania i kompresji oraz liczbę losową o podobnym rozmiarze.



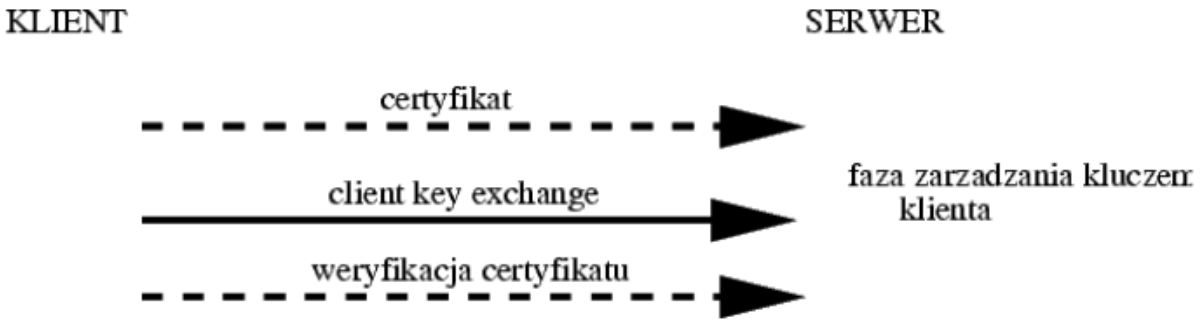
Faza **zarządzaniem kluczem serwera**

- **K ← S Certificate.** Serwer wysyła swój certyfikat pozwalając klientowi na sprawdzenie swojej tożsamości (ten etap jest opcjonalny, ale w większości przypadków występuje)
- **K ← S ServerKeyExchange.** Serwer wysyła informację o swoim kluczu publicznym. Rodzaj i długość tego klucza jest określony przez typ algorytmu przesłanego w poprzednim komunikacie.
- **K ← S CertificateRequest.** Serwer żąda przesłania certyfikatu klienta (opcja).
- **K ← S ServerHelloDone.** Serwer zawiadamia, że klient może przejść do następnej fazy zestawiania połączenia.



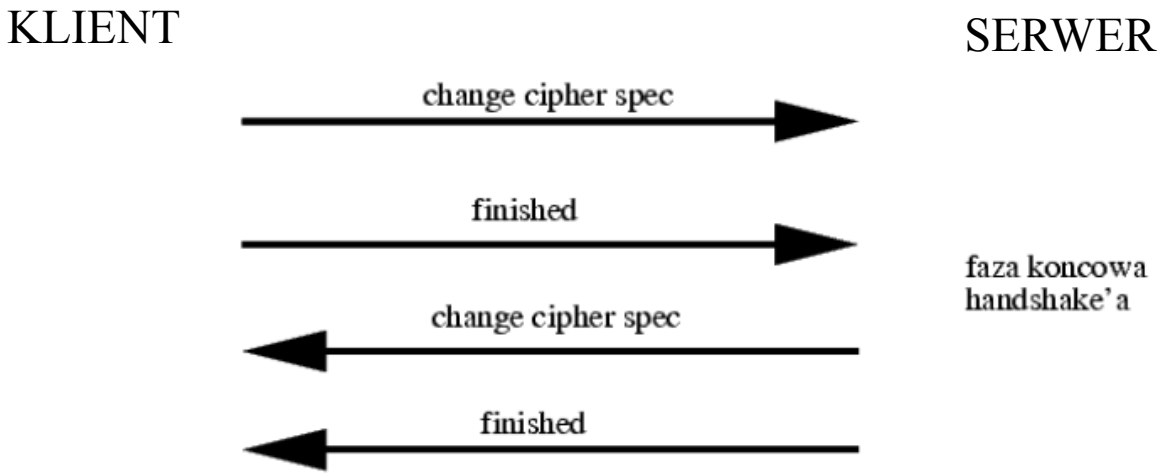
Faza **zarządzaniem kluczem klienta**

- **K → S ClientCertificate.** Ta wiadomość jest przesyłana, jeśli serwer zażądał od klienta przesłania certyfikatu. Jeśli certyfikat nie jest dostępny, klient musi przesłać wiadomość bez certyfikatu. Decyzja o ewentualnym przerwaniu komunikacji zależy od ustawień serwera.
- **K → S ClientKeyExchange.** Klient wysyła serwerowi wstępny klucz sesji, zaszyfrowany za pomocą klucza publicznego serwera. Na podstawie ustalonych w poprzednich komunikatach dwóch liczb losowych (klienta i serwera) oraz ustalonego przez klienta wstępnego klucza sesji obie strony generują klucz sesji używany do faktycznej wymiany danych. **Uwaga:** wygenerowany klucz jest kluczem algorytmu symetrycznego (typowo DES). Jest on jednak ustalony w sposób bezpieczny i znany jest tylko komunikującym się stronom.
- **K → S CertificateVerify.** Klient musi potwierdzić, że faktycznie posiada klucz prywatny odpowiadający wysłanemu certyfikatowi. W tym celu klient podpisuje swoim kluczem prywatnym skrót wszystkich dotychczas ustalonych danych o połączeniu i wysyła go korzystając z tego komunikatu.



Faza **końcowa**

- **K → S ChangeCipherSpec.** Klient zawiadamia, że serwer może przełączyć się na komunikację szyfrowaną.
- **K → S Finished.** ... oraz że jest gotowy do odbierania danych zaszyfrowanych.
- **K ← S ChangeCipherSpec.** Serwer zawiadamia, że wykonał polecenie – od tej pory wysyłał będzie tylko zaszyfrowane informacje...
- **K ← S Finished.** ...i przechodzi do sprawdzenia transmisji – ten komunikat jest już wysyłany bezpiecznym kanałem.



Protokoły SSL i TLS

Dostępne algorytmy:

- Wymiana kluczy: RSA lub różne wersje algorytmu Diffiego-Hellmana.
- Szyfrowanie asymetryczne: RSA
- Szyfrowanie symetryczne: RC2, RC4 (strumieniowy), RC5, DES, 3DES, IDEA, CAST, AES, Blowfish.
- Funkcje skrótu: MD5, SHA-1, SHA-2, HMAC.

Powyższy zestaw może ulegać modyfikacjom, zależnie od implementacji SSL (TLS).

Implementacja SSL (TLS):

Warunkiem koniecznym posługiwania się SSL (TLS) jest dysponowanie certyfikatem SSL (TLS). Certyfikat (standard X.509) pobierany jest z jednego z wielu centrów certyfikacyjnych (koszt od ok. 200 zł do ok. 1500 zł) i jest podpisany tajnym kluczem wybranego centrum. Część centrów oferuje możliwość bezpłatnego przetestowania certyfikatu np. Centrum Certyfikacji „Centrum" (http://www.centrum.pl). Warunki uzyskania oraz szczegóły postępowania zawiera instrukcja zawarta na stronie wybranego centrum certyfikacji.

Protokoły SSL i TLS

Sprawdzenie certyfikatu w SSL:

Po otrzymaniu komunikatu **ServerHelloDone** (ostatni komunikat fazy zarządzania kluczem serwera) klient powinien zweryfikować certyfikat serwera, jego ważność i ścieżkę certyfikacji, jak również wszelkie inne parametry przesłane przez serwer w komunikacie **ServerHelloDone**. Potwierdzenie autentyczności i poprawności certyfikatu serwera po stronie klienta polega na:

- Sprawdzeniu daty ważności certyfikatu.
- Sprawdzeniu, czy urząd certyfikacyjny, który wystawił certyfikat serwera, znajduje się na liście zaufanych urzędów certyfikacji po stronie klienta. Jeżeli CA, który wystawił certyfikat SSL serwera nie znajduje się na tej liście, klient rozpoczyna procedurę weryfikacji CA, który jest wystawcą tego certyfikatu. Jeżeli informacje o tym urzędzie **nie mogą być uzyskane, klient przerwie procedurę identyfikacji, zwracając błąd lub zwróci się o rozstrzygnięcie tego problemu do użytkownika**.

Protokoły SSL i TLS

Sprawdzenie certyfikatu w SSL cd.:

- Jeśli CA, który wydał certyfikat znajduje się na liście zaufanych CA po stronie klienta, to porównywany jest klucz publiczny CA zawarty w certyfikacie serwera z kluczem publicznym udostępnionym na tej liście.
- Klient sprawdza, czy nazwa domeny wymieniona w certyfikacie odpowiada nazwie serwera, który przedstawił się tym certyfikatem.
- Po prawidłowym przejściu wszystkich wymienionych kroków, serwer uznawany jest za uwierzytelniony. Wówczas klient odpowiada serwerowi jednym lub wieloma komunikatami. Jedynym koniecznym komunikatem jest przesłanie do serwera kluczy klienta w komunikacie **ClientKeyEexchange**, którego zawartość zależna jest od wynegocjowanej metody wymiany kluczy. Dodatkowo, jeżeli serwer tego zażądał, przesyłany jest certyfikat klienta i komunikat umożliwiający weryfikację tego certyfikatu. Wysłanie tych komunikatów kończy fazę zarządzania kluczem klienta (faza trzecia).

Protokół SSH

Jedną z najbardziej ryzykownych czynności wykonywanych przez użytkowników sieci komputerowych jest **logowanie się do odległego komputera poprzez publiczną sieć komputerową**. Przeciwnik może tutaj zastosować atak typu *man-in-the middle* kierując pakiety poprzez własny węzeł komunikacyjny celem przechwycenia np. hasła czy modyfikacji pakietu. Mechanizmy ochrony Internetu przy takim logowaniu nie dostarczają tutaj wystarczającej ochrony.

Pakiet **SSH** (*ang*. Secure Shell) **został pomyślany jako środek umożliwiający bezpieczne logowanie poprzez sieć nawet wtedy gdy przeciwnik ma nieograniczone możliwości manipulowania przepływającymi w sieci pakietami.** Autorem SSH jest Tatu Ylönen z uniwersytetu w Helsinkach. Pakiet SSH dba nie tylko o uwierzytelnienie użytkownika ale także o uwierzytelnienie komputerów biorących udział w protokole. Oferuje także działania zmierzające do zminimalizowania ewentualnych szkód powstałych w przypadku włamania się do maszyn biorących udział w obsłudze protokołu.

Protokół SSH

SSH jest następcą protokołu **Telnet**, służącego do połączeń terminalowych ze zdalnym urządzeniem. Główną różnicą pomiędzy dwoma protokołami jest to, iż w protokole SSH dane są szyfrowane oraz istniej identyfikacja użytkownika. Obecnie w użyciu są dwie wersje tj. SSH 1 oraz SSH 2. SSH 2 różni się od SSH 1 tym, iż daje możliwość użycia dowolnych sposobów szyfrowania danych oraz różnych sposobów uwierzytelniania. SSH obsługuje tylko stałą listę sposobów szyfrowania oraz dwóch sposobów uwierzytelniania (klucz RSA oraz zwykłe hasło). Na bazie SSH powstało wiele protokołów służących bezpiecznemu zdalnemu logowaniu (np. SCP, SFTP).

Podczas nawiązywania połączenia poprzez protokół SSH pomiędzy klientem, a serwerem występują cztery etapy (błąd któregokolwiek z nich skutkuje zerwaniem połączenia). **Każdy komputer w momencie instalacji pakietu otrzymuje parę kluczy algorytmu asymetrycznego: jeden z nich jest dostępny do czytania, tzw. *host key,* a drugi jest starannie chroniony.**

Protokół SSH

Działanie protokołu (w uproszczeniu):

- **Uwierzytelnianie serwera** – W momencie uruchomienia demona sshd (proces w systemie operacyjnym odpowiedzialny za obsługę ssh) zdalnego komputera tworzona jest dodatkowa para kluczy algorytmu asymetrycznego, tzw. *server keys.* Jeden z nich jest dostępny do czytania, a drugi jest przechowywany w pamięci komputera i *nie jest zapisywany na dysku.* Para tych kluczy jest często zmieniana przez demona sshd. Oba klucze generowane są za pomocą algorytmu RSA. Klucz *host key* zostaje wysłany wraz z kluczem serwera do komputera klienta natychmiast po nawiązaniu połączenia. Komputer klienta porównuje otrzymany *host key z* kluczem, na którym pracował uprzednio (gdy pierwsze logowanie nie jest zdalne).
- **Ustawianie bezpiecznego połączenia** – klient generuje liczbę losową o długości 256 bitów (klucz sesyjny), dokonuje szyfrowania kluczami, które otrzymał od serwera po nawiązaniu połączenia. Zaszyfrowana liczba pełni rolę klucza dla symetrycznego algorytmu szyfrowania danych przesyłanych zdalnie. Wersja SSH 1 ma do wyboru algorytmy: DES, 3DES, Blowfish, IDEA, RC4. Wersja SSH 2 obsługuje algorytmy: Blowfish, Twofish, IDEA, AES, Serpent, CAST, 3DES.

Protokół SSH

Działanie protokołu cd:

- **Uwierzytelnianie klienta** – może być wykonane na kilka różnych sposobów. Jedną z częściej stosowanych metod jest wykorzystanie pliku identyfikującego klienta oraz klucza publicznego RSA. Klucz publiczny klienta zostaje wysłany do serwera gdzie jest weryfikowany. Serwer porównuje pliki identyfikujące zawierające nazwę komputera oraz identyfikator ze swoimi danymi na temat klienta. Inne sposoby ograniczają się do stosowania tylko jednej metody: weryfikacji klucza RSA klienta oraz jego plików identyfikacyjnych. Możliwy jest też sposób uwierzytelniania klienta na podstawie **hasła** przyporządkowanego do danego użytkownika.
- **Logowanie klienta** – po poprawnym uwierzytelnianiu klienta następuje jego zalogowanie na serwerze.

KONIEC CZĘŚCI III